

Curso Profesional de JasperReports 6.20.0 Community

Módulo 3 — Conexión a datos

PRÁCTICAS

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Maven · SQLite · CSV · XML · JSON · Proyecto EditorialReports

Curso Profesional de JasperReports 6.20.0 Community

Módulo 3 - Conexión a datos - PRÁCTICAS

Proyecto: EditorialReports

Autor: JAIME GALLO

Baseline: Jaspersoft Studio 6.20.0 Community + JasperReports Library 6.20.0 + Java 8 + Maven

| Puntos incluidos

- 3.1 Bases de datos y JDBC
- 3.2 Ficheros CSV
- 3.3 Ficheros XML
- 3.4 Ficheros JSON
- 3.5 Consultas SQL
- 3.6 Fields

| Modelo pedagógico

Cada punto conserva el patrón A/B/C/D: **A** construcción visual en Jaspersoft Studio; **B** JRXML que representa el estado construido; **C** Java que compila, llena y exporta; **D** verificación del PDF y del árbol acumulativo. Los checkpoints M3/3.1 a M3/3.6 son soluciones acumulativas.

| Directorio de trabajo reproducible

Las ejecuciones Java se lanzan con EditorialReports como Working Directory. Por ello `reports/`, `data/` y `output/` son rutas locales al proyecto de informes, mientras que SQLite se referencia como `jdbc:sqlite:../EditorialReportsJava/data/editorial.db`. Maven resuelve el runtime completo desde `EditorialReportsJava/pom.xml`; la carpeta `lib` se documenta para el trabajo visual/manual, pero no sustituye la resolución reproducible de dependencias.

Punto 3.1 — Bases de datos y JDBC

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial

Punto: 3.1 — Bases de datos y JDBC

Objetivos de aprendizaje

- Comprender el papel de JDBC como interfaz de conexión entre JasperReports y las bases de datos.
- Instalar y configurar el driver JDBC de SQLite en el proyecto Java y en Jaspersoft Studio.
- Crear una base de datos SQLite con la tabla de libros del proyecto EditorialReports.
- Configurar un adaptador JDBC en Jaspersoft Studio y asociarlo al informe.
- Declarar una consulta SQL en el JRXML y verificar su ejecución desde la previsualización.
- Ejecutar el informe desde código Java con una conexión JDBC.
- Documentar la conexión a la base de datos en el proyecto.

Parte práctica

Parte A — Práctica visual

Paso 1: Crear la carpeta data en el proyecto Java

Acciones:

1. Hacer clic con el botón derecho sobre el nodo EditorialReportsJava en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Folder en el submenú.
4. Escribir exactamente data en el campo Folder name del diálogo.
5. Hacer clic sobre el botón Finish.

Verificación visual: el panel Project Explorer muestra la carpeta data al mismo nivel que lib y src dentro de EditorialReportsJava.

Qué hace: crea la carpeta que alojará la base de datos SQLite.

Por qué: la base de datos debe residir en una carpeta separada del código fuente y de las librerías.

Error común: crear la carpeta dentro de src en lugar de en la raíz del proyecto. La ruta de la base de datos en la URL JDBC no coincidiría.

Solución: eliminar la carpeta y crearla en la raíz del proyecto.

Analogía: es como habilitar una sala de archivo en la editorial para guardar la base de datos de los libros.

Paso 2: Copiar el driver JDBC de SQLite a la carpeta lib

Acciones:

1. Abrir el explorador de archivos del sistema operativo.
2. Navegar hasta la carpeta de recursos del curso donde se encuentra el archivo `sqlite-jdbc-3.44.0.0.jar`.
3. Hacer clic con el botón derecho sobre el archivo `sqlite-jdbc-3.44.0.0.jar` y seleccionar Copiar.
4. Volver a Jaspersoft Studio y hacer clic con el botón derecho sobre la carpeta lib en el panel Project Explorer.
5. Hacer clic sobre la opción Paste en el menú contextual.
6. Hacer clic con el botón derecho sobre el nodo EditorialReportsJava en el panel Project Explorer.
7. Hacer clic sobre la opción Refresh en el menú contextual.

Verificación visual: la carpeta `lib` del panel Project Explorer muestra el archivo `sqlite-jdbc-3.44.0.0.jar` en la carpeta local `lib` si se sigue el ejercicio manual; Maven resolverá la misma dependencia para las pruebas automatizadas.

Qué hace: incorpora el driver JDBC de SQLite a la carpeta de librerías del proyecto.

Por qué: el driver debe estar en el classpath para que el `DriverManager` lo localice al establecer la conexión.

Error común: copiar el archivo con un nombre distinto a `sqlite-jdbc-3.44.0.0.jar`. El driver no se registra con el nombre esperado.

Solución: conservar el nombre original del archivo.

Analogía: es como añadir a la imprenta la herramienta específica para leer la base de datos de los libros.

Paso 3: Añadir el driver JDBC al Build Path

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `EditorialReportsJava` en el panel Project Explorer.
2. Hacer clic sobre la opción `Properties` en el menú contextual.
3. Hacer clic sobre la categoría `Java Build Path` en el panel izquierdo del diálogo.
4. Hacer clic sobre la pestaña `Libraries` en el panel derecho.
5. Hacer clic sobre el botón `Add JARs....`
6. Expandir el nodo `EditorialReportsJava` y hacer clic sobre la carpeta `lib`.
7. Marcar la casilla del archivo `sqlite-jdbc-3.44.0.0.jar`.
8. Hacer clic sobre el botón `OK`.
9. Hacer clic sobre el botón `Apply and Close`.

Verificación visual: el panel Project Explorer muestra el archivo `sqlite-jdbc-3.44.0.0.jar` con un icono de librería referenciada.

Qué hace: registra el driver JDBC en el classpath del proyecto Java.

Por qué: sin esta acción, el `DriverManager` no encuentra el driver y lanza `No suitable driver found`.

Error común: añadir el JAR como External JAR con ruta absoluta. El proyecto deja de ser portable.

Solución: eliminarlo y añadirlo con `Add JARs` desde la carpeta `lib`.

Analogía: es como registrar en el catálogo de la imprenta la nueva herramienta para que los operarios sepan que está disponible.

Paso 4: Crear la clase InicializadorBD

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta `src` en el panel Project Explorer.
2. Hacer clic sobre la opción `New` en el menú contextual.
3. Hacer clic sobre la opción `Class` en el submenú.
4. Escribir exactamente `InicializadorBD` en el campo `Name` del diálogo.
5. Hacer clic sobre el botón `Finish`.
6. En el editor central, escribir el código completo de la clase `InicializadorBD` que se muestra en la Parte C de este punto.
7. Pulsar `Ctrl+S` para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `InicializadorBD.java` dentro de la carpeta `src`. El editor central muestra el código sin subrayados rojos.

Qué hace: crea la clase que inicializa la base de datos con la tabla de libros y los datos de ejemplo.

Por qué: la base de datos debe crearse antes de que el informe intente consultarla.

Error común: olvidar importar `java.sql.Connection` y `java.sql.Statement`. El compilador informa `cannot find symbol`.

Solución: revisar las importaciones y añadir las que falten.

Analogía: es como preparar el archivador de la editorial con las fichas de todos los libros antes de generar el catálogo.

Paso 5: Ejecutar la clase InicializadorBD

Acciones:

1. Hacer clic con el botón derecho sobre el archivo `InicializadorBD.java` en el panel Project Explorer.
2. Hacer clic sobre la opción Run As en el menú contextual.
3. Hacer clic sobre la opción Java Application en el submenú.
4. Hacer clic sobre la vista Console en el panel inferior y observar el resultado.

Verificación visual: la vista Console muestra la línea Base de datos inicializada correctamente en: El panel Project Explorer muestra el archivo `editorial.db` en la carpeta `data`.

Qué hace: ejecuta la clase que crea la base de datos con la tabla y los datos.

Por qué: la base de datos debe existir antes de configurar el adaptador JDBC.

Error común: ejecutar la clase desde un directorio distinto a la raíz del proyecto y obtener `SQLException: path to 'data/editorial.db': 'data' does not exist`.

Solución: comprobar en Run Configurations que el Working Directory apunta a la raíz del proyecto `EditorialReports`.

Analogía: es como imprimir las fichas de los libros y colocarlas en el archivador antes de componer el catálogo.

Paso 6: Verificar la creación de la base de datos

Acciones:

1. Abrir el explorador de archivos del sistema operativo.
2. Navegar hasta la carpeta `Documents\JasperProjects\EditorialReportsJava\data`.
3. Verificar que el archivo `editorial.db` existe.
4. Hacer clic con el botón derecho sobre el archivo `editorial.db` y seleccionar Properties.
5. Observar el tamaño del archivo en la pestaña General.
6. Hacer clic sobre el botón Close.

Verificación visual: el archivo `editorial.db` existe en la carpeta `data` con un tamaño superior a 0 bytes.

Qué hace: confirma que la base de datos se ha creado correctamente en el sistema de archivos.

Por qué: la base de datos es el origen de datos del informe y su existencia es condición necesaria.

Error común: encontrar el archivo en una ubicación distinta a la carpeta `data`.

Solución: comprobar la URL JDBC en el código de la clase `InicializadorBD` y asegurarse de que la ruta es `data/editorial.db`.

Analogía: es como comprobar que el archivador de la editorial contiene las fichas de los libros.

Paso 7: Crear el adaptador JDBC en Jaspersoft Studio

Acciones:

1. Hacer clic con el botón derecho sobre el nodo Data Adapters en el panel Repository Explorer (lateral izquierdo).
2. Hacer clic sobre la opción Create Data Adapter en el menú contextual.
3. Hacer clic sobre el elemento Database JDBC Connection en la lista de categorías del asistente.
4. Hacer clic sobre el botón Next.
5. Escribir exactamente SQLiteEditorial en el campo Name.
6. Hacer clic sobre el campo Driver Classpath y hacer clic sobre el botón Add....
7. Navegar hasta la carpeta Documents\JasperProjects\EditorialReportsJava\lib y seleccionar el archivo sqlite-jdbc-3.44.0.0.jar.
8. Hacer clic sobre el botón Open.
9. Hacer clic sobre el campo JDBC Driver y seleccionar org.sqlite.JDBC en el desplegable.
10. Hacer clic sobre el campo JDBC URL y escribir exactamente jdbc:sqlite:../EditorialReportsJava/data/editorial.db.
11. Hacer clic sobre el botón Test Connection.
12. Verificar que el diálogo muestra el mensaje Connection successful.
13. Hacer clic sobre el botón Finish.

Verificación visual: el panel Repository Explorer muestra el nodo SQLiteEditorial colgando de Data Adapters con un icono de base de datos.

Qué hace: registra un adaptador JDBC que Jaspersoft Studio utiliza para conectarse a la base de datos durante la previsualización.

Por qué: el adaptador permite ejecutar la consulta SQL del informe con datos reales desde el entorno de diseño.

Error común: olvidar añadir el driver al Classpath del adaptador y obtener el error `No suitable driver found for jdbc:sqlite:...`

Solución: añadir el JAR con el botón Add... del campo Driver Classpath.

Analogía: es como registrar en la editorial la conexión al archivador de libros para poder consultarlo durante la composición.

Paso 8: Declarar la consulta SQL en el JRXML**Acciones:**

1. Hacer doble clic sobre el archivo informe_concepto.jrxml en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
3. Localizar la línea que contiene `<field name="disponible" class="java.lang.Boolean"/>`.
4. Hacer clic al final de esa línea y pulsar Enter.
5. Escribir exactamente `<queryString language="sql">` y pulsar Enter.
6. Escribir exactamente `<![CDATA[SELECT titulo, precio, paginas, fecha_publicacion AS fechaPublicacion, CASE WHEN disponible=1 THEN 1 ELSE 0 END AS disponible FROM libros ORDER BY titulo]]>` y pulsar Enter.
7. Escribir exactamente `</queryString>` y pulsar Enter.
8. Pulsar Ctrl+S para guardar el archivo.
9. Hacer clic sobre la pestaña Design en la parte inferior del editor central.

Verificación visual: el editor central muestra el informe sin errores. En la vista Source, la consulta SQL aparece después de la declaración de campos y antes de las bandas.

Qué hace: declara la consulta SQL que el motor ejecutará contra la base de datos.

Por qué: la consulta define los registros que alimentan el informe.

Error común: olvidar el bloque CDATA y provocar un error de análisis XML porque la consulta contiene caracteres especiales como `>` o `ORDER BY`.

Solución: encerrar la consulta en `<![CDATA[...]]>`.

Analogía: es como escribir la consulta que el archivero debe ejecutar para recuperar las fichas de los libros.

Paso 9: Alinear el campo fechaPublicacion con SQLite

Acciones:

1. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
2. Localizar la declaración `<field name="fechaPublicacion" class="java.util.Date"/>` heredada del Módulo 2.
3. Sustituirla por `<field name="fechaPublicacion" class="java.lang.String"/>`.
4. Verificar que la consulta SQL usa el alias `fecha_publicacion AS fechaPublicacion`.
5. Localizar el Text Field del año y sustituir la expresión de fecha por `$F{fechaPublicacion}.substring(0,4)`.
6. Eliminar el atributo `pattern="yyyy"` de ese Text Field, porque el campo ya no es un Date.
7. Pulsar Ctrl+S y volver a Design.
8. Expandir Fields y verificar que siguen apareciendo los cinco campos del informe.

Verificación visual: el panel Outline muestra cinco campos y el año sigue apareciendo en Preview.

Qué hace: adapta el contrato del campo a la forma en que SQLite almacena `fecha_publicacion` en este curso: texto ISO `yyyy-MM-dd`.

Por qué: evita depender de una conversión implícita de `TEXT` a `java.util.Date` por parte del driver JDBC.

Error común: mantener `java.util.Date` y asumir que el driver convertirá siempre el texto ISO.

Solución: usar `String` y extraer el año explícitamente.

Analogía: es como respetar el formato real de la ficha del archivo en lugar de fingir que ya llega convertido.

Paso 10: Asociar el adaptador JDBC al informe

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `informe_concepto` en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la opción Properties en el menú contextual.
3. Hacer clic sobre la pestaña Dataset en el panel Properties (inferior derecho).
4. Hacer clic sobre el desplegable Data Adapter y seleccionar `SQLiteEditorial`.
5. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
6. Verificar que la línea `<property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>` está presente.
7. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Properties muestra el adaptador `SQLiteEditorial` seleccionado. La vista Source muestra la propiedad con el valor `SQLiteEditorial`.

Qué hace: asocia el adaptador JDBC al informe para que el botón Preview ejecute la consulta contra la base de datos.

Por qué: la asociación evita tener que seleccionar el adaptador en cada previsualización.

Error común: olvidar el paso 6 y provocar que el botón Preview solicite el adaptador en cada ejecución.

Solución: verificar que la propiedad está presente en el JRXML.

Analogía: es como anotar en el pliego del catálogo qué archivador debe utilizarse para recuperar las fichas.

Paso 11: Compilar y previsualizar con el adaptador JDBC

Acciones:

1. Pulsar Ctrl+Mayús+B para compilar el informe.
2. Hacer clic sobre el panel Problems y verificar que no hay errores.
3. Pulsar el botón Preview de la barra de herramientas superior.

4. En el diálogo, verificar que el adaptador `SQLiteEditorial` está seleccionado.
5. Hacer clic sobre el botón OK.
6. Esperar a que se abra la pestaña Preview en el editor central.

Verificación visual: la pestaña Preview muestra el informe con los catorce libros recuperados de la base de datos.

Qué hace: ejecuta la consulta SQL contra la base de datos y muestra el resultado en la vista previa.

Por qué: la previsualización confirma que la conexión JDBC funciona y que la consulta devuelve los registros esperados.

Error común: obtener `SQLException: no such table: libros`. Indica que la base de datos no tiene la tabla.

Solución: ejecutar de nuevo la clase `InicializadorBD`.

Analogía: es como revisar la prueba de color del catálogo con las fichas recuperadas del archivador.

Paso 12: Modificar el programa Java para usar la conexión JDBC

Acciones:

1. Hacer doble clic sobre el archivo `GeneradorInformeConcepto.java` en el panel Project Explorer.
2. Hacer clic sobre la línea que contiene `import java.util.HashMap`; y pulsar Enter al final.
3. Escribir exactamente `import java.sql.Connection`; y pulsar Enter.
4. Escribir exactamente `import java.sql.DriverManager`; y pulsar Enter.
5. Hacer clic sobre la línea que contiene `JasperPrint documento = JasperFillManager.fillReport(`.
6. Seleccionar el bloque completo desde esa línea hasta `new CatalogoDataSource(Libro.listaEjemplo());` y eliminar con la tecla Suprimir.
7. Escribir exactamente `try (Connection conexion = DriverManager.getConnection("jdbc:sqlite:../EditorialReportsJava/data/editorial.db"))` { y pulsar Enter.
8. Escribir exactamente `JasperPrint documento = JasperFillManager.fillReport(` y pulsar Enter.
9. Escribir exactamente `rutaJasper`, y pulsar Enter.
10. Escribir exactamente `parametros`, y pulsar Enter.
11. Escribir exactamente `conexion)`; y pulsar Enter.
12. Escribir exactamente `JasperExportManager.exportReportToPdfFile(documento, rutaPdf)`; y pulsar Enter.
13. Escribir exactamente `System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());` y pulsar Enter.
14. Escribir exactamente `System.out.println("Paginas del documento: " + documento.getPages().size());` y pulsar Enter.
15. Escribir exactamente `}` y pulsar Enter.
16. Pulsar Ctrl+S para guardar el archivo.
17. Observar el panel Problems y verificar que no hay errores.

Verificación visual: el editor central muestra la clase con el bloque `try-with-resources` que abre la conexión JDBC. El panel Problems permanece vacío.

Qué hace: modifica el programa para que utilice la conexión JDBC en lugar de la fuente de datos personalizada.

Por qué: el informe ahora obtiene los datos de la base de datos SQLite en lugar de la lista de ejemplo.

Error común: olvidar el cierre `}` del bloque `try-with-resources`. El compilador informa `Syntax error, insert "}" to complete Block`.

Solución: revisar la estructura del bloque y añadir el cierre.

Analogía: es como sustituir la bandeja de fichas de ejemplo por la consulta directa al archivador de libros.

Paso 13: Ejecutar el programa y verificar el PDF

Acciones:

1. Hacer clic con el botón derecho sobre el archivo `GeneradorInformeConcepto.java` en el panel Project Explorer.
2. Hacer clic sobre la opción Run As en el menú contextual.
3. Hacer clic sobre la opción Java Application en el submenú.
4. Hacer clic sobre la vista Console en el panel inferior y observar el resultado.
5. Abrir el explorador de archivos del sistema operativo.
6. Navegar hasta la carpeta output del proyecto EditorialReports.
7. Hacer doble clic sobre el archivo `informe_concepto.pdf`.
8. Verificar que el PDF muestra los catorce libros recuperados de la base de datos.

Verificación visual: la vista Console muestra la línea `Informe generado en: ...` con la ruta absoluta del PDF. El archivo PDF muestra los catorce libros.

Qué hace: ejecuta el programa Java con la conexión JDBC y genera el PDF.

Por qué: la ejecución confirma que el programa obtiene los datos de la base de datos y genera el informe correctamente.

Error común: ejecutar el programa desde un directorio distinto a la raíz del proyecto y obtener `SQLException: path to 'data/editorial.db': 'data' does not exist`.

Solución: comprobar en Run Configurations que el Working Directory apunta a la raíz del proyecto EditorialReports.

Analogía: es como imprimir la tirada del catálogo con las fichas recuperadas del archivador.

Paso 14: Documentar la conexión a la base de datos**Acciones:**

1. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción File en el submenú.
4. Escribir exactamente `BASEDATOS.md` en el campo File name del diálogo.
5. Hacer clic sobre el botón Finish.
6. En el editor central, escribir exactamente `# Conexión a base de datos` y pulsar Enter dos veces.
7. Escribir exactamente `## Motor` y pulsar Enter dos veces.
8. Escribir exactamente `- SQLite 3.44.0 (driver Xerial 3.44.0.0)` y pulsar Enter.
9. Escribir exactamente ``- Driver local opcional: sqlite-jdbc-3.44.0.0.jar``
 - Runtime reproducible: `org.xerial:sqlite-jdbc:3.44.0.0`` y pulsar Enter dos veces.
1. Escribir exactamente `## URL de conexión` y pulsar Enter dos veces.
2. Escribir exactamente `- jdbc:sqlite:../EditorialReportsJava/data/editorial.db` y pulsar Enter dos veces.
3. Escribir exactamente `## Adaptador de Jaspersoft Studio` y pulsar Enter dos veces.
4. Escribir exactamente `- Nombre: SQLiteEditorial` y pulsar Enter.
5. Escribir exactamente `- Driver Class: org.sqlite.JDBC` y pulsar Enter dos veces.
6. Escribir exactamente `## Tabla principal` y pulsar Enter dos veces.
7. Escribir exactamente `- libros (titulo, precio, paginas, fecha_publicacion, disponible)` y pulsar Enter.
8. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `BASEDATOS.md` en la raíz del proyecto EditorialReports con la documentación de la conexión.

Qué hace: incorpora al proyecto un documento que registra la configuración de la conexión a la base de datos.

Por qué: la documentación de la conexión facilita el mantenimiento y la reproducción del entorno en otro equipo.

Error común: olvidar documentar la URL de conexión.

Solución: incluir todas las secciones especificadas.

Analogía: es como dejar en la editorial una ficha técnica con la ubicación del archivador y su método de consulta.

Parte B — JRXML completo explicado línea por línea [VALIDADO]

Se reproduce la sección de contrato de datos modificada respecto a M2/2.6. El resto de bandas conserva el informe conceptual.

```
<property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
<queryString language="sql"><![CDATA[SELECT titulo, precio, paginas, fecha_publicacion AS fechaPublicacion, CASE WHEN disponible=1 THEN 1 ELSE 0 END AS disponible FROM libros ORDER BY titulo]]></queryString>
<field name="titulo" class="java.lang.String"/>
<field name="precio" class="java.lang.Double"/>
<field name="paginas" class="java.lang.Integer"/>
<field name="fechaPublicacion" class="java.lang.String"/>
<field name="disponible" class="java.lang.Boolean"/>
<variable name="TotalPrecios" class="java.lang.Double" calculation="Sum">
  <variableExpression><![CDATA[$F{precio}]]></variableExpression>
</variable>
```

Explicación línea por línea

Línea 1	<property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/> → Asocia el Data Adapter usado por Jaspersoft Studio durante Preview.
Línea 2	<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/> → Declara un estilo compatible con JasperReports 6.20.0.
Línea 3	<queryString language="sql"><![CDATA[SELECT titulo, precio, paginas, fecha_publicacion AS fechaPublicacion, CASE WHEN disponible=1 THEN 1 ELSE 0 END AS disponible FROM libros ORDER BY titulo]]></queryString> → Abre la consulta del dataset e indica el lenguaje de consulta.
Línea 4	<field name="titulo" class="java.lang.String"/> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 5	<field name="precio" class="java.lang.Double"/> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 6	<field name="paginas" class="java.lang.Integer"/> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 7	<field name="fechaPublicacion" class="java.lang.String"/> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 8	<field name="disponible" class="java.lang.Boolean"/> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 9	<variable name="TotalPrecios" class="java.lang.Double" calculation="Sum"> → Declara una variable calculada del informe.
Línea 10	<variableExpression><![CDATA[\$F{precio}]]></variableExpression> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 11	</variable> → Cierra el elemento XML abierto anteriormente.

Comprobación: las coordenadas se mantienen dentro de `columnWidth="555"`, el orden estructural es compatible con JasperReports 6.20.0 y no se usa sintaxis retirada de la baseline.

Parte C — Código Java explicado línea por línea [VALIDADO]

El código siguiente es el código real incluido en el checkpoint y ejecutado por el workflow E2E. Java no redibuja el informe: **compila -> llena -> exporta**.

Clase InicializadorBD.java

```
import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
```

```

public class InicializadorBD {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
        try {
            new File("../EditorialReportsJava/data").mkdirs();
            Class.forName("org.sqlite.JDBC");
            try (Connection conexion = DriverManager.getConnection(url);
                Statement sentencia = conexion.createStatement()) {
                sentencia.executeUpdate("DROP TABLE IF EXISTS libros");
                sentencia.executeUpdate("CREATE TABLE libros ( " +
                    "titulo TEXT PRIMARY KEY, " +
                    "precio REAL NOT NULL, " +
                    "paginas INTEGER NOT NULL, " +
                    "fecha_publicacion TEXT NOT NULL, " +
                    "disponible INTEGER NOT NULL)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('Cien años de soledad', 19.95, 471, '1967-06-05', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('Rayuela', 22.50, 736, '1963-06-28', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('La ciudad y los perros', 18.75, 432, '1963-10-15', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('Pedro Páramo', 15.90, 132, '1955-03-01', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('Ficciones', 21.00, 224, '1944-12-01', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('La casa de los espíritus', 23.40, 448, '1982-01-01', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('El amor en los tiempos del cólera', 20.80, 496, '1985-09-05', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('La muerte de Artemio Cruz', 17.60, 320, '1962-05-01', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('Doña Bárbara', 16.95, 400, '1929-02-01', 0)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('Martín Fierro', 14.50, 288, '1872-12-01', 0)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('Comala', 19.20, 148, '1955-09-01', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('Paradiso', 25.00, 576, '1966-01-01', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('La invención de Morel', 18.30, 128, '1940-01-01', 1)");
                sentencia.executeUpdate("INSERT INTO libros VALUES ('El túnel', 16.20, 160, '1948-01-01', 0)");

            }
            System.out.println("Base de datos inicializada correctamente en: " + url);
            System.out.println("Libros insertados: 14");

        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}

```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 2	<code>import java.sql.Connection;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 3	<code>import java.sql.DriverManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 4	<code>import java.sql.Statement;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 5	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 6	<code>public class InicializadorBD {</code> → Declara la clase Java del checkpoint.
Línea 7	<code>public static void main(String[] args) {</code> → Declara el punto de entrada ejecutable.
Línea 8	<code>String url = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Define la URL JDBC hacia EditorialReportsJava/data/editorial.db.
Línea 9	<code>try {</code> → Abre un bloque protegido; si contiene recursos, se cerrarán automáticamente.
Línea 10	<code>new File("../EditorialReportsJava/data").mkdirs();</code> → Crea el directorio necesario antes de escribir datos o salidas.
Línea 11	<code>Class.forName("org.sqlite.JDBC");</code> → Carga explícitamente el driver SQLite para que el ejemplo sea determinista.
Línea 12	<code>try (Connection conexion = DriverManager.getConnection(url);</code> → Abre una conexión JDBC; el bloque try-with-resources garantiza su cierre.
Línea 13	<code>Statement sentencia = conexion.createStatement();</code> → Crea el Statement usado para inicializar el esquema y los datos.
Línea 14	<code>sentencia.executeUpdate("DROP TABLE IF EXISTS libros");</code> → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 15	<code>sentencia.executeUpdate("CREATE TABLE libros (" +</code> → Ejecuta una sentencia DDL/DML contra SQLite.

Línea 16	"titulo TEXT PRIMARY KEY, " + → Forma parte de la lógica acumulativa del programa.
Línea 17	"precio REAL NOT NULL, " + → Forma parte de la lógica acumulativa del programa.
Línea 18	"paginas INTEGER NOT NULL, " + → Forma parte de la lógica acumulativa del programa.
Línea 19	"fecha_publicacion TEXT NOT NULL, " + → Forma parte de la lógica acumulativa del programa.
Línea 20	"disponible INTEGER NOT NULL)); → Forma parte de la lógica acumulativa del programa.
Línea 21	sentencia.executeUpdate("INSERT INTO libros VALUES ('Cien años de soledad', 19.95, 471, '1967-06-05', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 22	sentencia.executeUpdate("INSERT INTO libros VALUES ('Rayuela', 22.50, 736, '1963-06-28', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 23	sentencia.executeUpdate("INSERT INTO libros VALUES ('La ciudad y los perros', 18.75, 432, '1963-10-15', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 24	sentencia.executeUpdate("INSERT INTO libros VALUES ('Pedro Páramo', 15.90, 132, '1955-03-01', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 25	sentencia.executeUpdate("INSERT INTO libros VALUES ('Ficciones', 21.00, 224, '1944-12-01', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 26	sentencia.executeUpdate("INSERT INTO libros VALUES ('La casa de los espíritus', 23.40, 448, '1982-01-01', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 27	sentencia.executeUpdate("INSERT INTO libros VALUES ('El amor en los tiempos del cólera', 20.80, 496, '1985-09-05', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 28	sentencia.executeUpdate("INSERT INTO libros VALUES ('La muerte de Artemio Cruz', 17.60, 320, '1962-05-01', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 29	sentencia.executeUpdate("INSERT INTO libros VALUES ('Doña Bárbara', 16.95, 400, '1929-02-01', 0)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 30	sentencia.executeUpdate("INSERT INTO libros VALUES ('Martín Fierro', 14.50, 288, '1872-12-01', 0)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 31	sentencia.executeUpdate("INSERT INTO libros VALUES ('Comala', 19.20, 148, '1955-09-01', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 32	sentencia.executeUpdate("INSERT INTO libros VALUES ('Paradiso', 25.00, 576, '1966-01-01', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 33	sentencia.executeUpdate("INSERT INTO libros VALUES ('La invención de Morel', 18.30, 128, '1940-01-01', 1)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 34	sentencia.executeUpdate("INSERT INTO libros VALUES ('El túnel', 16.20, 160, '1948-01-01', 0)"); → Ejecuta una sentencia DDL/DML contra SQLite.
Línea 35	`` → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 36	} → Cierra el bloque Java actual.
Línea 37	System.out.println("Base de datos inicializada correctamente en: " + url); → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 38	System.out.println("Libros insertados: 14"); → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 39	`` → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 40	} catch (Exception e) { → Captura cualquier fallo de compilación, datos, llenado o exportación.
Línea 41	e.printStackTrace(); → Imprime la traza completa para facilitar el diagnóstico.
Línea 42	System.exit(1); → Termina con código distinto de cero para que CI detecte el fallo.

Línea 43	} → Cierra el bloque Java actual.
Línea 44	} → Cierra el bloque Java actual.
Línea 45	} → Cierra el bloque Java actual.

Clase GeneradorInformeConcepto.java

JAVA

```
import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeConcepto {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_concepto.jrxml";
            String rutaJasper = "reports/informe_concepto.jasper";
            String rutaPdf = "output/informe_concepto.pdf";
            String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
            new File("output").mkdirs();
            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
            Map<String, Object> parametros = new HashMap<String, Object>();
            try (Connection conexion = DriverManager.getConnection(urlBD)) {
                JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);
                JasperExportManager.exportReportToPdfFile(documento, rutaPdf);
                System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
                System.out.println("Paginas del documento: " + documento.getPages().size());
            }
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

Explicación línea por línea

Línea 1	import java.io.File; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 2	import java.sql.Connection; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 3	import java.sql.DriverManager; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 4	import java.util.HashMap; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 5	import java.util.Map; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 6	import net.sf.jasperreports.engine.JasperCompileManager; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 7	import net.sf.jasperreports.engine.JasperExportManager; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 8	import net.sf.jasperreports.engine.JasperFillManager; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 9	import net.sf.jasperreports.engine.JasperPrint; → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 10	`` → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 11	public class GeneradorInformeConcepto { → Declara la clase Java del checkpoint.
Línea 12	public static void main(String[] args) { → Declara el punto de entrada ejecutable.
Línea 13	try { → Abre un bloque protegido; si contiene recursos, se cerrarán automáticamente.
Línea 14	String rutaJrxml = "reports/informe_concepto.jrxml"; → Fija la ruta del JRXML desde el Working Directory EditorialReports.

Línea 15	<code>String rutaJasper = "reports/informe_concepto.jasper";</code> → Fija la ruta del artefacto <code>.jasper</code> compilado.
Línea 16	<code>String rutaPdf = "output/informe_concepto.pdf";</code> → Fija la ruta del PDF que se exportará.
Línea 17	<code>String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Define la URL JDBC hacia <code>EditorialReportsJava/data/editorial.db</code> .
Línea 18	<code>new File("output").mkdirs();</code> → Crea el directorio necesario antes de escribir datos o salidas.
Línea 19	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML a <code>.jasper</code> con JasperReports 6.20.0.
Línea 20	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa de parámetros que se entrega al motor de llenado.
Línea 21	<code>try (Connection conexion = DriverManager.getConnection(urlBD)) {</code> → Define la URL JDBC hacia <code>EditorialReportsJava/data/editorial.db</code> .
Línea 22	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);</code> → Llena el informe y obtiene un <code>JasperPrint</code> en memoria.
Línea 23	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → Exporta el <code>JasperPrint</code> a PDF.
Línea 24	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 25	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 26	<code>}</code> → Cierra el bloque Java actual.
Línea 27	<code>} catch (Exception e) {</code> → Captura cualquier fallo de compilación, datos, llenado o exportación.
Línea 28	<code>e.printStackTrace();</code> → Imprime la traza completa para facilitar el diagnóstico.
Línea 29	<code>System.exit(1);</code> → Termina con código distinto de cero para que CI detecte el fallo.
Línea 30	<code>}</code> → Cierra el bloque Java actual.
Línea 31	<code>}</code> → Cierra el bloque Java actual.
Línea 32	<code>}</code> → Cierra el bloque Java actual.

Criterio de fallo: todo `catch` termina con `System.exit(1)` para que una excepción no pueda aparecer como ejecución verde en CI.

Parte D — Simulación del PDF esperado y de la estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT

informe_concepto.jrxml
[Design] [Source]

Ruler: 0 100 200 300 400 500 555 600 650 700

Column Header
h = 25

Portada	Título	Precio	Páginas	Año
---------	--------	--------	---------	-----

Detail 1
h = 60

[IMG] [50x50]

[\${F{titulo}}]

[\${F{precio}}]

[\${F{pag}}]

[\${F{fech}}]

[\${F{disp}}]

[icono]

[#]

Categoría / Longitud / IVA / Antigüedad / Registro

[\${F{titulo}}.length()>30 ? ...]
[\${V{PrecioConIVA}}]
[...]

Panel Outline muestra:
Properties
└─ com.jaspersoft.studio.data.defaultdataadapter = SQLiteEditorial
Fields
└─ titulo [java.lang.String]

```

|  | precio          [java.lang.Double]
|  | paginas        [java.lang.Integer]
|  | fechaPublicacion [java.lang.String]
|  | disponible      [java.lang.Boolean]
|  |
|  | QueryString
|  |   SELECT titulo, precio, paginas, fecha_publicacion AS fechaPublicacion, disponible
|  |   FROM libros ORDER BY titulo
|  |
+-----+

```

Qué representa: la disposición del informe en el editor tras asociar el adaptador JDBC. El panel Outline muestra la propiedad del adaptador, los cinco campos declarados y la consulta SQL.

Cómo verificarlo: comparar la vista del editor con este esquema. El panel Outline debe mostrar el nodo QueryString con la consulta SQL.

D.2 — Jerarquía del Outline

```

informe_concepto
├── Properties
│   └── com.jaspersoft.studio.data.defaultdataadapter = SQLiteEditorial
├── Styles
│   └── (7 estilos del punto 2.5)
├── Fields
│   ├── titulo [java.lang.String]
│   ├── precio [java.lang.Double]
│   ├── paginas [java.lang.Integer]
│   ├── fechaPublicacion [java.lang.String]
│   └── disponible [java.lang.Boolean]
├── Variables
│   └── PrecioConIVA [java.lang.Double]
├── QueryString
│   └── SELECT titulo, precio, paginas, fecha_publicacion AS fechaPublicacion, disponible
│       FROM libros ORDER BY titulo
├── Title [band, height=100]
│   └── ...
├── Column Header [band, height=25]
│   └── ...
├── Detail 1 [band, height=60]
│   └── ...
└── Summary [band, height=70]
    └── ...

```

Qué representa: el árbol de nodos del informe tras asociar el adaptador JDBC. La novedad respecto al punto 2.6 es el nodo QueryString con la consulta SQL y la propiedad del adaptador.

Cómo verificarlo: expandir el nodo informe_concepto en el panel Outline y comparar la estructura. El nodo QueryString debe mostrar la sentencia SQL.

D.3 — Documento PDF resultante, página por página

```

INFORME: informe_concepto.pdf
PÁGINAS TOTALES: 1
TAMAÑO DE PÁGINA: 595 × 842 píxeles (A4 vertical)
ORIGEN DE DATOS: jdbc:sqlite:../EditorialReportsJava/data/editorial.db
REGISTROS OBTENIDOS: 14 (SELECT ... FROM libros ORDER BY titulo)
BANDAS EMITIDAS: Title, Page Header, Column Header, Detail (14 veces),
                  Column Footer, Last Page Footer, Summary, Background

```

Página 1 de 1

Catálogo Editorial - Informe Conceptual
Fecha de emisión: 22/09/2026

Portada	Título	Precio	Páginas	Año
IMG	Cien años de sol.	19,95 €	471	1967
	Categoría: Estándar	IVA: 24,14 €	Reg. 1	
IMG	Comala	19,20 €	148	1955
	Categoría: Estándar	IVA: 23,23 €	Reg. 2	
IMG	Doña Bárbara	16,95 €	400	1929
	Categoría: Estándar	IVA: 20,51 €	Reg. 3	

...

Total de páginas: 1
Total de libros: 14

Qué representa: la página única del PDF resultante con los catorce libros recuperados de la base de datos SQLite. El orden de los libros es alfabético porque la consulta incluye `ORDER BY título`. El primer libro es Cien años de soledad y el último es Rayuela.

Cómo verificarlo: abrir el archivo `output/informe_concepto.pdf` con un lector de PDF y comprobar que los libros aparecen en orden alfabético. Si el orden es distinto, revisar la cláusula `ORDER BY` de la consulta SQL.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```
EditorialReports/
├── ECOSISTEMA.md, ENTORNO.md, BANDAS.md, JRXML.md
├── TEXTO.md, CAMPOS.md, IMAGENES.md, ESTILOS.md, EXPRESIONES.md
├── BASEDATOS.md (documentación de la BD)
├── reports/
│   ├── informe_concepto.jrxml (con consulta SQL)
│   └── informe_concepto.jasper (artefacto compilado)
├── resources/
│   └── (logotipo, iconos y portadas)
├── output/
│   └── informe_concepto.pdf (con datos de la BD)
└── EditorialReportsJava/
    ├── lib/
    │   ├── jasperreports-6.20.0.jar
    │   ├── commons-digester-2.1.jar
    │   ├── commons-collections-3.2.2.jar
    │   ├── commons-logging-1.2.jar
    │   ├── ecj-3.24.0.jar
    │   └── sqlite-jdbc-3.44.0.0.jar (nuevo)
    ├── data/
    │   └── editorial.db (base de datos SQLite)
    └── src/
        ├── GeneradorInformeConcepto.java (modificada con JDBC)
        ├── Libro.java
        ├── CatalogoDataSource.java
        └── InicializadorBD.java (nueva clase)
```

Qué representa: el estado de los dos proyectos tras completar los catorce pasos. La novedad respecto al punto 2.6 es la carpeta `data` con la base de datos, el JAR del driver en `lib`, la clase `InicializadorBD` en `src` y el archivo `BASEDATOS.md` en la raíz del proyecto de informes.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si el archivo `editorial.db` no aparece, repetir el paso 5. Si el archivo `BASEDATOS.md` no aparece, repetir el paso 14.

Errores comunes del ejercicio completo

Error	Causa	Solución
No suitable driver found for jdbc:sqlite:...	El driver JDBC no está en el classpath	Copiar <code>sqlite-jdbc-3.44.0.0.jar</code> a la carpeta <code>lib</code> y añadirlo al Build Path
SQLException: path to 'data/editorial.db': 'data' does not exist	El programa se ejecuta desde un directorio distinto a la raíz del proyecto	Configurar el Working Directory en Run Configurations
SQLException: no such table: libros	La base de datos no tiene la tabla o no se ha ejecutado el inicializador	Ejecutar la clase <code>InicializadorBD</code> antes del programa principal
Field not found: fechaPublicacion	El campo no está declarado en el JRXML	Verificar el alias SQL <code>fecha_publicacion AS fechaPublicacion</code> y el campo <code>fechaPublicacion</code> de tipo String

Error	Causa	Solución
ClassCastException al resolver un campo	El tipo declarado no coincide con el tipo de la columna	Verificar que el tipo del campo coincide con el tipo de la columna
La conexión queda abierta y bloquea la base de datos	No se cerró la conexión después del llenado	Usar un bloque try-with-resources para garantizar el cierre
El adaptador JDBC no se conecta en Jaspersoft Studio	La ruta de la URL es relativa a un directorio distinto	Usar una ruta relativa correcta desde el espacio de trabajo o una ruta absoluta
Test Connection falla en Jaspersoft Studio	El driver no está en el Classpath del adaptador	Añadir el JAR con el botón Add... del campo Driver Classpath
El informe se previsualiza vacío	La consulta SQL no devuelve registros	Verificar que la tabla libros contiene datos con un cliente SQL
La conexión se cierra antes de que el motor termine el llenado	El bloque try-with-resources engloba solo la llamada a fillReport	Asegurarse de que el bloque engloba también la exportación a PDF

Reto resuelto paso a paso

Enunciado: añadir una segunda consulta que devuelva los libros disponibles ordenados por precio descendente y generar un segundo PDF con el resultado. El informe debe usar el mismo JRXML pero un parámetro que indique el orden de la consulta.

Paso 1. Hacer doble clic sobre el archivo informe_concepto.jrxml en el panel Project Explorer.

Paso 2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.

Paso 3. Localizar la línea que contiene `<queryString language="sql">` y seleccionar el bloque completo de la consulta.

Paso 4. Eliminar el bloque con la tecla Suprimir.

Paso 5. Escribir exactamente `<queryString language="sql">` y pulsar Enter.

Paso 6. Escribir exactamente `<![CDATA[SELECT titulo, precio, paginas, fecha_publicacion AS fechaPublicacion, disponible FROM libros WHERE disponible = 1 ORDER BY precio DESC]]>` y pulsar Enter.

Paso 7. Escribir exactamente `</queryString>` y pulsar Enter.

Paso 8. Pulsar Ctrl+S para guardar el archivo.

Paso 9. Pulsar Ctrl+Mayús+B para compilar el informe.

Paso 10. Pulsar el botón Preview y verificar que solo aparecen los libros disponibles ordenados por precio descendente.

Paso 11. Modificar temporalmente la consulta para eliminar el filtro `WHERE disponible = 1` y restaurarlo después.

Paso 12. Ejecutar el programa Java con Run As > Java Application.

Paso 13. Abrir el archivo output/informe_concepto.pdf y verificar que solo aparecen los libros disponibles ordenados por precio descendente.

Simulación ASCII del PDF tras el reto

TEXT

Portada	Título	Precio	Páginas	Año
IMG	La casa de los... Categoría: Premium Reg. 1	23,40 €	448	1982
IMG	Rayuela Categoría: Premium Reg. 2	22,50 €	736	1963
IMG	Ficciones Categoría: Premium Reg. 3	21,00 €	224	1944
...				

Resultado del reto: la consulta `SELECT ... WHERE disponible = 1 ORDER BY precio DESC` devuelve únicamente los once libros disponibles ordenados por precio descendente. El primer libro del informe es La casa de los espíritus con 23,40 € y el último es Martín Fierro con 14,50 €. Los libros no disponibles (Doña Bárbara, Martín Fierro y El túnel) no aparecen en el informe. El reto demuestra cómo modificar la consulta SQL para cambiar el conjunto de datos del informe sin modificar el JRXML.

Analogía final con el contexto de la editorial

La base de datos es el archivador de la editorial. La tabla `libros` es el conjunto de fichas que describen cada libro del catálogo. El driver JDBC es la herramienta que permite leer las fichas del archivador. La consulta SQL es la instrucción que indica qué fichas recuperar y en qué orden. La conexión JDBC es la llave que abre el archivador durante el tiempo necesario. El adaptador de Jaspersoft Studio es la configuración que recuerda dónde está el archivador y cómo abrirlo. El cierre de la conexión es la acción de volver a cerrar el archivador para que otros puedan consultarlo. La combinación de todos estos elementos permite generar el catálogo directamente desde las fichas almacenadas.

Resultado esperado

Al finalizar este punto, el alumno dispone de:

- La carpeta `data` con la base de datos `editorial.db` que contiene la tabla `libros` con catorce registros.
- El archivo `sqlite-jdbc-3.44.0.0.jar` en la carpeta `lib` y referenciado en el Build Path.
- La clase `InicializadorBD.java` que crea la base de datos y la rellena con los datos de ejemplo.
- El adaptador `SQLiteEditorial` en el panel Repository Explorer de Jaspersoft Studio.
- El archivo `reports/informe_concepto.jrxml` con el alias `fechaPublicacion` y la consulta SQL declarada.
- El programa `GeneradorInformeConcepto.java` modificado para usar la conexión JDBC con el bloque `try-with-resources`.
- El archivo `output/informe_concepto.pdf` con los catorce libros recuperados de la base de datos.
- El archivo `BASEDATOS.md` en la raíz del proyecto con la documentación de la conexión.
- Comprensión operativa de JDBC, del driver de SQLite, de la configuración de adaptadores y de la ejecución de informes con conexión a base de datos.

Conclusión y enlace al siguiente punto

El punto 3.1 ha introducido la conexión a bases de datos mediante JDBC y ha demostrado su uso con una base de datos SQLite. Han quedado configurados el driver, el adaptador de Jaspersoft Studio y la consulta SQL en el JRXML. El informe obtiene ahora los datos de la base de datos en lugar de una lista de ejemplo. La clase `InicializadorBD` permite regenerar la base de datos en cualquier momento y la clase `GeneradorInformeConcepto` utiliza el bloque `try-with-resources` para garantizar el cierre de la conexión.

El punto 3.2, «Ficheros CSV», introduce la lectura de datos desde archivos CSV y demuestra su uso con un archivo de catálogo de libros. El punto cubre la configuración de adaptadores CSV en Jaspersoft Studio, la lectura desde código Java con `JRBeanCollectionDataSource` y la combinación de datos de la base de datos con datos del archivo CSV.

Punto 3.2 — Ficheros CSV

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.2 — Ficheros CSV

Objetivos de aprendizaje

- Comprender el formato CSV y su papel como fuente de datos ligera.
- Configurar un adaptador CSV en Jaspersoft Studio y asociarlo a un informe.
- Declarar los campos del informe a partir de las columnas del archivo CSV.
- Leer un archivo CSV desde código Java sin depender de Jaspersoft Studio.
- Combinar datos de un archivo CSV con datos de la base de datos en un mismo informe.
- Documentar el uso de archivos CSV en el proyecto EditorialReports.

Parte práctica

Parte A — Práctica visual

Paso 1: Crear la carpeta data en el proyecto EditorialReports

Acciones:

1. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Folder en el submenú.
4. Escribir exactamente data en el campo Folder name del diálogo.
5. Hacer clic sobre el botón Finish.

Verificación visual: el panel Project Explorer muestra la carpeta data dentro del proyecto EditorialReports.

Qué hace: crea la carpeta que alojará el archivo CSV.

Por qué: el archivo CSV debe residir en una carpeta separada de las plantillas y de los recursos gráficos.

Error común: crear la carpeta con el nombre Data con mayúscula inicial. Las rutas del código Java son sensibles a mayúsculas.

Solución: eliminar la carpeta y volver a crearla con el nombre exacto en minúsculas.

Analogía: es como habilitar una carpeta en la editorial para los listados en formato de texto.

Paso 2: Crear el archivo catalogo.csv

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta data en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción File en el submenú.
4. Escribir exactamente catalogo.csv en el campo File name del diálogo.
5. Hacer clic sobre el botón Finish.
6. En el editor central, escribir exactamente la primera línea:
 titulo,autor,precio,paginas,fecha_publicacion,disponible y pulsar Enter.
7. Escribir exactamente Cien años de soledad,Gabriel García Márquez,19.95,471,1967-06-05,true y pulsar Enter.
8. Escribir exactamente Rayuela,Julio Cortázar,22.50,736,1963-06-28,true y pulsar Enter.
9. Escribir exactamente La ciudad y los perros,Mario Vargas Llosa,18.75,432,1963-10-15,true y pulsar Enter.

10. Escribir exactamente Pedro Páramo, Juan Rulfo, 15.90, 132, 1955-03-01, true y pulsar Enter.
11. Escribir exactamente Ficciones, Jorge Luis Borges, 21.00, 224, 1944-12-01, true y pulsar Enter.
12. Escribir exactamente La casa de los espíritus, Isabel Allende, 23.40, 448, 1982-01-01, true y pulsar Enter.
13. Escribir exactamente El amor en los tiempos del cólera, Gabriel García Márquez, 20.80, 496, 1985-09-05, true y pulsar Enter.
14. Escribir exactamente La muerte de Artemio Cruz, Carlos Fuentes, 17.60, 320, 1962-05-01, true y pulsar Enter.
15. Escribir exactamente Doña Bárbara, Rómulo Gallegos, 16.95, 400, 1929-02-01, false y pulsar Enter.
16. Escribir exactamente Martín Fierro, José Hernández, 14.50, 288, 1872-12-01, false y pulsar Enter.
17. Escribir exactamente Comala, Juan Rulfo, 19.20, 148, 1955-09-01, true y pulsar Enter.
18. Escribir exactamente Paradiso, José Lezama Lima, 25.00, 576, 1966-01-01, true y pulsar Enter.
19. Escribir exactamente La invención de Morel, Adolfo Bioy Casares, 18.30, 128, 1940-01-01, true y pulsar Enter.
20. Escribir exactamente El túnel, Ernesto Sábato, 16.20, 160, 1948-01-01, false y pulsar Enter.
21. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo catalogo.csv dentro de la carpeta data. El editor central muestra la cabecera y los catorce registros.

Qué hace: crea el archivo CSV con la cabecera y los catorce libros del catálogo.

Por qué: el archivo CSV es la fuente de datos del informe de este punto.

Error común: olvidar la línea de cabecera y provocar que la primera fila de datos se interprete como cabecera.

Solución: verificar que la primera línea contiene los nombres de las columnas.

Analogía: es como preparar el listado de libros en formato de texto para intercambiarlo con un distribuidor.

Paso 3: Crear el adaptador CSV en Jaspersoft Studio

Acciones:

1. Hacer clic con el botón derecho sobre el nodo Data Adapters en el panel Repository Explorer (lateral izquierdo).
2. Hacer clic sobre la opción Create Data Adapter en el menú contextual.
3. Hacer clic sobre el elemento CSV File Data Source en la lista de categorías del asistente.
4. Hacer clic sobre el botón Next.
5. Escribir exactamente CatalogoCSV en el campo Name.
6. Hacer clic sobre el botón Browse... situado junto al campo File y navegar hasta la carpeta Documents\JasperProjects\EditorialReports\data.
7. Hacer clic sobre el archivo catalogo.csv y pulsar el botón Open.
8. Hacer clic sobre el campo Charset y escribir exactamente UTF-8.
9. Hacer clic sobre el campo Delimiter y escribir exactamente , (una coma).
10. Marcar la casilla Use First Row as Column Names.
11. Hacer clic sobre el botón Test Connection.
12. Verificar que el diálogo muestra el mensaje Connection successful y la lista de columnas detectadas.
13. Hacer clic sobre el botón Finish.

Verificación visual: el panel Repository Explorer muestra el nodo CatalogoCSV colgando de Data Adapters con un icono de archivo.

Qué hace: registra un adaptador CSV que Jaspersoft Studio utiliza para leer el archivo durante la previsualización.

Por qué: el adaptador permite ejecutar el informe con los datos del CSV desde el entorno de diseño.

Error común: olvidar marcar la casilla Use First Row as Column Names y provocar que la primera fila de datos se interprete como cabecera.

Solución: marcar la casilla antes de pulsar Test Connection.

Analogía: es como registrar en la editorial el listado de libros para poder consultarlo durante la composición.

Paso 4: Crear el informe informe_catalogo_csv.jrxml

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta `reports` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Jasper Report en el submenú.
4. Hacer clic sobre la plantilla Blank A4 en la lista de plantillas del asistente.
5. Hacer clic sobre el botón Next.
6. Escribir exactamente `informe_catalogo_csv` en el campo File name.
7. Hacer clic sobre el botón Next.
8. Hacer clic sobre CatalogoCSV en la lista de adaptadores disponibles.
9. Hacer clic sobre el botón Finish.
10. Observar que el asistente ha generado automáticamente las declaraciones de los cinco campos.

Verificación visual: el editor central muestra el archivo `informe_catalogo_csv.jrxml` con las bandas por defecto. El panel Outline muestra el nodo Fields con los cinco campos generados automáticamente.

Qué hace: crea un nuevo informe a partir del adaptador CSV con las declaraciones de campo generadas automáticamente.

Por qué: la generación automática evita errores de tipeo en los nombres de los campos.

Error común: seleccionar el adaptador SQLiteEditorial en lugar de CatalogoCSV. El informe se generaría con la consulta SQL en lugar de los campos del CSV.

Solución: cerrar el asistente y repetir el paso seleccionando el adaptador correcto.

Analogía: es como abrir un nuevo pliego del catálogo con la estructura del listado de libros ya preparada.

Paso 5: Ajustar las bandas del nuevo informe

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `informe_catalogo_csv` en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la opción Delete en el menú contextual para eliminar el nodo Page Header.
3. Hacer clic con el botón derecho sobre el nodo `informe_catalogo_csv` y eliminar el nodo Column Footer.
4. Hacer clic con el botón derecho sobre el nodo `informe_catalogo_csv` y eliminar el nodo Summary.
5. Hacer clic sobre el nodo Title en el panel Outline y ajustar su Band height a 60 píxeles desde el panel Properties.

Verificación visual: el panel Outline muestra solo las bandas Title, Column Header, Detail 1, Page Footer y Background.

Qué hace: simplifica el informe para que contenga solo las bandas necesarias.

Por qué: el informe de este punto es más sencillo que el informe conceptual y solo necesita las bandas básicas.

Error común: eliminar la banda Background pensando que no se usa.

Solución: si se elimina por error, cerrar el archivo sin guardar y volver a abrirlo.

Analogía: es como reducir el pliego del listado a las secciones estrictamente necesarias.

Paso 6: Añadir el título en la banda Title

Acciones:

1. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
2. Hacer clic sobre el icono Static Text (una letra T mayúscula).

3. Arrastrar el icono Static Text y soltarlo dentro de la banda Title, en la coordenada aproximada x=0, y=15.
4. Hacer doble clic sobre el Static Text creado en la acción anterior.
5. Escribir exactamente `Catálogo Editorial - Datos desde CSV`.
6. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
7. Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 0 y pulsar Enter.
8. Hacer clic sobre el campo Y, escribir 15 y pulsar Enter.
9. Hacer clic sobre el campo Width, escribir 555 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 30 y pulsar Enter.
11. Hacer clic sobre el campo Font size y escribir 18. Pulsar Enter.
12. Marcar la casilla Bold.
13. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.

Verificación visual: la banda Title muestra el texto `Catálogo Editorial - Datos desde CSV` centrado y en negrita.

Qué hace: inserta el título del informe de catálogo desde CSV.

Por qué: el título identifica el documento y su origen de datos.

Error común: olvidar el centrado y provocar que el título aparezca alineado a la izquierda.

Solución: seleccionar Center en el desplegable Horizontal Text Alignment.

Analogía: es como titular el listado con el nombre de la editorial y la indicación de que los datos proceden del listado de texto.

Paso 7: Añadir los encabezados de columna

Acciones:

1. Hacer clic sobre el nodo Column Header en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Static Text (una letra T mayúscula).
4. Arrastrar el icono Static Text y soltarlo dentro de la banda Column Header, en la coordenada aproximada x=0, y=5.
5. Hacer doble clic sobre el Static Text creado en la acción anterior.
6. Escribir exactamente `Título`.
7. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
8. Hacer clic sobre el campo Width en el panel Properties, escribir 250 y pulsar Enter.
9. Marcar la casilla Bold.
10. Repetir las acciones 3 a 9 para los encabezados Autor (x=250, ancho 150), Precio (x=400, ancho 80) y Páginas (x=480, ancho 75).

Verificación visual: la banda Column Header muestra los cuatro encabezados `Título`, `Autor`, `Precio` y `Páginas` en negrita.

Qué hace: inserta los encabezados de las columnas que se van a mostrar.

Por qué: los encabezados identifican las columnas de la tabla del listado.

Error común: dejar los encabezados sin negrita y provocar que no se distingan del cuerpo.

Solución: marcar la casilla Bold en el panel Properties.

Analogía: es como añadir los títulos de las columnas al listado de libros.

Paso 8: Añadir los campos en la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).

4. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=0, y=0.
5. Hacer clic sobre el campo Width en el panel Properties, pestaña Properties, escribir 250 y pulsar Enter.
6. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.
7. Hacer clic sobre el campo Text Field Expression y escribir exactamente `$F{titulo}` y pulsar Enter.
8. Repetir las acciones 3 a 7 para el campo autor (x=250, ancho 150) y el campo paginas (x=480, ancho 75).
9. Para el campo del precio, hacer clic sobre el icono Text Field y arrastrarlo dentro de la banda Detail 1, en la coordenada aproximada x=400, y=0.
10. Hacer clic sobre el campo Width y escribir 80. Pulsar Enter.
11. Hacer clic sobre el campo Height y escribir 20. Pulsar Enter.
12. Hacer clic sobre el campo Text Field Expression y escribir exactamente `Double.parseDouble($F{precio})` y pulsar Enter.
13. Hacer clic sobre el campo Pattern y escribir exactamente `#,##0.00 €` y pulsar Enter.
14. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Right.

Verificación visual: la banda Detail 1 muestra cuatro campos con las expresiones correspondientes. El campo del precio tiene el patrón numérico.

Qué hace: inserta los campos que se imprimen para cada registro del CSV.

Por qué: los campos resuelven los valores del CSV para cada libro.

Error común: declarar el campo precio como `java.lang.Double` en el JRXML. El adaptador CSV devuelve cadenas y el motor lanza `ClassCastException`.

Solución: declarar el campo como `java.lang.String` y convertir en la expresión con `Double.parseDouble`.

Analogía: es como rellenar las celdas del listado con los datos de cada libro.

Paso 9: Ajustar la altura de la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 20 y pulsar Enter.
3. Hacer clic sobre el desplegable Split Type en el panel Properties y verificar que está en `Stretch`.

Verificación visual: la banda Detail 1 aparece con 20 píxeles de altura.

Qué hace: fija la altura de la banda Detail para que cada registro ocupe una fila.

Por qué: la altura determina el espacio de cada fila y el número de filas que caben en una página.

Error común: dejar la altura por defecto y provocar que el informe ocupe más páginas de las necesarias.

Solución: ajustar la altura a 20 píxeles.

Analogía: es como ajustar la altura de cada fila del listado para que todos los libros quepan en una página.

Paso 10: Añadir el pie de página con paginación correcta y recuento

Acciones:

1. Seleccionar Page Footer en Outline y fijar Band height = 45.
2. Añadir un Static Text en x=0, y=3, ancho=150, alto=15 con el texto Registros CSV:.
3. Añadir un Text Field en x=150, y=3, ancho=70, alto=15, expresión `$V{REPORT_COUNT}` y marcar Bold.
4. Añadir un Text Field en x=170, y=23, ancho=190, alto=15.
5. Escribir exactamente "Página " + `$V{PAGE_NUMBER}` + " de" como expresión del primer campo de paginación.
6. Alinear ese campo a la derecha.
7. Añadir un segundo Text Field en x=365, y=23, ancho=30, alto=15.
8. Escribir `$V{PAGE_NUMBER}` como expresión del segundo campo.

9. En Properties > Text Field, seleccionar `Evaluation Time = Report` para ese segundo campo.

10. Fijar tamaño de fuente 9 en los dos campos de paginación y guardar con Ctrl+S.

Verificación visual: Page Footer muestra Registros CSV: <n> y Página N de M. En Source, el segundo campo contiene `evaluationTime="Report"`.

Qué hace: muestra el número de página actual, el total real de páginas evaluado al final del informe y los registros procesados.

Por qué: `PAGE_COUNT` no es el total de páginas; cuenta registros procesados en la página actual. El total de páginas se obtiene con `PAGE_NUMBER` evaluado con `evaluationTime="Report"`.

Error común: usar `$V{PAGE_COUNT}` como total de páginas.

Solución: usar un segundo Text Field con `$V{PAGE_NUMBER}` y `Evaluation Time = Report`.

Analogía: es como imprimir primero el número de pliego actual y, al cerrar la tirada, completar el total definitivo de pliegos.

****Paso 11:**

Paso 11: Compilar y previsualizar con el adaptador CSV

Acciones:

1. Pulsar Ctrl+S para guardar el archivo.
2. Pulsar Ctrl+Mayús+B para compilar el informe.
3. Hacer clic sobre el panel Problems (inferior) y verificar que no hay errores.
4. Pulsar el botón Preview de la barra de herramientas superior.
5. En el diálogo, verificar que el adaptador `CatalogoCSV` está seleccionado.
6. Hacer clic sobre el botón OK.
7. Esperar a que se abra la pestaña Preview en el editor central.

Verificación visual: la pestaña Preview muestra el informe con los catorce libros del CSV ordenados según el archivo. El campo del precio muestra el valor formateado con el símbolo del euro.

Qué hace: ejecuta el informe con los datos del CSV y muestra el resultado.

Por qué: la previsualización confirma que el adaptador CSV funciona y que los campos se resuelven correctamente.

Error común: obtener `ClassCastException` al resolver el campo del precio. Indica que el campo está declarado como `java.lang.Double` pero el CSV devuelve cadenas.

Solución: cambiar el tipo del campo a `java.lang.String` y convertir en la expresión.

Analogía: es como revisar la prueba de color del listado con los datos del archivo de texto.

Paso 12: Añadir el campo autor al informe

Acciones:

1. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
2. Localizar la línea que contiene `<field name="disponible" class="java.lang.String"/>`.
3. Verificar que el campo `autor` ya está declarado en el bloque de campos generados automáticamente.
4. Si no está declarado, hacer clic al final de la línea anterior y pulsar Enter.
5. Escribir exactamente `<field name="autor" class="java.lang.String"/>` y pulsar Enter.
6. Pulsar Ctrl+S para guardar el archivo.
7. Hacer clic sobre la pestaña Design en la parte inferior del editor central.

Verificación visual: el panel Outline muestra el nodo Fields con el campo `autor` declarado.

Qué hace: verifica o añade el campo autor a la declaración de campos del informe.

Por qué: el campo autor se utiliza en la banda Detail y debe estar declarado.

Error común: olvidar la declaración del campo y provocar `Field not found: autor` al compilar.

Solución: añadir la declaración en el bloque de campos.

Analogía: es como asegurarse de que el campo del autor está incluido en el listado de datos del catálogo.

Paso 13: Modificar el programa Java para leer el CSV

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta `src` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción `New` en el menú contextual.
3. Hacer clic sobre la opción `Class` en el submenú.
4. Escribir exactamente `GeneradorCatalogoCSV` en el campo `Name` del diálogo.
5. Marcar la casilla `public static void main(String[] args)`.
6. Hacer clic sobre el botón `Finish`.
7. En el editor central, escribir el código completo de la clase `GeneradorCatalogoCSV` que se muestra en la Parte C de este punto.
8. Pulsar `Ctrl+S` para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `GeneradorCatalogoCSV.java` dentro de la carpeta `src`.

Qué hace: crea la clase que genera el informe de catálogo desde el archivo CSV.

Por qué: el informe puede ejecutarse desde código Java sin depender de Jaspersoft Studio.

Error común: olvidar importar `net.sf.jasperreports.engine.data.JRCSVDataSource`. El compilador informa `cannot find symbol`.

Solución: añadir la importación correspondiente.

Analogía: es como preparar la consola de control para que el operario genere el listado desde el archivo de texto.

Paso 14: Ejecutar el programa y verificar el PDF

Acciones:

1. Hacer clic con el botón derecho sobre el archivo `GeneradorCatalogoCSV.java` en el panel Project Explorer.
2. Hacer clic sobre la opción `Run As` en el menú contextual.
3. Hacer clic sobre la opción `Java Application` en el submenú.
4. Hacer clic sobre la vista `Console` en el panel inferior y observar el resultado.
5. Abrir el explorador de archivos del sistema operativo.
6. Navegar hasta la carpeta `output` del proyecto `EditorialReports`.
7. Hacer doble clic sobre el archivo `informe_catalogo_csv.pdf`.
8. Verificar que el PDF muestra los catorce libros con el título, el autor, el precio formateado y el número de páginas.

Verificación visual: la vista `Console` muestra la línea `Informe generado en: ...` con la ruta absoluta del PDF. El archivo PDF muestra los catorce libros del CSV.

Qué hace: ejecuta el programa Java que lee el CSV y genera el informe.

Por qué: la ejecución confirma que el programa lee el CSV y genera el informe correctamente.

Error común: ejecutar el programa desde un directorio distinto a la raíz del proyecto y obtener `FileNotFoundException: data/catalogo.csv`.

Solución: comprobar en Run Configurations que el Working Directory apunta a la raíz del proyecto.

Analogía: es como imprimir el listado de libros desde el archivo de texto.

Paso 15: Documentar el uso de archivos CSV

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `EditorialReports` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción `New` en el menú contextual.
3. Hacer clic sobre la opción `File` en el submenú.
4. Escribir exactamente `CSV.md` en el campo File name del diálogo.
5. Hacer clic sobre el botón `Finish`.
6. En el editor central, escribir exactamente `# Ficheros CSV` y pulsar `Enter` dos veces.
7. Escribir exactamente `## Archivos del proyecto` y pulsar `Enter` dos veces.
8. Escribir exactamente `- data/catalogo.csv: catálogo de libros con título, autor, precio, páginas, fecha de publicación y disponibilidad.` y pulsar `Enter` dos veces.
9. Escribir exactamente `## Adaptador de Jaspersoft Studio` y pulsar `Enter` dos veces.
10. Escribir exactamente `- Nombre: CatalogoCSV` y pulsar `Enter`.
11. Escribir exactamente `- Delimitador: ,` y pulsar `Enter`.
12. Escribir exactamente `- Codificación: UTF-8` y pulsar `Enter`.
13. Escribir exactamente `- Primera fila como cabecera: sí` y pulsar `Enter` dos veces.
14. Escribir exactamente `## Lectura desde Java` y pulsar `Enter` dos veces.
15. Escribir exactamente `- Clase: net.sf.jasperreports.engine.data.JRCsvDataSource` y pulsar `Enter`.
16. Pulsar `Ctrl+S` para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `CSV.md` en la raíz del proyecto `EditorialReports`.

Qué hace: incorpora al proyecto un documento que registra el uso de archivos CSV.

Por qué: la documentación de las fuentes de datos facilita el mantenimiento y la incorporación de nuevos desarrolladores.

Error común: olvidar documentar el delimitador.

Solución: incluir las cuatro propiedades del adaptador.

Analogía: es como dejar en la editorial una ficha técnica con la ubicación del listado y su formato.

Parte B — JRXML completo explicado línea por línea [VALIDADO]

JRXML canónico del checkpoint. Es el mismo archivo que se compila en GitHub Actions.

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_catalogo_csv"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
  leftMargin="20"
  rightMargin="20"
  topMargin="20"
  bottomMargin="20"
  uuid="9a3d2b5f-2e4c-5a6b-8d1f-3c7e9a0b2d44">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="CatalogoCSV"/>
  <style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
  <field name="titulo" class="java.lang.String"/>
  <field name="autor" class="java.lang.String"/>
```

XML

```

<field name="precio" class="java.lang.String"/>
<field name="paginas" class="java.lang.String"/>
<field name="fecha_publicacion" class="java.lang.String"/>
<field name="disponible" class="java.lang.String"/>
<background>
  <band height="0"/>
</background>

<title>
  <band height="60">
    <staticText>
      <reportElement x="0" y="15" width="555" height="30" uuid="1b2c3d4e-5f6a-7b8c-9d0e-1f2a3b4c5d6e"/>
      <textElement textAlignment="Center" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="18" isBold="true"/>
      </textElement>
      <text><![CDATA[Catálogo Editorial - Datos desde CSV]]></text>
    </staticText>
  </band>
</title>
<columnHeader>
  <band height="25">
    <staticText>
      <reportElement x="0" y="5" width="250" height="15" uuid="2c3d4e5f-6a7b-8c9d-0e1f-2a3b4c5d6e7f"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10" isBold="true"/>
      </textElement>
      <text><![CDATA[Título]]></text>
    </staticText>
    <staticText>
      <reportElement x="250" y="5" width="150" height="15" uuid="3d4e5f6a-7b8c-9d0e-1f2a-3b4c5d6e7f8a"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10" isBold="true"/>
      </textElement>
      <text><![CDATA[Autor]]></text>
    </staticText>
    <staticText>
      <reportElement x="400" y="5" width="80" height="15" uuid="4e5f6a7b-8c9d-0e1f-2a3b-4c5d6e7f8a9b"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10" isBold="true"/>
      </textElement>
      <text><![CDATA[Precio]]></text>
    </staticText>
    <staticText>
      <reportElement x="480" y="5" width="75" height="15" uuid="5f6a7b8c-9d0e-1f2a-3b4c-5d6e7f8a9b0c"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10" isBold="true"/>
      </textElement>
      <text><![CDATA[Páginas]]></text>
    </staticText>
  </band>
</columnHeader>
<detail>
  <band height="20" splitType="Stretch">
    <textField textAdjust="StretchHeight">
      <reportElement x="0" y="0" width="250" height="20" uuid="6a7b8c9d-0e1f-2a3b-4c5d-6e7f8a9b0c1d"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[${F{titulo}}]></textFieldExpression>
    </textField>
    <textField>
      <reportElement x="250" y="0" width="150" height="20" uuid="7b8c9d0e-1f2a-3b4c-5d6e-7f8a9b0c1d2e"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[${F{autor}}]></textFieldExpression>
    </textField>
    <textField pattern="#.##0.00 €">
      <reportElement x="400" y="0" width="80" height="20" uuid="8c9d0e1f-2a3b-4c5d-6e7f-8a9b0c1d2e3f"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[Double.parseDouble(${F{precio}})]></textFieldExpression>
    </textField>
    <textField>
      <reportElement x="480" y="0" width="75" height="20" uuid="9d0e1f2a-3b4c-5d6e-7f8a-9b0c1d2e3f4a"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[Integer.valueOf(${F{paginas}})]></textFieldExpression>
    </textField>
  </band>
</detail>
<pageFooter>
  <band height="45">
    <staticText>
      <reportElement x="0" y="3" width="150" height="15" uuid="11111111-1111-4111-8111-111111111111"/>
      <textElement verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
      <text><![CDATA[Registros CSV:]]></text>
    </staticText>
    <textField>
      <reportElement x="150" y="3" width="70" height="15" uuid="11111111-1111-4111-8111-111111111112"/>

```

```

        <textElement verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9" isBold="true"/></textElement>
        <textFieldExpression><![CDATA[${REPORT_COUNT}]]></textFieldExpression>
    </textField>
    <textField>
        <reportElement x="170" y="23" width="190" height="15" uuid="11111111-1111-4111-8111-111111111113"/>
        <textElement textAlignment="Right" verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
        <textFieldExpression><![CDATA["Página " + ${PAGE_NUMBER} + " de "]]></textFieldExpression>
    </textField>
    <textField evaluationTime="Report">
        <reportElement x="365" y="23" width="30" height="15" uuid="11111111-1111-4111-8111-111111111114"/>
        <textElement textAlignment="Left" verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
        <textFieldExpression><![CDATA[${PAGE_NUMBER}]]></textFieldExpression>
    </textField>
</band>
</pageFooter>
</jasperReport>

```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → Declara el documento XML y la codificación UTF-8.
Línea 2	<code><jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"</code> → Abre la plantilla JasperReports y define sus atributos principales.
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"</code> → Completa la definición declarativa del informe.
Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Declara el espacio de nombres/XSD usado para validar el JRXML.
Línea 5	<code>name="informe_catalogo_csv"</code> → Completa la definición declarativa del informe.
Línea 6	<code>language="java"</code> → Completa la definición declarativa del informe.
Línea 7	<code>pageWidth="595"</code> → Completa la definición declarativa del informe.
Línea 8	<code>pageHeight="842"</code> → Completa la definición declarativa del informe.
Línea 9	<code>columnWidth="555"</code> → Completa la definición declarativa del informe.
Línea 10	<code>leftMargin="20"</code> → Completa la definición declarativa del informe.
Línea 11	<code>rightMargin="20"</code> → Completa la definición declarativa del informe.
Línea 12	<code>topMargin="20"</code> → Completa la definición declarativa del informe.
Línea 13	<code>bottomMargin="20"</code> → Completa la definición declarativa del informe.
Línea 14	<code>uuid="9a3d2b5f-2e4c-5a6b-8d1f-3c7e9a0b2d44"></code> → Completa la definición declarativa del informe.
Línea 15	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="CatalogoCSV"/></code> → Asocia el Data Adapter usado por Jaspersoft Studio durante Preview.
Línea 16	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/></code> → Declara un estilo compatible con JasperReports 6.20.0.
Línea 17	<code><field name="titulo" class="java.lang.String"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 18	<code><field name="autor" class="java.lang.String"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 19	<code><field name="precio" class="java.lang.String"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 20	<code><field name="paginas" class="java.lang.String"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 21	<code><field name="fecha_publicacion" class="java.lang.String"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.

Línea 22	<code><field name="disponible" class="java.lang.String" /></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 23	<code><background></code> → Abre una sección/banda estructural del informe.
Línea 24	<code><band height="0" /></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 25	<code></background></code> → Cierra el elemento XML abierto anteriormente.
Línea 26	`` → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 27	<code><title></code> → Abre una sección/banda estructural del informe.
Línea 28	<code><band height="60"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 29	<code><staticText></code> → Abre un elemento de texto estático.
Línea 30	<code><reportElement x="0" y="15" width="555" height="30" uuid="1b2c3d4e-5f6a-7b8c-9d0e-1f2a3b4c5d6e" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 31	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 32	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 33	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 34	<code><text><![CDATA[Catálogo Editorial - Datos desde CSV]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 35	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 36	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 37	<code></title></code> → Cierra el elemento XML abierto anteriormente.
Línea 38	<code><columnHeader></code> → Abre una sección/banda estructural del informe.
Línea 39	<code><band height="25"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 40	<code><staticText></code> → Abre un elemento de texto estático.
Línea 41	<code><reportElement x="0" y="5" width="250" height="15" uuid="2c3d4e5f-6a7b-8c9d-0e1f-2a3b4c5d6e7f" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 42	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 43	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 44	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 45	<code><text><![CDATA[Título]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 46	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 47	<code><staticText></code> → Abre un elemento de texto estático.
Línea 48	<code><reportElement x="250" y="5" width="150" height="15" uuid="3d4e5f6a-7b8c-9d0e-1f2a-3b4c5d6e7f8a" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 49	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 50	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 51	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.

Línea 52	<code><text><![CDATA[Autor]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 53	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 54	<code><staticText></code> → Abre un elemento de texto estático.
Línea 55	<code><reportElement x="400" y="5" width="80" height="15" uuid="4e5f6a7b-8c9d-0e1f-2a3b-4c5d6e7f8a9b"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 56	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 57	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 58	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 59	<code><text><![CDATA[Precio]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 60	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 61	<code><staticText></code> → Abre un elemento de texto estático.
Línea 62	<code><reportElement x="480" y="5" width="75" height="15" uuid="5f6a7b8c-9d0e-1f2a-3b4c-5d6e7f8a9b0c"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 63	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 64	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 65	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 66	<code><text><![CDATA[Páginas]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 67	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 68	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 69	<code></columnHeader></code> → Cierra el elemento XML abierto anteriormente.
Línea 70	<code><detail></code> → Abre una sección/banda estructural del informe.
Línea 71	<code><band height="20" splitType="Stretch"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 72	<code><textField textAdjust="StretchHeight"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 73	<code><reportElement x="0" y="0" width="250" height="20" uuid="6a7b8c9d-0e1f-2a3b-4c5d-6e7f8a9b0c1d"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 74	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 75	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 76	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 77	<code><textFieldExpression><![CDATA[\$F{titulo}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 78	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 79	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.

Línea 80	<code><reportElement x="250" y="0" width="150" height="20" uuid="7b8c9d0e-1f2a-3b4c-5d6e-7f8a9b0c1d2e"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 81	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 82	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 83	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 84	<code><textFieldExpression><![CDATA[\${autor}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 85	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 86	<code><textField pattern="#.##0.00 €"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 87	<code><reportElement x="400" y="0" width="80" height="20" uuid="8c9d0e1f-2a3b-4c5d-6e7f-8a9b0c1d2e3f"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 88	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 89	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 90	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 91	<code><textFieldExpression><![CDATA[Double.parseDouble(\${precio}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 92	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 93	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 94	<code><reportElement x="480" y="0" width="75" height="20" uuid="9d0e1f2a-3b4c-5d6e-7f8a-9b0c1d2e3f4a"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 95	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 96	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 97	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 98	<code><textFieldExpression><![CDATA[Integer.valueOf(\${paginas}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 99	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 100	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 101	<code></detail></code> → Cierra el elemento XML abierto anteriormente.
Línea 102	<code><pageFooter></code> → Abre una sección/banda estructural del informe.
Línea 103	<code><band height="45"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 104	<code><staticText></code> → Abre un elemento de texto estático.
Línea 105	<code><reportElement x="0" y="3" width="150" height="15" uuid="11111111-1111-4111-8111-111111111111"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 106	<code><textElement verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.

Línea 107	<code><text><![CDATA[Registros CSV:]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 108	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 109	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 110	<code><reportElement x="150" y="3" width="70" height="15" uuid="11111111-1111-4111-8111-111111111112"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 111	<code><textElement verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 112	<code><textFieldExpression><![CDATA[\${REPORT_COUNT}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 113	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 114	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 115	<code><reportElement x="170" y="23" width="190" height="15" uuid="11111111-1111-4111-8111-111111111113"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 116	<code><textElement textAlignment="Right" verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 117	<code><textFieldExpression><![CDATA["Página " + \${PAGE_NUMBER} + " de"]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 118	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 119	<code><textField evaluationTime="Report"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 120	<code><reportElement x="365" y="23" width="30" height="15" uuid="11111111-1111-4111-8111-111111111114"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 121	<code><textElement textAlignment="Left" verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 122	<code><textFieldExpression><![CDATA[\${PAGE_NUMBER}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 123	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 124	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 125	<code></pageFooter></code> → Cierra el elemento XML abierto anteriormente.
Línea 126	<code></jasperReport></code> → Cierra el elemento XML abierto anteriormente.

Comprobación: las coordenadas se mantienen dentro de `columnWidth="555"`, el orden estructural es compatible con JasperReports 6.20.0 y no se usa sintaxis retirada de la baseline.

Parte C — Código Java explicado línea por línea [VALIDADO]

El código siguiente es el código real incluido en el checkpoint y ejecutado por el workflow E2E. Java no redibuja el informe: **compila -> llena -> exporta**.

Clase `GeneradorCatalogoCSV.java`

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
```

```
import net.sf.jasperreports.engine.data.JRCsvDataSource;

public class GeneradorCatalogoCSV {
    public static void main(String[] args) {
        JRCsvDataSource dataSource = null;
        try {
            String rutaJrxml = "reports/informe_catalogo_csv.jrxml";
            String rutaJasper = "reports/informe_catalogo_csv.jasper";
            String rutaPdf = "output/informe_catalogo_csv.pdf";
            String rutaCsv = "data/catalogo.csv";
            new File("output").mkdirs();
            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
            dataSource = new JRCsvDataSource(new File(rutaCsv), "UTF-8");
            dataSource.setFieldDelimiter(',');
            dataSource.setUseFirstRowAsHeader(true);
            Map<String, Object> parametros = new HashMap<>();
            JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, dataSource);
            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);
            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
            System.out.println("Registros CSV esperados: 14");
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        } finally {
            if (dataSource != null) dataSource.close();
        }
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 2	<code>import java.util.HashMap;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 3	<code>import java.util.Map;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 4	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 5	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 6	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 7	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 8	<code>import net.sf.jasperreports.engine.data.JRCsvDataSource;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 9	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 10	<code>public class GeneradorCatalogoCSV {</code> → Declara la clase Java del checkpoint.
Línea 11	<code>public static void main(String[] args) {</code> → Declara el punto de entrada ejecutable.
Línea 12	<code>JRCsvDataSource dataSource = null;</code> → Construye la fuente CSV que alimentará el informe.
Línea 13	<code>try {</code> → Abre un bloque protegido; si contiene recursos, se cerrarán automáticamente.
Línea 14	<code>String rutaJrxml = "reports/informe_catalogo_csv.jrxml";</code> → Fija la ruta del JRXML desde el Working Directory EditorialReports.
Línea 15	<code>String rutaJasper = "reports/informe_catalogo_csv.jasper";</code> → Fija la ruta del artefacto <code>.jasper</code> compilado.
Línea 16	<code>String rutaPdf = "output/informe_catalogo_csv.pdf";</code> → Fija la ruta del PDF que se exportará.
Línea 17	<code>String rutaCsv = "data/catalogo.csv";</code> → Forma parte de la lógica acumulativa del programa.
Línea 18	<code>new File("output").mkdirs();</code> → Crea el directorio necesario antes de escribir datos o salidas.

Línea 19	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML a .jasper con JasperReports 6.20.0.
Línea 20	<code>dataSource = new JRCSVDataSource(new File(rutaCsv), "UTF-8");</code> → Construye la fuente CSV que alimentará el informe.
Línea 21	<code>dataSource.setFieldDelimiter(',');</code> → Configura la coma como delimitador del CSV.
Línea 22	<code>dataSource.setUseFirstRowAsHeader(true);</code> → Usa la primera fila del CSV como nombres de columnas/campos.
Línea 23	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa de parámetros que se entrega al motor de llenado.
Línea 24	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, dataSource);</code> → Llena el informe y obtiene un JasperPrint en memoria.
Línea 25	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → Exporta el JasperPrint a PDF.
Línea 26	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 27	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 28	<code>System.out.println("Registros CSV esperados: 14");</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 29	<code>} catch (Exception e) {</code> → Captura cualquier fallo de compilación, datos, llenado o exportación.
Línea 30	<code>e.printStackTrace();</code> → Imprime la traza completa para facilitar el diagnóstico.
Línea 31	<code>System.exit(1);</code> → Termina con código distinto de cero para que CI detecte el fallo.
Línea 32	<code>} finally {</code> → Forma parte de la lógica acumulativa del programa.
Línea 33	<code>if (dataSource != null) dataSource.close();</code> → Forma parte de la lógica acumulativa del programa.
Línea 34	<code>}</code> → Cierra el bloque Java actual.
Línea 35	<code>}</code> → Cierra el bloque Java actual.
Línea 36	<code>}</code> → Cierra el bloque Java actual.

Criterio de fallo: todo catch termina con `System.exit(1)` para que una excepción no pueda aparecer como ejecución verde en CI.

Parte D — Simulación del PDF esperado y de la estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

<pre> +-----+ informe_catalogo_csv.jrxml [Design] [Source] +-----+ Ruler: 0 100 200 300 400 500 555 +-----+ Title ----- h = 60 ----- Catálogo Editorial - Datos desde CSV Column Header ----- h = 25 ----- Título Autor Precio Páginas Detail 1 ----- h = 20 ----- [\${titulo}] [\${autor}] [Double.parseDouble(\${precio})] [\${paginas}] Page Footer ----- h = 30 ----- "Página " + \${PAGE_NUMBER} + " de " + [total: \${PAGE_NUMBER}] con evaluationTime=Report]... +-----+ </pre>	
---	--

```

Panel Outline muestra:
Properties
└─ com.jaspersoft.studio.data.defaultdataadapter = CatalogoCSV
Fields
├─ titulo [java.lang.String]
├─ autor [java.lang.String]
├─ precio [java.lang.String]
├─ paginas [java.lang.String]
├─ fecha_publicacion [java.lang.String]
└─ disponible [java.lang.String]

```

Qué representa: la disposición del informe en el editor tras completar los quince pasos. El panel Outline muestra la propiedad del adaptador CSV y los cinco campos declarados con tipo String.

Cómo verificarlo: comparar la vista del editor con este esquema. El panel Outline debe mostrar el nodo Fields con los cinco campos generados automáticamente.

D.2 — Jerarquía del Outline

TEXT

```

informe_catalogo_csv
├─ Properties
│   └─ com.jaspersoft.studio.data.defaultdataadapter = CatalogoCSV
├─ Styles
│   └─ Sans_Normal [isDefault=true]
├─ Fields
│   ├── titulo [java.lang.String]
│   ├── autor [java.lang.String]
│   ├── precio [java.lang.String]
│   ├── paginas [java.lang.String]
│   ├── fecha_publicacion [java.lang.String]
│   └─ disponible [java.lang.String]
├─ Title [band, height=60]
│   └─ staticText "Catálogo Editorial - Datos desde CSV"
├─ Column Header [band, height=25]
│   ├── staticText "Título" (bold)
│   ├── staticText "Autor" (bold)
│   ├── staticText "Precio" (bold, right)
│   └─ staticText "Páginas" (bold, right)
├─ Detail 1 [band, height=20, splitType=Stretch]
│   ├── textField [textAdjust=StretchHeight] ${titulo}
│   ├── textField ${autor}
│   ├── textField [pattern=#,##0.00 €] Double.parseDouble(${F{precio}})
│   └─ textField ${F{paginas}}
├─ Page Footer [band, height=30]
│   └─ textField "Página " + ${PAGE_NUMBER} + " de " + [total: ${PAGE_NUMBER} con evaluationTime=Report] + ...
└─ Background [band, height=0]

```

Qué representa: el árbol de nodos del informe tal como aparece en el panel Outline tras completar los quince pasos. La novedad respecto al punto 3.1 es la ausencia de consulta SQL y la presencia de campos de tipo String que corresponden a las columnas del CSV.

Cómo verificarlo: expandir el nodo informe_catalogo_csv en el panel Outline y comparar la estructura.

D.3 — Documento PDF resultante, página por página

TEXT

```

INFORME: informe_catalogo_csv.pdf
PÁGINAS TOTALES: 1
TAMAÑO DE PÁGINA: 595 × 842 píxeles (A4 vertical)
ORIGEN DE DATOS: data/catalogo.csv (delimitador=',', cabecera=sí, charset=UTF-8)
REGISTROS OBTENIDOS: 14
BANDAS EMITIDAS: Title, Column Header, Detail (14 veces),
                  Page Footer, Background

```

Página 1 de 1

Catálogo Editorial - Datos desde CSV			
Título	Autor	Precio	Páginas
Cien años de soledad	Gabriel G. M.	19,95 €	471
Rayuela	Julio Cortázar	22,50 €	736
La ciudad y los perros	Mario V. Llosa	18,75 €	432
Pedro Páramo	Juan Rulfo	15,90 €	132
Ficciones	Jorge L. Borges	21,00 €	224

La casa de los espíritus	Isabel Allende	23,40 €	448
El amor en los tiempos del cólera	Gabriel G. M.	20,80 €	496
La muerte de Artemio Cruz	Carlos Fuentes	17,60 €	320
Dofa Bárbara	Rómulo Gallegos	16,95 €	400
Martín Fierro	José Hernández	14,50 €	288
Comala	Juan Rulfo	19,20 €	148
Paradiso	José Lezama L.	25,00 €	576
La invención de Morel	Adolfo B. C.	18,30 €	128
El túnel	Ernesto Sábato	16,20 €	160

Página 1 de 1 - Total de registros: 14

Qué representa: la página única del PDF resultante con los catorce libros del CSV. Los datos aparecen en el mismo orden en que están en el archivo porque no se ha declarado ninguna cláusula de ordenación. El campo del precio convierte la cadena a Double y aplica el patrón numérico.

Cómo verificarlo: abrir el archivo `output/informe_catalogo_csv.pdf` con un lector de PDF y comprobar que aparecen los catorce libros. Si el precio no se muestra formateado, revisar la expresión del campo.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├── ECOSISTEMA.md, ENTORNO.md, BANDAS.md, JRXML.md
├── TEXTO.md, CAMPOS.md, IMAGENES.md, ESTILOS.md, EXPRESIONES.md
├── BASEDATOS.md, CSV.md (nuevo)
├── data/
│   └── catalogo.csv (archivo CSV de ejemplo)
├── reports/
│   ├── informe_concepto.jrxml (informe del Módulo 2)
│   ├── informe_concepto.jasper
│   ├── informe_catalogo_csv.jrxml (nuevo informe)
│   └── informe_catalogo_csv.jasper
├── resources/
│   └── (logotipo, iconos y portadas)
├── output/
│   ├── informe_concepto.pdf
│   └── informe_catalogo_csv.pdf (nuevo PDF)
├── EditorialReportsJava/
│   ├── lib/
│   │   └── (6 JAR de JasperReports y SQLite)
│   ├── data/
│   │   └── editorial.db
│   └── src/
│       ├── GeneradorInformeConcepto.java
│       ├── GeneradorCatalogoCSV.java (nueva clase)
│       ├── Libro.java
│       ├── CatalogoDataSource.java
│       └── InicializadorBD.java

```

Qué representa: el estado de los dos proyectos tras completar los quince pasos. La novedad respecto al punto 3.1 es la carpeta `data` del proyecto de informes con el archivo CSV, el informe `informe_catalogo_csv.jrxml`, la clase `GeneradorCatalogoCSV` y el archivo `CSV.md`.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si el archivo `catalogo.csv` no aparece, repetir el paso 2. Si el archivo `CSV.md` no aparece, repetir el paso 15.

Errores comunes del ejercicio completo

Error	Causa	Solución
<code>FileNotFoundException: data/catalogo.csv</code>	El programa se ejecuta desde un directorio distinto a la raíz del proyecto	Configurar el Working Directory en Run Configurations
<code>ClassCastException: java.lang.String cannot be cast to java.lang.Double</code>	El campo <code>precio</code> está declarado como <code>java.lang.Double</code> pero el CSV devuelve cadenas	Declarar el campo como <code>java.lang.String</code> y convertir en la expresión

Error	Causa	Solución
La primera fila de datos se interpreta como cabecera	La casilla Use First Row as Column Names está desmarcada	Marcar la casilla en el adaptador CSV o llamar a <code>setUseFirstRowAsHeader(true)</code>
Las vocales acentuadas aparecen corruptas	El archivo CSV no está guardado en UTF-8	Guardar el archivo como UTF-8 y configurar el charset del adaptador
Field not found: autor	El campo no está declarado en el JRXML	Añadir <code><field name="autor" class="java.lang.String"/></code>
El precio no se formatea con el patrón	El campo devuelve una cadena sin convertir	Convertir en la expresión con <code>Double.parseDouble(\${precio})</code>
El delimitador no se reconoce	El archivo usa punto y coma en lugar de coma	Cambiar el delimitador en el adaptador o en <code>setFieldDelimiter(';')</code>
El adaptador CSV no se conecta en Jaspersoft Studio	La ruta del archivo es relativa a un directorio distinto	Usar una ruta relativa correcta desde el espacio de trabajo
El informe se previsualiza con cero registros	El archivo CSV está vacío o solo contiene la cabecera	Verificar que el archivo contiene las catorce líneas de datos
La columna del autor no aparece en el PDF	El campo no se ha añadido a la banda Detail	Añadir el <code>textField</code> con la expresión <code>\${autor}</code>

Reto resuelto paso a paso

Enunciado: añadir una columna al informe que muestre la fecha de publicación del libro en formato dd/MM/yyyy. La columna debe aparecer a la derecha del campo de páginas.

Paso 1. Antes de añadir la nueva columna, reducir los anchos existentes para liberar los últimos 100 píxeles: Título 205, Autor 125, Precio 70 y Páginas 55; recolocar sus X en 0, 205, 330 y 400 respectivamente.

Paso 2. Hacer doble clic sobre el archivo `informe_catalogo_csv.jrxml` en el panel Project Explorer.

Paso 2. Hacer clic sobre la pestaña Design en la parte inferior del editor central.

Paso 3. Hacer clic sobre el nodo Column Header en el panel Outline.

Paso 4. Hacer clic sobre la pestaña Elements en el panel Palette.

Paso 5. Hacer clic sobre el icono Static Text.

Paso 6. Arrastrar el icono Static Text y soltarlo dentro de la banda Column Header, en la coordenada aproximada x=455, y=5.

Paso 7. Hacer doble clic sobre el Static Text creado en la acción anterior.

Paso 8. Escribir exactamente Fecha publicación.

Paso 9. Hacer clic sobre una zona vacía del editor central para confirmar el texto.

Paso 10. Hacer clic sobre el campo Width en el panel Properties, escribir 100 y pulsar Enter.

Paso 11. Marcar la casilla Bold.

Paso 12. Hacer clic sobre el nodo Detail 1 en el panel Outline.

Paso 13. Hacer clic sobre la pestaña Elements en el panel Palette.

Paso 14. Hacer clic sobre el icono Text Field.

Paso 15. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=455, y=0.

Paso 16. Hacer clic sobre el campo Width en el panel Properties, escribir 100 y pulsar Enter.

Paso 17. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.

Paso 18. Hacer clic sobre el campo Text Field Expression y escribir exactamente `${fecha_publicacion}.substring(8,10) + "/" + ${fecha_publicacion}.substring(5,7) + "/" + ${fecha_publicacion}.substring(0,4)` y pulsar Enter.

Paso 19. Dejar Pattern vacío, porque la expresión ya devuelve el texto con formato dd/MM/yyyy.

Paso 20. Pulsar Ctrl+S para guardar el archivo.

Paso 21. Pulsar Ctrl+Mayús+B para compilar el informe.

Paso 22. Hacer clic con el botón derecho sobre `GeneradorCatalogoCSV.java` y seleccionar Run As > Java Application.

Paso 23. Abrir el archivo `output/informe_catalogo_csv.pdf` y verificar que la nueva columna muestra las fechas en formato dd/MM/yyyy.

Simulación ASCII del PDF tras el reto

Título	Autor	Precio	Páginas	Fecha pub.
Cien años de soledad	Gabriel G. M.	19,95 €	471	05/06/1967
Rayuela	Julio Cortázar	22,50 €	736	28/06/1963
La ciudad y los perros	Mario V. Llosa	18,75 €	432	15/10/1963
...				

Resultado del reto: la nueva columna muestra la fecha de publicación en formato dd/MM/yyyy. La expresión reorganiza directamente el texto ISO yyyy-MM-dd con substring, devuelve un String ya formateado y no necesita SimpleDateFormat ni un patrón de Date.

Analogía final con el contexto de la editorial

El archivo CSV es el listado de libros que la editorial recibe de un distribuidor o que exporta desde una aplicación heredada. Es un formato sencillo que cualquier sistema puede leer y escribir. El adaptador CSV de Jaspersoft Studio es la herramienta que permite consultar ese listado desde la mesa de diseño. La clase JRCsvDataSource es la herramienta equivalente desde la consola de control. El mapeo de columnas a campos es la correspondencia entre los nombres de las columnas del listado y los nombres de los datos que el catálogo espera. La conversión de tipos es el trabajo de adaptar las cadenas del listado a los tipos que el catálogo necesita. El CSV es el formato de intercambio más universal y su integración en el sistema de informes permite aprovechar los datos que llegan en ese formato sin tener que importarlos previamente a una base de datos.

Resultado esperado

Al finalizar este punto, el alumno dispone de:

- El archivo data/catalogo.csv con la cabecera y los catorce libros.
- El adaptador CatalogoCSV en el panel Repository Explorer de Jaspersoft Studio.
- El archivo reports/informe_catalogo_csv.jrxml con los campos generados automáticamente desde el CSV.
- El archivo output/informe_catalogo_csv.pdf con los catorce libros y el precio formateado.
- La clase GeneradorCatalogoCSV.java que lee el CSV desde código Java.
- El archivo CSV.md en la raíz del proyecto con la documentación del uso de CSV.
- Comprensión operativa de la lectura de CSV, del mapeo de columnas a campos y de la conversión de tipos.

Conclusión y enlace al siguiente punto

El punto 3.2 ha introducido la lectura de archivos CSV como fuente de datos para el informe. Han quedado configurados el adaptador CSV en Jaspersoft Studio y la clase JRCsvDataSource en el programa Java. El informe informe_catalogo_csv.jrxml se alimenta del archivo catalogo.csv y muestra los catorce libros con el título, el autor, el precio formateado y el número de páginas. La conversión de tipos desde las cadenas del CSV a los tipos del informe se ha demostrado con el ejemplo del precio.

El punto 3.3, «Ficheros XML», introduce la lectura de datos desde archivos XML y demuestra su uso con un archivo de distribución de libros a librerías. El punto cubre la estructura de un archivo XML, la configuración de adaptadores XML en Jaspersoft Studio y la lectura desde código Java con JRXmlDataSource.

Punto 3.3 — Ficheros XML

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.3 — Ficheros XML

Objetivos de aprendizaje

- Comprender el formato XML y su uso como fuente de datos estructurada.
- Configurar un adaptador XML en Jaspersoft Studio y asociarlo a un informe.
- Utilizar expresiones XPath para seleccionar registros y mapear campos.
- Leer un archivo XML desde código Java con `JRXmlDataSource`.
- Combinar datos jerárquicos de un XML con datos planos de un CSV.
- Documentar el uso de archivos XML en el proyecto EditorialReports.

Parte práctica

Parte A — Práctica visual

Paso 1: Crear el archivo `distribucion.xml`

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta `data` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción `New` en el menú contextual.
3. Hacer clic sobre la opción `File` en el submenú.
4. Escribir exactamente `distribucion.xml` en el campo `File name` del diálogo.
5. Hacer clic sobre el botón `Finish`.
6. En el editor central, escribir exactamente la primera línea: `<?xml version="1.0" encoding="UTF-8"?>` y pulsar `Enter`.
7. Escribir exactamente `<distribucion>` y pulsar `Enter`.
8. Escribir exactamente `<entrega>` y pulsar `Enter`.
9. Escribir exactamente `<libreria>Librería Central</libreria>` y pulsar `Enter`.
10. Escribir exactamente `<ciudad>Madrid</ciudad>` y pulsar `Enter`.
11. Escribir exactamente `<cantidad>25</cantidad>` y pulsar `Enter`.
12. Escribir exactamente `<fecha_entrega>2026-09-15</fecha_entrega>` y pulsar `Enter`.
13. Escribir exactamente `<titulo>Cien años de soledad</titulo>` y pulsar `Enter`.
14. Escribir exactamente `</entrega>` y pulsar `Enter`.
15. Repetir las acciones 8 a 14 para las siguientes entregas:
 - Librería del Prado, Barcelona, 15, 2026-09-16, Rayuela
 - Librería Ateneo, Sevilla, 30, 2026-09-17, La ciudad y los perros
 - Librería Cálamo, Zaragoza, 20, 2026-09-18, Pedro Páramo
 - Librería Proteo, Málaga, 12, 2026-09-19, Ficciones
 - Librería La Central, Barcelona, 18, 2026-09-20, La casa de los espíritus
 - Librería Antonio Machado, Madrid, 22, 2026-09-21, El amor en los tiempos del cólera
 - Librería Cervantes, Valladolid, 8, 2026-09-22, La muerte de Artemio Cruz
16. Escribir exactamente `</distribucion>` y pulsar `Enter`.
17. Pulsar `Ctrl+S` para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `distribucion.xml` dentro de la carpeta `data`. El editor central muestra la estructura completa del XML con el elemento raíz y los ocho elementos `entrega`.

Qué hace: crea el archivo XML con la estructura de distribución a librerías.

Por qué: el archivo XML es la fuente de datos del informe de este punto.

Error común: olvidar la declaración XML en la primera línea y provocar que el motor no reconozca el archivo como XML válido.

Solución: añadir `<?xml version="1.0" encoding="UTF-8"?>` en la primera línea.

Analogía: es como preparar el listado de entregas a librerías en formato estructurado.

Paso 2: Crear el adaptador XML en Jaspersoft Studio

Acciones:

1. Hacer clic con el botón derecho sobre el nodo Data Adapters en el panel Repository Explorer (lateral izquierdo).
2. Hacer clic sobre la opción Create Data Adapter en el menú contextual.
3. Hacer clic sobre el elemento XML File Data Source en la lista de categorías del asistente.
4. Hacer clic sobre el botón Next.
5. Escribir exactamente DistribucionXML en el campo Name.
6. Hacer clic sobre el botón Browse... situado junto al campo File y navegar hasta la carpeta `Documents\JasperProjects\EditorialReports\data`.
7. Hacer clic sobre el archivo `distribucion.xml` y pulsar el botón Open.
8. Hacer clic sobre el campo XPath y escribir exactamente `/distribucion/entrega`.
9. Hacer clic sobre el botón Test Connection.
10. Verificar que el diálogo muestra el mensaje `Connection successful`.
11. Hacer clic sobre el botón Finish.

Verificación visual: el panel Repository Explorer muestra el nodo DistribucionXML colgando de Data Adapters con un icono de archivo XML.

Qué hace: registra un adaptador XML que Jaspersoft Studio utiliza para leer el archivo durante la previsualización.

Por qué: el adaptador permite ejecutar el informe con los datos del XML desde el entorno de diseño.

Error común: olvidar la barra inclinada inicial en la expresión XPath y escribir `distribucion/entrega`. La expresión se interpreta como relativa y no selecciona ningún nodo.

Solución: escribir la expresión con la barra inicial: `/distribucion/entrega`.

Analogía: es como registrar en la editorial el listado de entregas para poder consultarlo durante la composición.

Paso 3: Crear el informe `informe_distribucion_xml.jrxml`

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta `reports` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Jasper Report en el submenú.
4. Hacer clic sobre la plantilla Blank A4 en la lista de plantillas del asistente.
5. Hacer clic sobre el botón Next.
6. Escribir exactamente `informe_distribucion_xml` en el campo File name.
7. Hacer clic sobre el botón Next.
8. Hacer clic sobre DistribucionXML en la lista de adaptadores disponibles.
9. Hacer clic sobre el botón Finish.
10. Observar que el asistente ha generado la consulta XPath y las declaraciones de los campos.

Verificación visual: el editor central muestra el archivo `informe_distribucion_xml.jrxml` con las bandas por defecto. El panel Outline muestra el nodo QueryString con la expresión XPath y el nodo Fields con los cinco campos generados.

Qué hace: crea un nuevo informe a partir del adaptador XML con la consulta XPath y los campos generados.

Por qué: la generación automática evita errores en la expresión XPath y en los nombres de los campos.

Error común: olvidar seleccionar el adaptador correcto.

Solución: cerrar el asistente y repetir el paso seleccionando DistribucionXML.

Analogía: es como abrir un nuevo pliego del catálogo con la estructura del listado de entregas preparada.

Paso 4: Ajustar las bandas del nuevo informe

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `informe_distribucion_xml` en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la opción Delete en el menú contextual para eliminar el nodo Page Header.
3. Hacer clic con el botón derecho sobre el nodo `informe_distribucion_xml` y eliminar el nodo Column Footer.
4. Hacer clic con el botón derecho sobre el nodo `informe_distribucion_xml` y eliminar el nodo Summary.
5. Hacer clic sobre el nodo Title en el panel Outline y ajustar su Band height a 60 píxeles desde el panel Properties.

Verificación visual: el panel Outline muestra solo las bandas Title, Column Header, Detail 1, Page Footer y Background.

Qué hace: simplifica el informe para que contenga solo las bandas necesarias.

Por qué: el informe de distribución necesita un número reducido de bandas.

Error común: eliminar la banda Background pensando que no se usa.

Solución: si se elimina por error, cerrar el archivo sin guardar y volver a abrirlo.

Analogía: es como reducir el pliego del listado de entregas a las secciones estrictamente necesarias.

Paso 5: Añadir el título en la banda Title

Acciones:

1. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
2. Hacer clic sobre el icono Static Text (una letra T mayúscula).
3. Arrastrar el icono Static Text y soltarlo dentro de la banda Title, en la coordenada aproximada x=0, y=15.
4. Hacer doble clic sobre el Static Text creado en la acción anterior.
5. Escribir exactamente `Distribución Editorial - Datos desde XML`.
6. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
7. Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 0 y pulsar Enter.
8. Hacer clic sobre el campo Y, escribir 15 y pulsar Enter.
9. Hacer clic sobre el campo Width, escribir 555 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 30 y pulsar Enter.
11. Hacer clic sobre el campo Font size y escribir 18. Pulsar Enter.
12. Marcar la casilla Bold.
13. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.

Verificación visual: la banda Title muestra el texto `Distribución Editorial - Datos desde XML` centrado y en negrita.

Qué hace: inserta el título del informe de distribución desde XML.

Por qué: el título identifica el documento y su origen de datos.

Error común: olvidar el centrado y provocar que el título aparezca alineado a la izquierda.

Solución: seleccionar Center en el desplegable Horizontal Text Alignment.

Analogía: es como titular el listado de entregas con el nombre de la editorial.

Paso 6: Añadir los encabezados de columna**Acciones:**

1. Hacer clic sobre el nodo Column Header en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Static Text.
4. Arrastrar el icono Static Text y soltarlo dentro de la banda Column Header, en la coordenada aproximada x=0, y=5.
5. Hacer doble clic sobre el Static Text creado en la acción anterior.
6. Escribir exactamente **Librería**.
7. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
8. Hacer clic sobre el campo Width en el panel Properties, escribir 180 y pulsar Enter.
9. Marcar la casilla Bold.
10. Repetir las acciones 3 a 9 para los encabezados Ciudad (x=180, ancho 120), Cantidad (x=300, ancho 80), Fecha (x=380, ancho 90) y Título (x=470, ancho 85).

Verificación visual: la banda Column Header muestra los cinco encabezados **Librería**, **Ciudad**, **Cantidad**, **Fecha** y **Título** en negrita.

Qué hace: inserta los encabezados de las columnas del informe de distribución.

Por qué: los encabezados identifican las columnas de la tabla del listado.

Error común: dejar los encabezados sin negrita y provocar que no se distingan del cuerpo.

Solución: marcar la casilla Bold.

Analogía: es como añadir los títulos de las columnas al listado de entregas.

Paso 7: Añadir los campos en la banda Detail**Acciones:**

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).
4. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=0, y=0.
5. Hacer clic sobre el campo Width en el panel Properties, pestaña Properties, escribir 180 y pulsar Enter.
6. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.
7. Hacer clic sobre el campo Text Field Expression y escribir exactamente `$F{libreria}` y pulsar Enter.
8. Repetir las acciones 3 a 7 para los campos ciudad (x=180, ancho 120), cantidad (x=300, ancho 80) y fecha_entrega (x=380, ancho 90).
9. Para el campo del título del libro, hacer clic sobre el icono Text Field y arrastrarlo dentro de la banda Detail 1, en la coordenada aproximada x=470, y=0.
10. Hacer clic sobre el campo Width y escribir 85. Pulsar Enter.
11. Hacer clic sobre el campo Height y escribir 20. Pulsar Enter.
12. Hacer clic sobre el campo Text Field Expression y escribir exactamente `$F{titulo}` y pulsar Enter.

Verificación visual: la banda Detail 1 muestra cinco campos con las expresiones correspondientes.

Qué hace: inserta los campos que se imprimen para cada entrega del XML.

Por qué: los campos resuelven los valores de los nodos del XML para cada registro.

Error común: usar `$F{librería}` con tilde en la expresión. El nombre del campo en el XML es `libreria` sin tilde y el motor no encuentra el campo.

Solución: usar el nombre exacto sin tilde.

Analogía: es como rellenar las celdas del listado de entregas con los datos de cada librería.

Paso 8: Ajustar la altura de la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 20 y pulsar Enter.
3. Hacer clic sobre el desplegable Split Type en el panel Properties y verificar que está en Stretch.

Verificación visual: la banda Detail 1 aparece con 20 píxeles de altura.

Qué hace: fija la altura de la banda Detail para que cada entrega ocupe una fila.

Por qué: la altura determina el espacio de cada fila y el número de filas que caben en una página.

Error común: dejar la altura por defecto y provocar que el informe ocupe más páginas de las necesarias.

Solución: ajustar la altura a 20 píxeles.

Analogía: es como ajustar la altura de cada fila del listado de entregas para que todas quepan en una página.

Paso 9: Añadir el pie de página con totales

Acciones:

1. Hacer clic sobre el nodo Page Footer en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 40 y pulsar Enter.
3. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
4. Hacer clic sobre el icono Static Text.
5. Arrastrar el icono Static Text y soltarlo dentro de la banda Page Footer, en la coordenada aproximada x=0, y=5.
6. Hacer doble clic sobre el Static Text creado en la acción anterior.
7. Escribir exactamente Total de entregas:.
8. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
9. Hacer clic sobre el campo Width en el panel Properties, escribir 150 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
11. Hacer clic sobre la pestaña Elements en el panel Palette.
12. Hacer clic sobre el icono Text Field.
13. Arrastrar el icono Text Field y soltarlo dentro de la banda Page Footer, a la derecha del rótulo, en la coordenada aproximada x=150, y=5.
14. Hacer clic sobre el campo Width y escribir 50. Pulsar Enter.
15. Hacer clic sobre el campo Height y escribir 15. Pulsar Enter.
16. Hacer clic sobre el campo Text Field Expression y escribir exactamente \$V{REPORT_COUNT} y pulsar Enter.
17. Marcar la casilla Bold.

Verificación visual: la banda Page Footer muestra el rótulo Total de entregas: seguido del campo con la expresión \$V{REPORT_COUNT} en negrita.

Qué hace: inserta un pie de página con el recuento total de entregas.

Por qué: el recuento total es un dato agregado que informa del volumen de distribución.

Error común: usar \$P{REPORT_COUNT} en lugar de \$V{REPORT_COUNT}.

Solución: cambiar el prefijo a \$V{.

Analogía: es como anotar al pie del listado de entregas cuántas entregas se han registrado.

Paso 10: Añadir la paginación correcta bajo el total de entregas

Acciones:

1. Mantener Page Footer con altura 45.
2. Conservar en la primera fila Total de entregas: y `{REPORT_COUNT}`.
3. Añadir un Text Field en x=170, y=23, ancho=190, alto=15.
4. Escribir "Página " + `{PAGE_NUMBER}` + " de " y alinear a la derecha.
5. Añadir un segundo Text Field en x=365, y=23, ancho=30, alto=15.
6. Escribir `{PAGE_NUMBER}` como expresión.
7. En Properties > Text Field, seleccionar Evaluation Time = Report para el segundo campo.
8. Fijar tamaño 9 y guardar.

Verificación visual: el pie muestra Página N de M; en Source el segundo Text Field usa `evaluationTime="Report"`.

Qué hace: resuelve la paginación con las variables correctas de JasperReports 6.20.0.

Por qué: `{PAGE_COUNT}` cuenta registros de la página, no páginas totales.

Error común: usar `PAGE_COUNT` como denominador.

Solución: `PAGE_NUMBER` con evaluación Report.

Analogía: el total definitivo se conoce cuando la tirada completa ha terminado.

****Paso 11:**

Paso 11: Compilar y previsualizar con el adaptador XML**Acciones:**

1. Pulsar Ctrl+S para guardar el archivo.
2. Pulsar Ctrl+Mayús+B para compilar el informe.
3. Hacer clic sobre el panel Problems (inferior) y verificar que no hay errores.
4. Pulsar el botón Preview de la barra de herramientas superior.
5. En el diálogo, verificar que el adaptador `DistribucionXML` está seleccionado.
6. Hacer clic sobre el botón OK.
7. Esperar a que se abra la pestaña Preview en el editor central.

Verificación visual: la pestaña Preview muestra el informe con las ocho entregas del XML. Cada fila muestra la librería, la ciudad, la cantidad, la fecha y el título del libro.

Qué hace: ejecuta el informe con los datos del XML y muestra el resultado.

Por qué: la previsualización confirma que el adaptador XML funciona y que las expresiones XPath se resuelven correctamente.

Error común: obtener `JRException: XPath expression failed`. Indica que la expresión XPath es incorrecta.

Solución: revisar la expresión y verificar que la barra inclinada inicial está presente.

Analogía: es como revisar la prueba de color del listado de entregas con los datos del archivo XML.

Paso 12: Crear la clase GeneradorDistribucionXML**Acciones:**

1. Hacer clic con el botón derecho sobre la carpeta src en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Class en el submenú.
4. Escribir exactamente `GeneradorDistribucionXML` en el campo Name del diálogo.
5. Marcar la casilla `public static void main(String[] args)`.
6. Hacer clic sobre el botón Finish.

7. En el editor central, escribir el código completo de la clase `GeneradorDistribucionXML` que se muestra en la Parte C de este punto.
8. Pulsar `Ctrl+S` para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `GeneradorDistribucionXML.java` dentro de la carpeta `src`.

Qué hace: crea la clase que genera el informe de distribución desde el archivo XML.

Por qué: el informe puede ejecutarse desde código Java sin depender de Jaspersoft Studio.

Error común: olvidar importar `net.sf.jasperreports.engine.data.JRXmlDataSource`. El compilador informa `cannot find symbol`.

Solución: añadir la importación correspondiente.

Analogía: es como preparar la consola de control para que el operario genere el listado de entregas desde el archivo XML.

Paso 13: Ejecutar el programa y verificar el PDF

Acciones:

1. Hacer clic con el botón derecho sobre el archivo `GeneradorDistribucionXML.java` en el panel Project Explorer.
2. Hacer clic sobre la opción `Run As` en el menú contextual.
3. Hacer clic sobre la opción `Java Application` en el submenú.
4. Hacer clic sobre la vista `Console` en el panel inferior y observar el resultado.
5. Abrir el explorador de archivos del sistema operativo.
6. Navegar hasta la carpeta `output` del proyecto `EditorialReports`.
7. Hacer doble clic sobre el archivo `informe_distribucion_xml.pdf`.
8. Verificar que el PDF muestra las ocho entregas con la librería, la ciudad, la cantidad, la fecha y el título.

Verificación visual: la vista `Console` muestra la línea `Informe generado en: ...` con la ruta absoluta del PDF. El archivo PDF muestra las ocho entregas del XML.

Qué hace: ejecuta el programa Java que lee el XML y genera el informe.

Por qué: la ejecución confirma que el programa lee el XML y genera el informe correctamente.

Error común: ejecutar el programa desde un directorio distinto a la raíz del proyecto y obtener `FileNotFoundException: data/distribucion.xml`.

Solución: comprobar en `Run Configurations` que el `Working Directory` apunta a la raíz del proyecto.

Analogía: es como imprimir el listado de entregas desde el archivo XML.

Paso 14: Documentar el uso de archivos XML

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `EditorialReports` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción `New` en el menú contextual.
3. Hacer clic sobre la opción `File` en el submenú.
4. Escribir exactamente `XML.md` en el campo `File name` del diálogo.
5. Hacer clic sobre el botón `Finish`.
6. En el editor central, escribir exactamente `# Ficheros XML` y pulsar `Enter` dos veces.
7. Escribir exactamente `## Archivos del proyecto` y pulsar `Enter` dos veces.
8. Escribir exactamente `- data/distribucion.xml: entregas a librerías con librería, ciudad, cantidad, fecha y título.` y pulsar `Enter` dos veces.
9. Escribir exactamente `## Adaptador de Jaspersoft Studio` y pulsar `Enter` dos veces.
10. Escribir exactamente `- Nombre: DistribucionXML` y pulsar `Enter`.

11. Escribir exactamente - Expresión XPath de selección: /distribucion/entrega y pulsar Enter dos veces.
12. Escribir exactamente ## Lectura desde Java y pulsar Enter dos veces.
13. Escribir exactamente - Clase: net.sf.jasperreports.engine.data.JRXmlDataSource y pulsar Enter dos veces.
14. Escribir exactamente ## Expresiones XPath de los campos y pulsar Enter dos veces.
15. Escribir exactamente - libreria, ciudad, cantidad, fecha_entrega, titulo y pulsar Enter.
16. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo XML .md en la raíz del proyecto EditorialReports.

Qué hace: incorpora al proyecto un documento que registra el uso de archivos XML.

Por qué: la documentación de las fuentes de datos facilita el mantenimiento y la incorporación de nuevos desarrolladores.

Error común: olvidar documentar la expresión XPath de selección.

Solución: incluir las secciones especificadas.

Analogía: es como dejar en la editorial una ficha técnica con la ubicación del listado de entregas y su estructura.

Parte B — JRXML completo explicado línea por línea [VALIDADO]

JRXML canónico del checkpoint. Es el mismo archivo que se compila en GitHub Actions.

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_distribucion_xml"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
  leftMargin="20"
  rightMargin="20"
  topMargin="20"
  bottomMargin="20"
  uuid="b4e5f6a7-c8d9-0e1f-2a3b-4c5d6e7f8a9b">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="DistribucionXML"/>
  <style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
  <queryString language="XPath">
    <![CDATA[/distribucion/entrega]]>
  </queryString>
  <field name="libreria" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="libreria"/></field>
  <field name="ciudad" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="ciudad"/></field>
  <field name="cantidad" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="cantidad"/></field>
  <field name="fecha_entrega" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression"
value="fecha_entrega"/></field>
  <field name="titulo" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="titulo"/></field>
  <background>
    <band height="0"/>
  </background>

  <title>
    <band height="60">
      <staticText>
        <reportElement x="0" y="15" width="555" height="30" uuid="c5f6a7b8-d9e0-1f2a-3b4c-5d6e7f8a9b0c"/>
        <textElement textAlignment="Center" verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="18" isBold="true"/>
        </textElement>
        <text><![CDATA[Distribución Editorial - Datos desde XML]]></text>
      </staticText>
    </band>
  </title>
  <columnHeader>
    <band height="25">
      <staticText>
        <reportElement x="0" y="5" width="180" height="15" uuid="d6a7b8c9-e0f1-2a3b-4c5d-6e7f8a9b0c1d"/>
        <textElement verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Librería]]></text>
      </staticText>
      <staticText>
```

XML

```

        <reportElement x="180" y="5" width="120" height="15" uuid="e7b8c9d0-f1a2-3b4c-5d6e-7f8a9b0c1d2e"/>
        <textElement verticalAlignment="Middle">
            <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Ciudad]]></text>
    </staticText>
    <staticText>
        <reportElement x="300" y="5" width="80" height="15" uuid="f8c9d0e1-a2b3-4c5d-6e7f-8a9b0c1d2e3f"/>
        <textElement textAlignment="Right" verticalAlignment="Middle">
            <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Cantidad]]></text>
    </staticText>
    <staticText>
        <reportElement x="380" y="5" width="90" height="15" uuid="a9d0e1f2-b3c4-5d6e-7f8a-9b0c1d2e3f4a"/>
        <textElement textAlignment="Center" verticalAlignment="Middle">
            <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Fecha]]></text>
    </staticText>
    <staticText>
        <reportElement x="470" y="5" width="85" height="15" uuid="b0e1f2a3-c4d5-6e7f-8a9b-0c1d2e3f4a5b"/>
        <textElement verticalAlignment="Middle">
            <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Título]]></text>
    </staticText>
</band>
</columnHeader>
<detail>
    <band height="20" splitType="Stretch">
        <textField textAdjust="StretchHeight">
            <reportElement x="0" y="0" width="180" height="20" uuid="c1f2a3b4-d5e6-7f8a-9b0c-1d2e3f4a5b6c"/>
            <textElement verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F(libreria)}]]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="180" y="0" width="120" height="20" uuid="d2a3b4c5-e6f7-8a9b-0c1d-2e3f4a5b6c7d"/>
            <textElement verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F(ciudad)}]]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="300" y="0" width="80" height="20" uuid="e3b4c5d6-f7a8-9b0c-1d2e-3f4a5b6c7d8e"/>
            <textElement textAlignment="Right" verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F(cantidad)}]]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="380" y="0" width="90" height="20" uuid="f4c5d6e7-a8b9-0c1d-2e3f-4a5b6c7d8e9f"/>
            <textElement textAlignment="Center" verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F{fecha_entrega}.substring(8,10)} + "/" + ${F{fecha_entrega}.substring(5,7)} + "/" +
${F{fecha_entrega}.substring(0,4)}]]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="470" y="0" width="85" height="20" uuid="a5d6e7f8-b9c0-1d2e-3f4a-5b6c7d8e9f0a"/>
            <textElement verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F{titulo}}]]></textFieldExpression>
        </textField>
    </band>
</detail>
<pageFooter>
    <band height="45">
        <staticText>
            <reportElement x="0" y="3" width="150" height="15" uuid="22222222-2222-4222-8222-222222222221"/>
            <textElement verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
            <text><![CDATA[Total de entregas:]]></text>
        </staticText>
        <textField>
            <reportElement x="150" y="3" width="70" height="15" uuid="22222222-2222-4222-8222-222222222222"/>
            <textElement verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9" isBold="true"/></textElement>
            <textFieldExpression><![CDATA[${V{REPORT_COUNT}}]]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="170" y="23" width="190" height="15" uuid="22222222-2222-4222-8222-222222222223"/>
            <textElement textAlignment="Right" verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
            <textFieldExpression><![CDATA["Página " + ${V{PAGE_NUMBER}} + " de"]></textFieldExpression>
        </textField>
        <textField evaluationTime="Report">
            <reportElement x="365" y="23" width="30" height="15" uuid="22222222-2222-4222-8222-222222222224"/>
            <textElement textAlignment="Left" verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
            <textFieldExpression><![CDATA[${V{PAGE_NUMBER}}]]></textFieldExpression>
        </textField>
    </band>

```

```
</pageFooter>
</jasperReport>
```

Explicación línea por línea

Línea 1	<code><?xml version="1.0" encoding="UTF-8"?></code> → Declara el documento XML y la codificación UTF-8.
Línea 2	<code><jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"</code> → Abre la plantilla JasperReports y define sus atributos principales.
Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"</code> → Completa la definición declarativa del informe.
Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Declara el espacio de nombres/XSD usado para validar el JRXML.
Línea 5	<code>name="informe_distribucion_xml"</code> → Completa la definición declarativa del informe.
Línea 6	<code>language="java"</code> → Completa la definición declarativa del informe.
Línea 7	<code>pageWidth="595"</code> → Completa la definición declarativa del informe.
Línea 8	<code>pageHeight="842"</code> → Completa la definición declarativa del informe.
Línea 9	<code>columnWidth="555"</code> → Completa la definición declarativa del informe.
Línea 10	<code>leftMargin="20"</code> → Completa la definición declarativa del informe.
Línea 11	<code>rightMargin="20"</code> → Completa la definición declarativa del informe.
Línea 12	<code>topMargin="20"</code> → Completa la definición declarativa del informe.
Línea 13	<code>bottomMargin="20"</code> → Completa la definición declarativa del informe.
Línea 14	<code>uuid="b4e5f6a7-c8d9-0e1f-2a3b-4c5d6e7f8a9b"></code> → Completa la definición declarativa del informe.
Línea 15	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="DistribucionXML"/></code> → Asocia el Data Adapter usado por Jaspersoft Studio durante Preview.
Línea 16	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/></code> → Declara un estilo compatible con JasperReports 6.20.0.
Línea 17	<code><queryString language="XPath"></code> → Abre la consulta del dataset e indica el lenguaje de consulta.
Línea 18	<code><![CDATA[/distribucion/entrega]]></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 19	<code></queryString></code> → Cierra el elemento XML abierto anteriormente.
Línea 20	<code><field name="libreria" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="libreria"/></field></code> → Declara la expresión de mapeo del campo para el origen jerárquico.
Línea 21	<code><field name="ciudad" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="ciudad"/></field></code> → Declara la expresión de mapeo del campo para el origen jerárquico.
Línea 22	<code><field name="cantidad" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="cantidad"/></field></code> → Declara la expresión de mapeo del campo para el origen jerárquico.
Línea 23	<code><field name="fecha_entrega" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="fecha_entrega"/></field></code> → Declara la expresión de mapeo del campo para el origen jerárquico.
Línea 24	<code><field name="titulo" class="java.lang.String"><property name="net.sf.jasperreports.xpath.field.expression" value="titulo"/></field></code> → Declara la expresión de mapeo del campo para el origen jerárquico.

Línea 25	<code><background></code> → Abre una sección/banda estructural del informe.
Línea 26	<code><band height="0"/></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 27	<code></background></code> → Cierra el elemento XML abierto anteriormente.
Línea 28	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 29	<code><title></code> → Abre una sección/banda estructural del informe.
Línea 30	<code><band height="60"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 31	<code><staticText></code> → Abre un elemento de texto estático.
Línea 32	<code><reportElement x="0" y="15" width="555" height="30" uuid="c5f6a7b8-d9e0-1f2a-3b4c-5d6e7f8a9b0c"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 33	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 34	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 35	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 36	<code><text><![CDATA[Distribución Editorial - Datos desde XML]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 37	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 38	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 39	<code></title></code> → Cierra el elemento XML abierto anteriormente.
Línea 40	<code><columnHeader></code> → Abre una sección/banda estructural del informe.
Línea 41	<code><band height="25"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 42	<code><staticText></code> → Abre un elemento de texto estático.
Línea 43	<code><reportElement x="0" y="5" width="180" height="15" uuid="d6a7b8c9-e0f1-2a3b-4c5d-6e7f8a9b0c1d"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 44	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 45	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 46	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 47	<code><text><![CDATA[Librería]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 48	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 49	<code><staticText></code> → Abre un elemento de texto estático.
Línea 50	<code><reportElement x="180" y="5" width="120" height="15" uuid="e7b8c9d0-f1a2-3b4c-5d6e-7f8a9b0c1d2e"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 51	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 52	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 53	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 54	<code><text><![CDATA[Ciudad]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.

Línea 55	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 56	<code><staticText></code> → Abre un elemento de texto estático.
Línea 57	<code><reportElement x="300" y="5" width="80" height="15" uuid="f8c9d0e1-a2b3-4c5d-6e7f-8a9b0c1d2e3f" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 58	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 59	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 60	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 61	<code><text><![CDATA[Cantidad]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 62	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 63	<code><staticText></code> → Abre un elemento de texto estático.
Línea 64	<code><reportElement x="380" y="5" width="90" height="15" uuid="a9d0e1f2-b3c4-5d6e-7f8a-9b0c1d2e3f4a" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 65	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 66	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 67	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 68	<code><text><![CDATA[Fecha]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 69	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 70	<code><staticText></code> → Abre un elemento de texto estático.
Línea 71	<code><reportElement x="470" y="5" width="85" height="15" uuid="b0e1f2a3-c4d5-6e7f-8a9b-0c1d2e3f4a5b" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 72	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 73	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 74	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 75	<code><text><![CDATA[Título]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 76	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 77	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 78	<code></columnHeader></code> → Cierra el elemento XML abierto anteriormente.
Línea 79	<code><detail></code> → Abre una sección/banda estructural del informe.
Línea 80	<code><band height="20" splitType="Stretch"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 81	<code><textField textAdjust="StretchHeight"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 82	<code><reportElement x="0" y="0" width="180" height="20" uuid="c1f2a3b4-d5e6-7f8a-9b0c-1d2e3f4a5b6c" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.

Línea 83	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 84	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 85	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 86	<code><textFieldExpression><![CDATA[\${libreria}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 87	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 88	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 89	<code><reportElement x="180" y="0" width="120" height="20" uuid="d2a3b4c5-e6f7-8a9b-0c1d-2e3f4a5b6c7d"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 90	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 91	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 92	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 93	<code><textFieldExpression><![CDATA[\${ciudad}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 94	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 95	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 96	<code><reportElement x="300" y="0" width="80" height="20" uuid="e3b4c5d6-f7a8-9b0c-1d2e-3f4a5b6c7d8e"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 97	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 98	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 99	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 100	<code><textFieldExpression><![CDATA[\${cantidad}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 101	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 102	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 103	<code><reportElement x="380" y="0" width="90" height="20" uuid="f4c5d6e7-a8b9-0c1d-2e3f-4a5b6c7d8e9f"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 104	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 105	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 106	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 107	<code><textFieldExpression><![CDATA[\${fecha_entrega}.substring(8,10) + "/" + \${fecha_entrega}.substring(5,7) + "/" + \${fecha_entrega}.substring(0,4)}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 108	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 109	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.

Línea 110	<code><reportElement x="470" y="0" width="85" height="20" uuid="a5d6e7f8-b9c0-1d2e-3f4a-5b6c7d8e9f0a" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 111	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 112	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 113	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 114	<code><textFieldExpression><![CDATA[\${titulo}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 115	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 116	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 117	<code></detail></code> → Cierra el elemento XML abierto anteriormente.
Línea 118	<code><pageFooter></code> → Abre una sección/banda estructural del informe.
Línea 119	<code><band height="45"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 120	<code><staticText></code> → Abre un elemento de texto estático.
Línea 121	<code><reportElement x="0" y="3" width="150" height="15" uuid="22222222-2222-4222-8222-222222222221" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 122	<code><textElement verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 123	<code><text><![CDATA[Total de entregas:]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 124	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 125	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 126	<code><reportElement x="150" y="3" width="70" height="15" uuid="22222222-2222-4222-8222-222222222222" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 127	<code><textElement verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 128	<code><textFieldExpression><![CDATA[\${REPORT_COUNT}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 129	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 130	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 131	<code><reportElement x="170" y="23" width="190" height="15" uuid="22222222-2222-4222-8222-222222222223" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 132	<code><textElement textAlignment="Right" verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 133	<code><textFieldExpression><![CDATA["Página " + \${PAGE_NUMBER} + " de"]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 134	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 135	<code><textField evaluationTime="Report"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.

Línea 136	<code><reportElement x="365" y="23" width="30" height="15" uuid="22222222-2222-4222-8222-222222222224"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 137	<code><textElement textAlignment="Left" verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 138	<code><textFieldExpression><![CDATA[\${PAGE_NUMBER}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 139	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 140	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 141	<code></pageFooter></code> → Cierra el elemento XML abierto anteriormente.
Línea 142	<code></jasperReport></code> → Cierra el elemento XML abierto anteriormente.

Comprobación: las coordenadas se mantienen dentro de `columnWidth="555"`, el orden estructural es compatible con JasperReports 6.20.0 y no se usa sintaxis retirada de la baseline.

Parte C — Código Java explicado línea por línea [VALIDADO]

El código siguiente es el código real incluido en el checkpoint y ejecutado por el workflow E2E. Java no redibuja el informe: **compila -> llena -> exporta**.

Clase GeneradorDistribucionXML.java

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.data.JRXmlDataSource;

public class GeneradorDistribucionXML {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_distribucion_xml.jrxml";
            String rutaJasper = "reports/informe_distribucion_xml.jasper";
            String rutaPdf = "output/informe_distribucion_xml.pdf";
            String rutaXml = "data/distribucion.xml";
            new File("output").mkdirs();
            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
            JRXmlDataSource dataSource = new JRXmlDataSource(rutaXml, "/distribucion/entrega");
            Map<String, Object> parametros = new HashMap<String, Object>();
            JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, dataSource);
            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);
            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
            System.out.println("Entregas XML esperadas: 8");
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 2	<code>import java.util.HashMap;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 3	<code>import java.util.Map;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 4	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 5	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.

Línea 6	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 7	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 8	<code>import net.sf.jasperreports.engine.data.JRXmlDataSource;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 9	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 10	<code>public class GeneradorDistribucionXML {</code> → Declara la clase Java del checkpoint.
Línea 11	<code>public static void main(String[] args) {</code> → Declara el punto de entrada ejecutable.
Línea 12	<code>try {</code> → Abre un bloque protegido; si contiene recursos, se cerrarán automáticamente.
Línea 13	<code>String rutaJrxml = "reports/informe_distribucion_xml.jrxml";</code> → Fija la ruta del JRXML desde el Working Directory EditorialReports.
Línea 14	<code>String rutaJasper = "reports/informe_distribucion_xml.jasper";</code> → Fija la ruta del artefacto <code>.jasper</code> compilado.
Línea 15	<code>String rutaPdf = "output/informe_distribucion_xml.pdf";</code> → Fija la ruta del PDF que se exportará.
Línea 16	<code>String rutaXml = "data/distribucion.xml";</code> → Forma parte de la lógica acumulativa del programa.
Línea 17	<code>new File("output").mkdirs();</code> → Crea el directorio necesario antes de escribir datos o salidas.
Línea 18	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML a <code>.jasper</code> con JasperReports 6.20.0.
Línea 19	<code>JRXmlDataSource dataSource = new JRXmlDataSource(rutaXml, "/distribucion/entrega");</code> → Construye la fuente XML y aplica la XPath de selección de registros.
Línea 20	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa de parámetros que se entrega al motor de llenado.
Línea 21	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, dataSource);</code> → Llena el informe y obtiene un <code>JasperPrint</code> en memoria.
Línea 22	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → Exporta el <code>JasperPrint</code> a PDF.
Línea 23	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 24	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 25	<code>System.out.println("Entregas XML esperadas: 8");</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 26	<code>} catch (Exception e) {</code> → Captura cualquier fallo de compilación, datos, llenado o exportación.
Línea 27	<code>e.printStackTrace();</code> → Imprime la traza completa para facilitar el diagnóstico.
Línea 28	<code>System.exit(1);</code> → Termina con código distinto de cero para que CI detecte el fallo.
Línea 29	<code>}</code> → Cierra el bloque Java actual.
Línea 30	<code>}</code> → Cierra el bloque Java actual.
Línea 31	<code>}</code> → Cierra el bloque Java actual.

Criterio de fallo: todo `catch` termina con `System.exit(1)` para que una excepción no pueda aparecer como ejecución verde en CI.

Parte D — Simulación del PDF esperado y de la estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT



Qué representa: la disposición del informe en el editor tras completar los catorce pasos. El panel Outline muestra la consulta XPath y los cinco campos.

Cómo verificarlo: comparar la vista del editor con este esquema. El panel Outline debe mostrar el nodo QueryString con la expresión /distribucion/entrega.

D.2 — Jerarquía del Outline

TEXT

```
informe_distribucion_xml
├── Properties
│   └── com.jaspersoft.studio.data.defaultdataadapter = DistribucionXML
├── Styles
│   └── Sans_Normal [isDefault=true]
├── QueryString
│   └── /distribucion/entrega [language=xPath]
├── Fields
│   ├── libreria [java.lang.String]
│   ├── ciudad [java.lang.String]
│   ├── cantidad [java.lang.String]
│   ├── fecha_entrega [java.lang.String]
│   └── titulo [java.lang.String]
├── Title [band, height=60]
│   └── staticText "Distribución Editorial - Datos desde XML"
├── Column Header [band, height=25]
│   ├── staticText "Librería" (bold)
│   ├── staticText "Ciudad" (bold)
│   ├── staticText "Cantidad" (bold, right)
│   ├── staticText "Fecha" (bold, center)
│   └── staticText "Titulo" (bold)
├── Detail 1 [band, height=20, splitType=Stretch]
│   ├── textField [textAdjust=StretchHeight] ${libreria}
│   ├── textField ${ciudad}
│   ├── textField [right] ${cantidad}
│   ├── textField [center] texto ISO reordenado a dd/MM/yyyy mediante substring
│   └── textField ${titulo}
└── Page Footer [band, height=40]
```

```

├─ staticText "Total de entregas: "
├─ textField [bold] ${REPORT_COUNT}
├─ textField [center] "Página " + ${PAGE_NUMBER} + " de " + [total: ${PAGE_NUMBER} con evaluationTime=Report]
└─ Background [band, height=0]

```

Qué representa: el árbol de nodos del informe tal como aparece en el panel Outline tras completar los catorce pasos. La novedad respecto al punto 3.2 es el nodo QueryString con la expresión XPath.

Cómo verificarlo: expandir el nodo informe_distribucion_xml en el panel Outline y comparar la estructura.

D.3 — Documento PDF resultante, página por página

TEXT

INFORME: informe_distribucion_xml.pdf
 PÁGINAS TOTALES: 1
 TAMAÑO DE PÁGINA: 595 × 842 píxeles (A4 vertical)
 ORIGEN DE DATOS: data/distribucion.xml (XPath: /distribucion/entrega)
 NODOS SELECCIONADOS: 8
 BANDAS EMITIDAS: Title, Column Header, Detail (8 veces),
 Page Footer, Background

————— Página 1 de 1 —————

Distribución Editorial - Datos desde XML				
Librería	Ciudad	Cantidad	Fecha	Tít.
Librería Central	Madrid	25	15/09/26	Cien
Librería del Prado	Barcelona	15	16/09/26	Rayu
Librería Ateneo	Sevilla	30	17/09/26	La c
Librería Cálamo	Zaragoza	20	18/09/26	Pedr
Librería Proteo	Málaga	12	19/09/26	Ficc
Librería La Central	Barcelona	18	20/09/26	La c
Librería A. Machado	Madrid	22	21/09/26	El a
Total de entregas: 8				
Página 1 de 1				

Qué representa: la página única del PDF resultante con las ocho entregas del XML. Los datos aparecen en el orden en que están en el archivo porque la expresión XPath no incluye ordenación. La columna de fecha muestra la fecha formateada como dd/MM/yyyy.

Cómo verificarlo: abrir el archivo output/informe_distribucion_xml.pdf con un lector de PDF y comprobar que aparecen las ocho entregas con la librería, la ciudad, la cantidad, la fecha y el título.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├─ ECOSISTEMA.md, ENTORNO.md, BANDAS.md, JRXML.md
├─ TEXTO.md, CAMPOS.md, IMAGENES.md, ESTILOS.md, EXPRESIONES.md
├─ BASEDATOS.md, CSV.md, XML.md (nuevo)
├─ data/
│  └─ catalogo.csv
│     └─ distribucion.xml (nuevo)
├─ reports/
│  └─ informe_concepto.jrxml
│     └─ informe_catalogo_csv.jrxml
│        └─ informe_distribucion_xml.jrxml (nuevo)
├─ resources/
│  └─ (logotipo, iconos y portadas)
├─ output/
│  └─ informe_concepto.pdf
│     └─ informe_catalogo_csv.pdf
│        └─ informe_distribucion_xml.pdf (nuevo)
└─ EditorialReportsJava/
   └─ lib/
      └─ (6 JAR de JasperReports y SQLite)
   └─ data/
      └─ editorial.db
   └─ src/
      └─ GeneradorInformeConcepto.java
         └─ GeneradorCatalogoCSV.java

```

```

├── GeneradorDistribucionXML.java      (nueva clase)
├── Libro.java
├── CatalogoDataSource.java
└── InicializadorBD.java

```

Qué representa: el estado de los dos proyectos tras completar los catorce pasos. La novedad respecto al punto 3.2 es el archivo `distribucion.xml`, el informe `informe_distribucion.xml.jrxml`, la clase `GeneradorDistribucionXML` y el archivo `XML.md`.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si el archivo `distribucion.xml` no aparece, repetir el paso 1. Si el archivo `XML.md` no aparece, repetir el paso 14.

Errores comunes del ejercicio completo

Error	Causa	Solución
<code>FileNotFoundException: data/distribucion.xml</code>	El programa se ejecuta desde un directorio distinto a la raíz del proyecto	Configurar el Working Directory en Run Configurations
<code>JRException: XPath expression failed</code>	La expresión XPath de selección es incorrecta	Verificar que la expresión comienza por <code>/</code> y que los nombres coinciden con el XML
Field not found: libreria	El campo no está declarado o el nombre no coincide con el elemento XML	Añadir <code><field name="libreria" class="java.lang.String" /></code>
La fecha no se formatea correctamente	El patrón no coincide con la expresión	Reordenar el texto ISO con <code>substring</code> o convertir explícitamente con una estrategia que gestione errores
<code>ClassCastException</code> al resolver un campo	El tipo declarado no coincide con el valor devuelto	Declarar todos los campos del XML como <code>java.lang.String</code>
El adaptador XML no se conecta en JasperSoft Studio	La expresión XPath es relativa y no selecciona nodos	Usar una expresión XPath absoluta que comience por <code>/</code>
El informe se previsualiza vacío	La expresión XPath no selecciona ningún nodo	Verificar la estructura del XML con un editor y ajustar la expresión
Las vocales acentuadas aparecen corruptas	El archivo XML no está guardado en UTF-8	Guardar el archivo como UTF-8 y verificar la declaración XML
<code>SAXParseException</code> al leer el XML	El archivo XML no está bien formado	Validar el archivo con un editor XML y corregir los errores de estructura
La conexión queda abierta después del llenado	<code>JRXmlDataSource</code> mantiene el archivo en memoria	No requiere cierre explícito pero conviene liberarlo cuando ya no se usa

Reto resuelto paso a paso

Enunciado: añadir un campo calculado que muestre un descuento del 5% sobre la cantidad entregada cuando la cantidad sea superior a 20. El campo debe mostrar el valor del descuento o Sin descuento según el caso.

Paso 1. Hacer doble clic sobre el archivo `informe_distribucion.xml.jrxml` en el panel Project Explorer.

Paso 2. Hacer clic sobre la pestaña Design en la parte inferior del editor central.

Paso 3. Hacer clic sobre el nodo Detail 1 en el panel Outline.

Paso 4. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 35 y pulsar Enter.

Paso 5. Hacer clic sobre la pestaña Elements en el panel Palette.

Paso 6. Hacer clic sobre el icono Text Field.

Paso 7. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada `x=0, y=20`.

Paso 8. Hacer clic sobre el campo Width en el panel Properties, escribir 555 y pulsar Enter.

Paso 9. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.

Paso 10. Hacer clic sobre el campo X, escribir 0 y pulsar Enter.

Paso 11. Hacer clic sobre el campo Y, escribir 20 y pulsar Enter.

Paso 12. Hacer clic sobre el campo Text Field Expression y escribir exactamente `Integer.parseInt($F{cantidad}) > 20 ? "Descuento 5%: " + (Integer.parseInt($F{cantidad}) * 0.05) + " uds." : "Sin descuento"` y pulsar Enter.

Paso 13. Hacer clic sobre el campo Font size y escribir 9. Pulsar Enter.

Paso 14. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.

Paso 15. Pulsar Ctrl+S para guardar el archivo.

Paso 16. Pulsar Ctrl+Mayús+B para compilar el informe.

Paso 17. Hacer clic con el botón derecho sobre `GeneradorDistribucionXML.java` y seleccionar Run As > Java Application.

Paso 18. Abrir el archivo `output/informe_distribucion_xml.pdf` y verificar que las entregas con cantidad superior a 20 muestran el descuento y las demás muestran Sin descuento.

Simulación ASCII del PDF tras el reto

TEXT

```

|| Librería      | Ciudad  | Cantidad | Fecha  | Tít. ||
||-----|-----|-----|-----|-----||
|| Librería Central | Madrid  | 25      | 15/09/26 | Cien ||
|| Descuento 5%: 1.25 uds. ||
|| Librería del Prado | Barcelona | 15      | 16/09/26 | Rayu ||
|| Sin descuento ||
|| Librería Ateneo    | Sevilla  | 30      | 17/09/26 | La c ||
|| Descuento 5%: 1.5 uds. ||
|| ... ||

```

Resultado del reto: las entregas con cantidad superior a 20 (Librería Central con 25, Librería Ateneo con 30, Librería A. Machado con 22) muestran el descuento calculado. Las demás muestran Sin descuento. La expresión utiliza el método `Integer.parseInt` para convertir la cadena del XML en un entero y el operador ternario para elegir entre los dos textos posibles.

Analogía final con el contexto de la editorial

El archivo XML es el listado de entregas estructurado jerárquicamente. Cada elemento entrega es una entrega a una librería. Cada elemento hijo contiene un dato: el nombre de la librería, la ciudad, la cantidad, la fecha y el título del libro. La expresión XPath de selección es la instrucción que indica al motor qué elementos del XML deben actuar como registros. Las expresiones XPath de los campos son las instrucciones que indican al motor qué dato extraer de cada registro. El adaptador XML de Jaspersoft Studio es la herramienta que permite consultar el archivo desde la mesa de diseño. La clase `JRXmlDataSource` es la herramienta equivalente desde la consola de control. El XML es el formato que permite representar relaciones complejas y que se integra sin problemas en el sistema de informes.

Resultado esperado

Al finalizar este punto, el alumno dispone de:

- El archivo `data/distribucion.xml` con el elemento raíz y los ocho elementos entrega.
- El adaptador `DistribucionXML` en el panel Repository Explorer de Jaspersoft Studio.
- El archivo `reports/informe_distribucion_xml.jrxml` con la consulta XPath y los cinco campos generados automáticamente.
- El archivo `output/informe_distribucion_xml.pdf` con las ocho entregas y la fecha formateada.
- La clase `GeneradorDistribucionXML.java` que lee el XML desde código Java.
- El archivo `XML.md` en la raíz del proyecto con la documentación del uso de XML.
- Comprensión operativa de la lectura de XML, de las expresiones XPath y del mapeo de nodos a campos.

Conclusión y enlace al siguiente punto

El punto 3.3 ha introducido la lectura de archivos XML como fuente de datos para el informe. Han quedado configurados el adaptador XML en Jaspersoft Studio y la clase `JRXmlDataSource` en el programa Java. El informe `informe_distribucion_xml.jrxml` se alimenta del archivo `distribucion.xml` y muestra las ocho entregas con la librería, la ciudad, la cantidad, la fecha formateada y el título del libro. La conversión de tipos desde las cadenas del XML a los tipos del informe se ha demostrado con el ejemplo de la fecha.

El punto 3.4, «Ficheros JSON», introduce la lectura de datos desde archivos JSON y demuestra su uso con un archivo de autores de libros. El punto cubre la estructura de un archivo JSON, la configuración de adaptadores JSON en Jaspersoft Studio y la lectura desde código Java con `JsonDataSource`.

Punto 3.4 — Ficheros JSON

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.4 — Ficheros JSON

Objetivos de aprendizaje

- Comprender el formato JSON y su uso como fuente de datos estructurada.
- Configurar un adaptador JSON en Jaspersoft Studio y asociarlo a un informe.
- Utilizar expresiones JSON para seleccionar registros y mapear campos.
- Leer un archivo JSON desde código Java con `JsonDataSource`.
- Combinar datos de un JSON con datos de otras fuentes del proyecto.
- Documentar el uso de archivos JSON en el proyecto EditorialReports.

Parte práctica

Parte A — Práctica visual

Paso 1: Copiar el archivo jackson al proyecto Java

Acciones:

1. Abrir el explorador de archivos del sistema operativo.
2. Navegar hasta la carpeta de instalación de Jaspersoft Studio 6.20.0, subcarpeta `plugins`. En Windows: `C:\JaspersoftStudio-6.20.0\plugins`.
3. Localizar el archivo cuyo nombre comienza por `jackson-core-` y termina en `.jar`.
4. Hacer clic con el botón derecho sobre el archivo y seleccionar Copiar.
5. Volver a Jaspersoft Studio y hacer clic con el botón derecho sobre la carpeta `lib` del proyecto EditorialReportsJava en el panel Project Explorer.
6. Hacer clic sobre la opción Paste en el menú contextual.
7. Repetir las acciones 3 a 6 para los archivos `jackson-databind-*.jar` y `jackson-annotations-*.jar`.
8. Hacer clic con el botón derecho sobre el nodo EditorialReportsJava y seleccionar Refresh.

Verificación visual: la carpeta `lib` del panel Project Explorer muestra los tres archivos JAR de Jackson junto a los JAR existentes.

Qué hace: incorpora los JAR del módulo Jackson a la carpeta de librerías del proyecto.

Por qué: el módulo JSON de JasperReports utiliza Jackson para analizar los archivos JSON.

Error común: copiar solo el archivo `jackson-core` y omitir los otros dos. El motor lanza `NoClassDefFoundError` al analizar el archivo.

Solución: copiar los tres JAR.

Analogía: es como añadir a la imprenta las herramientas específicas para leer archivos JSON.

Paso 2: Añadir los JAR de Jackson al Build Path

Acciones:

1. Hacer clic con el botón derecho sobre el nodo EditorialReportsJava en el panel Project Explorer.
2. Hacer clic sobre la opción Properties en el menú contextual.
3. Hacer clic sobre la categoría Java Build Path en el panel izquierdo del diálogo.
4. Hacer clic sobre la pestaña Libraries en el panel derecho.

5. Hacer clic sobre el botón Add JARs....
6. Expandir el nodo EditorialReportsJava y hacer clic sobre la carpeta lib.
7. Marcar las casillas de los tres archivos JAR de Jackson.
8. Hacer clic sobre el botón OK.
9. Hacer clic sobre el botón Apply and Close.

Verificación visual: el panel Project Explorer muestra los tres JAR de Jackson con un icono de librería referenciada.

Qué hace: registra los JAR de Jackson en el classpath del proyecto Java.

Por qué: sin esta acción, el módulo JSON no encuentra las clases de Jackson y lanza `NoClassDefFoundError`.

Error común: añadir los JAR como External JARs con ruta absoluta. El proyecto deja de ser portable.

Solución: eliminarlos y añadirlos con Add JARs desde la carpeta lib.

Analogía: es como registrar en el catálogo de la imprenta las nuevas herramientas.

Paso 3: Crear el archivo autores.json

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta data en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción File en el submenú.
4. Escribir exactamente `autores.json` en el campo File name del diálogo.
5. Hacer clic sobre el botón Finish.
6. En el editor central, escribir exactamente la primera línea: `{` y pulsar Enter.
7. Escribir exactamente `"autores": [` y pulsar Enter.
8. Escribir exactamente `{` y pulsar Enter.
9. Escribir exactamente `"nombre": "Gabriel García Márquez",` y pulsar Enter.
10. Escribir exactamente `"nacionalidad": "Colombiana",` y pulsar Enter.
11. Escribir exactamente `"nacimiento": "1927-03-06",` y pulsar Enter.
12. Escribir exactamente `"premios": 1,` y pulsar Enter.
13. Escribir exactamente `"vivo": false` y pulsar Enter.
14. Escribir exactamente `}`, y pulsar Enter.
15. Repetir las acciones 8 a 14 para los siguientes autores:
 - Julio Cortázar, Argentina, 1914-08-26, 0, false
 - Jorge Luis Borges, Argentina, 1899-08-24, 2, false
 - Juan Rulfo, Mexicana, 1917-05-16, 1, false
 - Isabel Allende, Chilena, 1942-08-02, 2, true
 - Mario Vargas Llosa, Peruana, 1936-03-28, 3, true
16. Escribir exactamente `]` y pulsar Enter.
17. Escribir exactamente `}` y pulsar Enter.
18. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `autores.json` dentro de la carpeta data. El editor central muestra la estructura completa del JSON.

Qué hace: crea el archivo JSON con la información de los seis autores.

Por qué: el archivo JSON es la fuente de datos del informe de este punto.

Error común: olvidar la coma entre objetos del arreglo y provocar un error de análisis JSON.

Solución: revisar que cada objeto excepto el último termina con coma.

Analogía: es como preparar el listado de autores en formato JSON.

Paso 4: Crear el adaptador JSON en Jaspersoft Studio

Acciones:

1. Hacer clic con el botón derecho sobre el nodo Data Adapters en el panel Repository Explorer (lateral izquierdo).
2. Hacer clic sobre la opción Create Data Adapter en el menú contextual.
3. Hacer clic sobre el elemento JSON File Data Source en la lista de categorías del asistente.
4. Hacer clic sobre el botón Next.
5. Escribir exactamente Autores JSON en el campo Name.
6. Hacer clic sobre el botón Browse... situado junto al campo File y navegar hasta la carpeta Documents\JasperProjects\EditorialReports\data.
7. Hacer clic sobre el archivo autores.json y pulsar el botón Open.
8. Hacer clic sobre el campo Select Expression y escribir exactamente autores.
9. Hacer clic sobre el botón Test Connection.
10. Verificar que el diálogo muestra el mensaje Connection successful y la lista de campos detectados.
11. Hacer clic sobre el botón Finish.

Verificación visual: el panel Repository Explorer muestra el nodo Autores JSON colgando de Data Adapters con un icono de archivo JSON.

Qué hace: registra un adaptador JSON que Jaspersoft Studio utiliza para leer el archivo durante la previsualización.

Por qué: el adaptador permite ejecutar el informe con los datos del JSON desde el entorno de diseño.

Error común: olvidar la expresión de selección y provocar que el adaptador intente leer el objeto raíz como registro único.

Solución: escribir autores en el campo Select Expression.

Analogía: es como registrar en la editorial el listado de autores para poder consultarlo durante la composición.

Paso 5: Crear el informe informe_autores_json.jrxml

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta reports en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Jasper Report en el submenú.
4. Hacer clic sobre la plantilla Blank A4 en la lista de plantillas del asistente.
5. Hacer clic sobre el botón Next.
6. Escribir exactamente informe_autores_json en el campo File name.
7. Hacer clic sobre el botón Next.
8. Hacer clic sobre Autores JSON en la lista de adaptadores disponibles.
9. Hacer clic sobre el botón Finish.
10. Observar que el asistente ha generado la consulta JSON y las declaraciones de los campos con sus tipos.

Verificación visual: el editor central muestra el archivo informe_autores_json.jrxml. El panel Outline muestra el nodo QueryString con la expresión autores y el nodo Fields con los cinco campos.

Qué hace: crea un nuevo informe a partir del adaptador JSON.

Por qué: la generación automática evita errores en la expresión de selección y en los nombres de los campos.

Error común: olvidar seleccionar el adaptador correcto.

Solución: cerrar el asistente y repetir el paso seleccionando Autores JSON.

Analogía: es como abrir un nuevo pliego del catálogo con la estructura del listado de autores preparada.

Paso 6: Ajustar las bandas del nuevo informe

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `informe_autores_json` en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la opción Delete en el menú contextual para eliminar el nodo Page Header.
3. Hacer clic con el botón derecho sobre el nodo `informe_autores_json` y eliminar el nodo Column Footer.
4. Hacer clic con el botón derecho sobre el nodo `informe_autores_json` y eliminar el nodo Summary.
5. Hacer clic sobre el nodo Title en el panel Outline y ajustar su Band height a 60 píxeles desde el panel Properties.

Verificación visual: el panel Outline muestra solo las bandas Title, Column Header, Detail 1, Page Footer y Background.

Qué hace: simplifica el informe para que contenga solo las bandas necesarias.

Por qué: el informe de autores necesita un número reducido de bandas.

Error común: eliminar la banda Background pensando que no se usa.

Solución: si se elimina por error, cerrar el archivo sin guardar y volver a abrirlo.

Analogía: es como reducir el pliego del listado de autores a las secciones estrictamente necesarias.

Paso 7: Añadir el título en la banda Title

Acciones:

1. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
2. Hacer clic sobre el icono Static Text (una letra T mayúscula).
3. Arrastrar el icono Static Text y soltarlo dentro de la banda Title, en la coordenada aproximada x=0, y=15.
4. Hacer doble clic sobre el Static Text creado en la acción anterior.
5. Escribir exactamente Catálogo de Autores - Datos desde JSON.
6. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
7. Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 0 y pulsar Enter.
8. Hacer clic sobre el campo Y, escribir 15 y pulsar Enter.
9. Hacer clic sobre el campo Width, escribir 555 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 30 y pulsar Enter.
11. Hacer clic sobre el campo Font size y escribir 18. Pulsar Enter.
12. Marcar la casilla Bold.
13. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.

Verificación visual: la banda Title muestra el texto Catálogo de Autores - Datos desde JSON centrado y en negrita.

Qué hace: inserta el título del informe de autores desde JSON.

Por qué: el título identifica el documento y su origen de datos.

Error común: olvidar el centrado y provocar que el título aparezca alineado a la izquierda.

Solución: seleccionar Center.

Analogía: es como titular el listado de autores con el nombre de la editorial.

Paso 8: Añadir los encabezados de columna

Acciones:

1. Hacer clic sobre el nodo Column Header en el panel Outline (inferior izquierdo).

2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Static Text.
4. Arrastrar el icono Static Text y soltarlo dentro de la banda Column Header, en la coordenada aproximada x=0, y=5.
5. Hacer doble clic sobre el Static Text creado en la acción anterior.
6. Escribir exactamente Nombre.
7. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
8. Hacer clic sobre el campo Width en el panel Properties, escribir 220 y pulsar Enter.
9. Marcar la casilla Bold.
10. Repetir las acciones 3 a 9 para los encabezados Nacionalidad (x=220, ancho 120), Nacimiento (x=340, ancho 90), Premios (x=430, ancho 60) y Estado (x=490, ancho 65).

Verificación visual: la banda Column Header muestra los cinco encabezados en negrita.

Qué hace: inserta los encabezados de las columnas del informe de autores.

Por qué: los encabezados identifican las columnas de la tabla.

Error común: dejar los encabezados sin negrita.

Solución: marcar la casilla Bold.

Analogía: es como añadir los títulos de las columnas al listado de autores.

Paso 9: Añadir los campos en la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).
4. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=0, y=0.
5. Hacer clic sobre el campo Width en el panel Properties, pestaña Properties, escribir 220 y pulsar Enter.
6. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.
7. Hacer clic sobre el campo Text Field Expression y escribir exactamente \$F{nombre} y pulsar Enter.
8. Repetir las acciones 3 a 7 para los campos nacionalidad (x=220, ancho 120) y premios (x=430, ancho 60).
9. Para el campo de nacimiento, hacer clic sobre el icono Text Field y arrastrarlo dentro de la banda Detail 1, en la coordenada aproximada x=340, y=0.
10. Hacer clic sobre el campo Width y escribir 90. Pulsar Enter.
11. Hacer clic sobre el campo Height y escribir 20. Pulsar Enter.
12. Hacer clic sobre el campo Text Field Expression y escribir exactamente \$F{nacimiento} y pulsar Enter.
13. Dejar Pattern vacío. El campo nacimiento es String ISO; si se desea dd/MM/yyyy, usar una expresión con substring como la del JRXML canónico.
14. Para el campo del estado, hacer clic sobre el icono Text Field y arrastrarlo dentro de la banda Detail 1, en la coordenada aproximada x=490, y=0.
15. Hacer clic sobre el campo Width y escribir 65. Pulsar Enter.
16. Hacer clic sobre el campo Height y escribir 20. Pulsar Enter.
17. Hacer clic sobre el campo Text Field Expression y escribir exactamente \$F{vivo}.booleanValue() ? "Activo" : "Inactivo" y pulsar Enter.

Verificación visual: la banda Detail 1 muestra cinco campos con las expresiones correspondientes. El campo del estado utiliza el operador ternario sobre el campo booleano.

Qué hace: inserta los campos que se imprimen para cada autor del JSON.

Por qué: los campos resuelven los valores de las propiedades del JSON para cada registro.

Error común: escribir \$F{vivo} sin invocar booleanValue(). El compilador informa un error de tipo en la operación ternaria.

Solución: usar `$F{vivo}.booleanValue()` para convertir el objeto `Boolean` a un valor primitivo.

Analogía: es como rellenar las celdas del listado de autores con los datos de cada uno.

Paso 10: Ajustar la altura de la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 20 y pulsar Enter.
3. Hacer clic sobre el desplegable Split Type en el panel Properties y verificar que está en *Stretch*.

Verificación visual: la banda Detail 1 aparece con 20 píxeles de altura.

Qué hace: fija la altura de la banda Detail para que cada autor ocupe una fila.

Por qué: la altura determina el espacio de cada fila y el número de filas que caben en una página.

Error común: dejar la altura por defecto.

Solución: ajustar la altura a 20 píxeles.

Analogía: es como ajustar la altura de cada fila del listado de autores.

Paso 11: Añadir el pie de página con totales

Acciones:

1. Hacer clic sobre el nodo Page Footer en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 40 y pulsar Enter.
3. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
4. Hacer clic sobre el icono Static Text.
5. Arrastrar el icono Static Text y soltarlo dentro de la banda Page Footer, en la coordenada aproximada x=0, y=5.
6. Hacer doble clic sobre el Static Text creado en la acción anterior.
7. Escribir exactamente `Total de autores:`.
8. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
9. Hacer clic sobre el campo Width en el panel Properties, escribir 150 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
11. Hacer clic sobre la pestaña Elements en el panel Palette.
12. Hacer clic sobre el icono Text Field.
13. Arrastrar el icono Text Field y soltarlo dentro de la banda Page Footer, a la derecha del rótulo, en la coordenada aproximada x=150, y=5.
14. Hacer clic sobre el campo Width y escribir 50. Pulsar Enter.
15. Hacer clic sobre el campo Height y escribir 15. Pulsar Enter.
16. Hacer clic sobre el campo Text Field Expression y escribir exactamente `$V{REPORT_COUNT}` y pulsar Enter.
17. Marcar la casilla Bold.
18. Hacer clic sobre la pestaña Elements en el panel Palette y hacer clic sobre el icono Text Field.
19. Arrastrar el icono Text Field y soltarlo dentro de la banda Page Footer, debajo del rótulo anterior, en la coordenada aproximada x=0, y=20.
20. Hacer clic sobre el campo Width y escribir 555. Pulsar Enter.
21. Hacer clic sobre el campo Height y escribir 15. Pulsar Enter.
22. Hacer clic sobre el campo Text Field Expression y escribir exactamente `"Página " + $V{PAGE_NUMBER} + " de " + [segundo campo PAGE_NUMBER con evaluationTime=Report]` y pulsar Enter.
23. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.
24. Hacer clic sobre el campo Font size y escribir 9. Pulsar Enter.

Verificación visual: la banda Page Footer muestra el rótulo del total de autores con su valor en negrita y el campo de paginación centrado debajo.

Qué hace: inserta un pie de página con el recuento total de autores y la paginación.

Por qué: el recuento y la paginación informan al lector sobre el volumen y la posición en el documento.

Error común: olvidar el centrado del campo de paginación.

Solución: seleccionar Center.

Analogía: es como anotar al pie del listado de autores el total y la página.

Paso 12: Compilar y previsualizar con el adaptador JSON

Acciones:

1. Pulsar Ctrl+S para guardar el archivo.
2. Pulsar Ctrl+Mayús+B para compilar el informe.
3. Hacer clic sobre el panel Problems (inferior) y verificar que no hay errores.
4. Pulsar el botón Preview de la barra de herramientas superior.
5. En el diálogo, verificar que el adaptador Autores JSON está seleccionado.
6. Hacer clic sobre el botón OK.
7. Esperar a que se abra la pestaña Preview en el editor central.

Verificación visual: la pestaña Preview muestra el informe con los seis autores del JSON. Cada fila muestra el nombre, la nacionalidad, la fecha de nacimiento, el número de premios y el estado.

Qué hace: ejecuta el informe con los datos del JSON y muestra el resultado.

Por qué: la previsualización confirma que el adaptador JSON funciona y que los campos se resuelven correctamente.

Error común: obtener NoClassDefFoundError: com/fasterxml/jackson/... Indica que los JAR de Jackson no están en el classpath.

Solución: añadirlos al Build Path.

Analogía: es como revisar la prueba de color del listado de autores con los datos del JSON.

Paso 13: Crear la clase GeneradorAutoresJSON

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta src en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Class en el submenú.
4. Escribir exactamente GeneradorAutoresJSON en el campo Name del diálogo.
5. Marcar la casilla public static void main(String[] args).
6. Hacer clic sobre el botón Finish.
7. En el editor central, escribir el código completo de la clase GeneradorAutoresJSON que se muestra en la Parte C de este punto.
8. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo GeneradorAutoresJSON.java dentro de la carpeta src.

Qué hace: crea la clase que genera el informe de autores desde el archivo JSON.

Por qué: el informe puede ejecutarse desde código Java sin depender de Jaspersoft Studio.

Error común: olvidar importar net.sf.jasperreports.engine.data.JsonDataSource.

Solución: añadir la importación correspondiente.

Analogía: es como preparar la consola de control para que el operario genere el listado de autores desde el JSON.

Paso 14: Ejecutar el programa y verificar el PDF

Acciones:

1. Hacer clic con el botón derecho sobre el archivo `GeneradorAutoresJSON.java` en el panel Project Explorer.
2. Hacer clic sobre la opción Run As en el menú contextual.
3. Hacer clic sobre la opción Java Application en el submenú.
4. Hacer clic sobre la vista Console en el panel inferior y observar el resultado.
5. Abrir el explorador de archivos del sistema operativo.
6. Navegar hasta la carpeta output del proyecto EditorialReports.
7. Hacer doble clic sobre el archivo `informe_autores_json.pdf`.
8. Verificar que el PDF muestra los seis autores con todos sus datos.

Verificación visual: la vista Console muestra la línea `Informe generado en: ...` con la ruta absoluta del PDF. El archivo PDF muestra los seis autores.

Qué hace: ejecuta el programa Java que lee el JSON y genera el informe.

Por qué: la ejecución confirma que el programa lee el JSON y genera el informe correctamente.

Error común: ejecutar el programa desde un directorio distinto a la raíz del proyecto y obtener `FileNotFoundException: data/autores.json`.

Solución: comprobar en Run Configurations que el Working Directory apunta a la raíz del proyecto.

Analogía: es como imprimir el listado de autores desde el archivo JSON.

Paso 15: Documentar el uso de archivos JSON

Acciones:

1. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción File en el submenú.
4. Escribir exactamente `JSON.md` en el campo File name del diálogo.
5. Hacer clic sobre el botón Finish.
6. En el editor central, escribir exactamente `# Ficheros JSON` y pulsar Enter dos veces.
7. Escribir exactamente `## Archivos del proyecto` y pulsar Enter dos veces.
8. Escribir exactamente `- data/autores.json: información de autores con nombre, nacionalidad, nacimiento, premios y estado.` y pulsar Enter dos veces.
9. Escribir exactamente `## Adaptador de Jaspersoft Studio` y pulsar Enter dos veces.
10. Escribir exactamente `- Nombre: AutoresJSON` y pulsar Enter.
11. Escribir exactamente `- Expresión de selección: autores` y pulsar Enter dos veces.
12. Escribir exactamente `## Lectura desde Java` y pulsar Enter dos veces.
13. Escribir exactamente `- Clase: net.sf.jasperreports.engine.data.JsonDataSource` y pulsar Enter.
14. Escribir exactamente `- Dependencia: jackson-core, jackson-databind, jackson-annotations` y pulsar Enter dos veces.
15. Escribir exactamente `## Campos` y pulsar Enter dos veces.
16. Escribir exactamente `- nombre, nacionalidad, nacimiento, premios, vivo` y pulsar Enter.
17. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `JSON.md` en la raíz del proyecto EditorialReports.

Qué hace: incorpora al proyecto un documento que registra el uso de archivos JSON.

Por qué: la documentación de las fuentes de datos facilita el mantenimiento y la incorporación de nuevos desarrolladores.

Error común: olvidar documentar la dependencia de Jackson.

Solución: incluir las secciones especificadas.

Analogía: es como dejar en la editorial una ficha técnica con la ubicación del listado de autores y su formato.

Parte B — JRXML completo explicado línea por línea [VALIDADO]

JRXML canónico del checkpoint. Es el mismo archivo que se compila en GitHub Actions.

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
name="informe_autores_json"
language="java"
pageWidth="595"
pageHeight="842"
columnWidth="555"
leftMargin="20"
rightMargin="20"
topMargin="20"
bottomMargin="20"
uuid="d5a6b7c8-e9f0-1a2b-3c4d-5e6f7a8b9c0d">
<property name="com.jaspersoft.studio.data.defaultdataadapter" value="AutoresJSON"/>
<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
<queryString language="json">
<![CDATA[autores]]>
</queryString>
<field name="nombre" class="java.lang.String"><property name="net.sf.jasperreports.json.field.expression" value="nombre"/></field>
<field name="nacionalidad" class="java.lang.String"><property name="net.sf.jasperreports.json.field.expression"
value="nacionalidad"/></field>
<field name="nacimientto" class="java.lang.String"><property name="net.sf.jasperreports.json.field.expression" value="nacimientto"/></
field>
<field name="premios" class="java.lang.Integer"><property name="net.sf.jasperreports.json.field.expression" value="premios"/></
field>
<field name="vivo" class="java.lang.Boolean"><property name="net.sf.jasperreports.json.field.expression" value="vivo"/></field>
<background>
<band height="0"/>
</background>

<title>
<band height="60">
<staticText>
<reportElement x="0" y="15" width="555" height="30" uuid="e6b7c8d9-f0a1-2b3c-4d5e-6f7a8b9c0d1e"/>
<textElement textAlignment="Center" verticalAlignment="Middle">
<font fontName="DejaVu Sans" size="18" isBold="true"/>
</textElement>
<text><![CDATA[Catálogo de Autores - Datos desde JSON]]></text>
</staticText>
</band>
</title>
<columnHeader>
<band height="25">
<staticText>
<reportElement x="0" y="5" width="220" height="15" uuid="f7c8d9e0-a1b2-3c4d-5e6f-7a8b9c0d1e2f"/>
<textElement verticalAlignment="Middle">
<font fontName="DejaVu Sans" size="10" isBold="true"/>
</textElement>
<text><![CDATA[Nombre]]></text>
</staticText>
<staticText>
<reportElement x="220" y="5" width="120" height="15" uuid="a8d9e0f1-b2c3-4d5e-6f7a-8b9c0d1e2f3a"/>
<textElement verticalAlignment="Middle">
<font fontName="DejaVu Sans" size="10" isBold="true"/>
</textElement>
<text><![CDATA[Nacionalidad]]></text>
</staticText>
<staticText>
<reportElement x="340" y="5" width="90" height="15" uuid="b9e0f1a2-c3d4-5e6f-7a8b-9c0d1e2f3a4b"/>
<textElement textAlignment="Center" verticalAlignment="Middle">
<font fontName="DejaVu Sans" size="10" isBold="true"/>
</textElement>
<text><![CDATA[Nacimientto]]></text>
</staticText>
<staticText>
<reportElement x="430" y="5" width="60" height="15" uuid="c0f1a2b3-d4e5-6f7a-8b9c-0d1e2f3a4b5c"/>
<textElement textAlignment="Right" verticalAlignment="Middle">
<font fontName="DejaVu Sans" size="10" isBold="true"/>
```

```

        </textElement>
        <text><![CDATA[Premios]]></text>
    </staticText>
    <staticText>
        <reportElement x="490" y="5" width="65" height="15" uuid="d1a2b3c4-e5f6-7a8b-9c0d-1e2f3a4b5c6d"/>
        <textElement textAlignment="Center" verticalAlignment="Middle">
            <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Estado]]></text>
    </staticText>
</band>
</columnHeader>
<detail>
    <band height="20" splitType="Stretch">
        <textField textAdjust="StretchHeight">
            <reportElement x="0" y="0" width="220" height="20" uuid="e2b3c4d5-f6a7-8b9c-0d1e-2f3a4b5c6d7e"/>
            <textElement verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F{nombre}}]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="220" y="0" width="120" height="20" uuid="f3c4d5e6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"/>
            <textElement verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F{nacionalidad}}]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="340" y="0" width="90" height="20" uuid="a4d5e6f7-b8c9-0d1e-2f3a-4b5c6d7e8f9a"/>
            <textElement textAlignment="Center" verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F{nacimiento}.substring(8,10) + "/" + ${F{nacimiento}.substring(5,7) + "/" +
${F{nacimiento}.substring(0,4)}}]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="430" y="0" width="60" height="20" uuid="b5e6f7a8-c9d0-1e2f-3a4b-5c6d7e8f9a0b"/>
            <textElement textAlignment="Right" verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F{premios}}]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="490" y="0" width="65" height="20" uuid="c6f7a8b9-d0e1-2f3a-4b5c-6d7e8f9a0b1c"/>
            <textElement textAlignment="Center" verticalAlignment="Middle">
                <font fontName="DejaVu Sans" size="10"/>
            </textElement>
            <textFieldExpression><![CDATA[${F{vivo}.booleanValue() ? "Activo" : "Inactivo"}]></textFieldExpression>
        </textField>
    </band>
</detail>
<pageFooter>
    <band height="45">
        <staticText>
            <reportElement x="0" y="3" width="150" height="15" uuid="33333333-3333-4333-8333-333333333331"/>
            <textElement verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
            <text><![CDATA[Total de autores:]]></text>
        </staticText>
        <textField>
            <reportElement x="150" y="3" width="70" height="15" uuid="33333333-3333-4333-8333-333333333332"/>
            <textElement verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9" isBold="true"/></textElement>
            <textFieldExpression><![CDATA[${V{REPORT_COUNT}}]></textFieldExpression>
        </textField>
        <textField>
            <reportElement x="170" y="23" width="190" height="15" uuid="33333333-3333-4333-8333-333333333333"/>
            <textElement textAlignment="Right" verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
            <textFieldExpression><![CDATA["Página " + ${V{PAGE_NUMBER}} + " de"]]></textFieldExpression>
        </textField>
        <textField evaluationTime="Report">
            <reportElement x="365" y="23" width="30" height="15" uuid="33333333-3333-4333-8333-333333333334"/>
            <textElement textAlignment="Left" verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
            <textFieldExpression><![CDATA[${V{PAGE_NUMBER}}]></textFieldExpression>
        </textField>
    </band>
</pageFooter>
</jasperReport>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara el documento XML y la codificación UTF-8.
Línea 2	<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" → Abre la plantilla JasperReports y define sus atributos principales.

Línea 3	<code>xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Completa la definición declarativa del informe.</code>
Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd" → Declara el espacio de nombres/XSD usado para validar el JRXML.</code>
Línea 5	<code>name="informe_autores_json" → Completa la definición declarativa del informe.</code>
Línea 6	<code>language="java" → Completa la definición declarativa del informe.</code>
Línea 7	<code>pageWidth="595" → Completa la definición declarativa del informe.</code>
Línea 8	<code>pageHeight="842" → Completa la definición declarativa del informe.</code>
Línea 9	<code>columnWidth="555" → Completa la definición declarativa del informe.</code>
Línea 10	<code>leftMargin="20" → Completa la definición declarativa del informe.</code>
Línea 11	<code>rightMargin="20" → Completa la definición declarativa del informe.</code>
Línea 12	<code>topMargin="20" → Completa la definición declarativa del informe.</code>
Línea 13	<code>bottomMargin="20" → Completa la definición declarativa del informe.</code>
Línea 14	<code>uuid="d5a6b7c8-e9f0-1a2b-3c4d-5e6f7a8b9c0d"> → Completa la definición declarativa del informe.</code>
Línea 15	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="AutoresJSON"/> → Asocia el Data Adapter usado por JasperSoft Studio durante Preview.</code>
Línea 16	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/> → Declara un estilo compatible con JasperReports 6.20.0.</code>
Línea 17	<code><queryString language="json"> → Abre la consulta del dataset e indica el lenguaje de consulta.</code>
Línea 18	<code><![CDATA[autores]]> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.</code>
Línea 19	<code></queryString> → Cierra el elemento XML abierto anteriormente.</code>
Línea 20	<code><field name="nombre" class="java.lang.String"><property name="net.sf.jasperreports.json.field.expression" value="nombre"/></field> → Declara la expresión de mapeo del campo para el origen jerárquico.</code>
Línea 21	<code><field name="nacionalidad" class="java.lang.String"><property name="net.sf.jasperreports.json.field.expression" value="nacionalidad"/></field> → Declara la expresión de mapeo del campo para el origen jerárquico.</code>
Línea 22	<code><field name="nacimiento" class="java.lang.String"><property name="net.sf.jasperreports.json.field.expression" value="nacimiento"/></field> → Declara la expresión de mapeo del campo para el origen jerárquico.</code>
Línea 23	<code><field name="premios" class="java.lang.Integer"><property name="net.sf.jasperreports.json.field.expression" value="premios"/></field> → Declara la expresión de mapeo del campo para el origen jerárquico.</code>
Línea 24	<code><field name="vivo" class="java.lang.Boolean"><property name="net.sf.jasperreports.json.field.expression" value="vivo"/></field> → Declara la expresión de mapeo del campo para el origen jerárquico.</code>
Línea 25	<code><background> → Abre una sección/banda estructural del informe.</code>
Línea 26	<code><band height="0"/> → Define la altura y, cuando procede, la política de división de la banda.</code>
Línea 27	<code></background> → Cierra el elemento XML abierto anteriormente.</code>
Línea 28	<code>`` → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.</code>
Línea 29	<code><title> → Abre una sección/banda estructural del informe.</code>

Línea 30	<code><band height="60"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 31	<code><staticText></code> → Abre un elemento de texto estático.
Línea 32	<code><reportElement x="0" y="15" width="555" height="30" uuid="e6b7c8d9-f0a1-2b3c-4d5e-6f7a8b9c0d1e"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 33	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 34	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 35	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 36	<code><text><![CDATA[Catálogo de Autores - Datos desde JSON]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 37	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 38	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 39	<code></title></code> → Cierra el elemento XML abierto anteriormente.
Línea 40	<code><columnHeader></code> → Abre una sección/banda estructural del informe.
Línea 41	<code><band height="25"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 42	<code><staticText></code> → Abre un elemento de texto estático.
Línea 43	<code><reportElement x="0" y="5" width="220" height="15" uuid="f7c8d9e0-a1b2-3c4d-5e6f-7a8b9c0d1e2f"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 44	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 45	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 46	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 47	<code><text><![CDATA[Nombre]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 48	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 49	<code><staticText></code> → Abre un elemento de texto estático.
Línea 50	<code><reportElement x="220" y="5" width="120" height="15" uuid="a8d9e0f1-b2c3-4d5e-6f7a-8b9c0d1e2f3a"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 51	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 52	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 53	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 54	<code><text><![CDATA[Nacionalidad]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 55	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 56	<code><staticText></code> → Abre un elemento de texto estático.
Línea 57	<code><reportElement x="340" y="5" width="90" height="15" uuid="b9e0f1a2-c3d4-5e6f-7a8b-9c0d1e2f3a4b"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 58	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.

Línea 59	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 60	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 61	<code><text><![CDATA[Nacimiento]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 62	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 63	<code><staticText></code> → Abre un elemento de texto estático.
Línea 64	<code><reportElement x="430" y="5" width="60" height="15" uuid="c0f1a2b3-d4e5-6f7a-8b9c-0d1e2f3a4b5c"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 65	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 66	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 67	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 68	<code><text><![CDATA[Premios]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 69	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 70	<code><staticText></code> → Abre un elemento de texto estático.
Línea 71	<code><reportElement x="490" y="5" width="65" height="15" uuid="d1a2b3c4-e5f6-7a8b-9c0d-1e2f3a4b5c6d"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 72	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 73	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 74	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 75	<code><text><![CDATA[Estado]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 76	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 77	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 78	<code></columnHeader></code> → Cierra el elemento XML abierto anteriormente.
Línea 79	<code><detail></code> → Abre una sección/banda estructural del informe.
Línea 80	<code><band height="20" splitType="Stretch"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 81	<code><textField textAdjust="StretchHeight"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 82	<code><reportElement x="0" y="0" width="220" height="20" uuid="e2b3c4d5-f6a7-8b9c-0d1e-2f3a4b5c6d7e"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 83	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 84	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 85	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 86	<code><textFieldExpression><![CDATA[\${nombre}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.

Línea 87	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 88	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 89	<code><reportElement x="220" y="0" width="120" height="20" uuid="f3c4d5e6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 90	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 91	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 92	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 93	<code><textFieldExpression><![CDATA[\${nacionalidad}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 94	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 95	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 96	<code><reportElement x="340" y="0" width="90" height="20" uuid="a4d5e6f7-b8c9-0d1e-2f3a-4b5c6d7e8f9a"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 97	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 98	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 99	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 100	<code><textFieldExpression><![CDATA[\${nacimient}.substring(8,10) + "/" + \${nacimient}.substring(5,7) + "/" + \${nacimient}.substring(0,4)}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 101	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 102	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 103	<code><reportElement x="430" y="0" width="60" height="20" uuid="b5e6f7a8-c9d0-1e2f-3a4b-5c6d7e8f9a0b"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 104	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 105	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 106	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 107	<code><textFieldExpression><![CDATA[\${premios}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 108	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 109	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 110	<code><reportElement x="490" y="0" width="65" height="20" uuid="c6f7a8b9-d0e1-2f3a-4b5c-6d7e8f9a0b1c"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 111	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 112	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 113	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.

Línea 114	<code><textFieldExpression><![CDATA[\$F{vivo}.booleanValue() ? "Activo" : "Inactivo"]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 115	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 116	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 117	<code></detail></code> → Cierra el elemento XML abierto anteriormente.
Línea 118	<code><pageFooter></code> → Abre una sección/banda estructural del informe.
Línea 119	<code><band height="45"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 120	<code><staticText></code> → Abre un elemento de texto estático.
Línea 121	<code><reportElement x="0" y="3" width="150" height="15" uuid="33333333-3333-4333-8333-333333333331"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 122	<code><textElement verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 123	<code><text><![CDATA[Total de autores:]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 124	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 125	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 126	<code><reportElement x="150" y="3" width="70" height="15" uuid="33333333-3333-4333-8333-333333333332"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 127	<code><textElement verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 128	<code><textFieldExpression><![CDATA[\$V{REPORT_COUNT}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 129	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 130	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 131	<code><reportElement x="170" y="23" width="190" height="15" uuid="33333333-3333-4333-8333-333333333333"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 132	<code><textElement textAlignment="Right" verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 133	<code><textFieldExpression><![CDATA["Página " + \$V{PAGE_NUMBER} + " de"]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 134	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 135	<code><textField evaluationTime="Report"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 136	<code><reportElement x="365" y="23" width="30" height="15" uuid="33333333-3333-4333-8333-333333333334"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 137	<code><textElement textAlignment="Left" verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 138	<code><textFieldExpression><![CDATA[\$V{PAGE_NUMBER}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 139	<code></textField></code> → Cierra el elemento XML abierto anteriormente.

Línea 140	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 141	<code></pageFooter></code> → Cierra el elemento XML abierto anteriormente.
Línea 142	<code></jasperReport></code> → Cierra el elemento XML abierto anteriormente.

Comprobación: las coordenadas se mantienen dentro de `columnWidth="555"`, el orden estructural es compatible con JasperReports 6.20.0 y no se usa sintaxis retirada de la baseline.

Parte C — Código Java explicado línea por línea [VALIDADO]

El código siguiente es el código real incluido en el checkpoint y ejecutado por el workflow E2E. Java no redibuja el informe: **compila -> llena -> exporta**.

Clase GeneradorAutoresJSON.java

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.data.JsonDataSource;

public class GeneradorAutoresJSON {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_autores_json.jrxml";
            String rutaJasper = "reports/informe_autores_json.jasper";
            String rutaPdf = "output/informe_autores_json.pdf";
            String rutaJson = "data/autores.json";
            new File("output").mkdirs();
            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
            JsonDataSource dataSource = new JsonDataSource(new File(rutaJson), "autores");
            Map<String, Object> parametros = new HashMap<String, Object>();
            JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, dataSource);
            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);
            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
            System.out.println("Autores JSON esperados: 6");
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 2	<code>import java.util.HashMap;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 3	<code>import java.util.Map;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 4	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 5	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 6	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 7	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 8	<code>import net.sf.jasperreports.engine.data.JsonDataSource;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 9	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 10	<code>public class GeneradorAutoresJSON {</code> → Declara la clase Java del checkpoint.

Línea 11	<code>public static void main(String[] args) {</code> → Declara el punto de entrada ejecutable.
Línea 12	<code>try {</code> → Abre un bloque protegido; si contiene recursos, se cerrarán automáticamente.
Línea 13	<code>String rutaJrxml = "reports/informe_autores_jrxml.jrxml";</code> → Fija la ruta del JRXML desde el Working Directory EditorialReports.
Línea 14	<code>String rutaJasper = "reports/informe_autores_jrxml.jasper";</code> → Fija la ruta del artefacto <code>.jasper</code> compilado.
Línea 15	<code>String rutaPdf = "output/informe_autores_jrxml.pdf";</code> → Fija la ruta del PDF que se exportará.
Línea 16	<code>String rutaJson = "data/autores.json";</code> → Forma parte de la lógica acumulativa del programa.
Línea 17	<code>new File("output").mkdirs();</code> → Crea el directorio necesario antes de escribir datos o salidas.
Línea 18	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML a <code>.jasper</code> con JasperReports 6.20.0.
Línea 19	<code>JsonDataSource dataSource = new JsonDataSource(new File(rutaJson), "autores");</code> → Construye la fuente JSON y aplica explícitamente la selección <code>autores</code> .
Línea 20	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa de parámetros que se entrega al motor de llenado.
Línea 21	<code>JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, dataSource);</code> → Llena el informe y obtiene un <code>JasperPrint</code> en memoria.
Línea 22	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → Exporta el <code>JasperPrint</code> a PDF.
Línea 23	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 24	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 25	<code>System.out.println("Autores JSON esperados: 6");</code> → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 26	<code>} catch (Exception e) {</code> → Captura cualquier fallo de compilación, datos, llenado o exportación.
Línea 27	<code>e.printStackTrace();</code> → Imprime la traza completa para facilitar el diagnóstico.
Línea 28	<code>System.exit(1);</code> → Termina con código distinto de cero para que CI detecte el fallo.
Línea 29	<code>}</code> → Cierra el bloque Java actual.
Línea 30	<code>}</code> → Cierra el bloque Java actual.
Línea 31	<code>}</code> → Cierra el bloque Java actual.

Criterio de fallo: todo `catch` termina con `System.exit(1)` para que una excepción no pueda aparecer como ejecución verde en CI.

Parte D — Simulación del PDF esperado y de la estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

<pre> +-----+ informe_autores_jrxml [Design] [Source] +-----+ Ruler: 0 100 200 300 400 500 555 +-----+ Title h = 60 Catálogo de Autores - Datos desde JSON Column Header h = 25 Nombre Nacionalidad Nacim. Prem. Estado </pre>		TEXT
--	--	------



Qué representa: la disposición del informe en el editor tras completar los quince pasos. El panel Outline muestra la consulta JSON y los cinco campos con sus tipos.

Cómo verificarlo: comparar la vista del editor con este esquema. El panel Outline debe mostrar el nodo QueryString con la expresión autores y los cinco campos con sus tipos.

D.2 — Jerarquía del Outline

```

informe_autores_json
├── Properties
│   └── com.jaspersoft.studio.data.defaultdataadapter = AutoresJSON
├── Styles
│   └── Sans_Normal [isDefault=true]
├── QueryString
│   └── autores [language=json]
├── Fields
│   ├── nombre [java.lang.String]
│   ├── nacionalidad [java.lang.String]
│   ├── nacimiento [java.lang.String]
│   ├── premios [java.lang.Integer]
│   └── vivo [java.lang.Boolean]
├── Title [band, height=60]
│   └── staticText "Catálogo de Autores - Datos desde JSON"
├── Column Header [band, height=25]
│   ├── staticText "Nombre" (bold)
│   ├── staticText "Nacionalidad" (bold)
│   ├── staticText "Nacimiento" (bold, center)
│   ├── staticText "Premios" (bold, right)
│   └── staticText "Estado" (bold, center)
├── Detail 1 [band, height=20, splitType=Stretch]
│   ├── textField [textAdjust=StretchHeight] ${nombre}
│   ├── textField ${F{nacionalidad}}
│   ├── textField [center] nacimiento ISO reordenado con substring
│   ├── textField [right] ${F{premios}}
│   └── textField [center] ${F{vivo}}.booleanValue() ? "Activo" : "Inactivo"
├── Page Footer [band, height=40]
│   ├── staticText "Total de autores: "
│   ├── textField [bold] ${V{REPORT_COUNT}}
│   └── textField [center] "Página " + ${V{PAGE_NUMBER}} + " de " + [segundo campo PAGE_NUMBER con evaluationTime=Report]
└── Background [band, height=0]
  
```

Qué representa: el árbol de nodos del informe tal como aparece en el panel Outline. La novedad respecto al punto 3.3 es que los campos tienen tipos distintos (Date, Integer, Boolean) porque el adaptador JSON los convierte automáticamente.

Cómo verificarlo: expandir el nodo informe_autores_json en el panel Outline y comparar la estructura.

D.3 — Documento PDF resultante, página por página

```

INFORME: informe_autores_json.pdf
PÁGINAS TOTALES: 1
TAMAÑO DE PÁGINA: 595 × 842 píxeles (A4 vertical)
ORIGEN DE DATOS: data/autores.json (expresión: autores)
  
```

ELEMENTOS SELECCIONADOS: 6 autores
 BANDAS EMITIDAS: Title, Column Header, Detail (6 veces),
 Page Footer, Background

Página 1 de 1

Catálogo de Autores - Datos desde JSON				
Nombre	Nacionalidad	Nacim.	Prem.	Estado
Gabriel García M.	Colombiana	06/03/27	1	Inactivo
Julio Cortázar	Argentina	26/08/14	0	Inactivo
Jorge Luis Borges	Argentina	24/08/99	2	Inactivo
Juan Rulfo	Mexicana	16/05/17	1	Inactivo
Isabel Allende	Chilena	02/08/42	2	Activo
Mario Vargas Llosa	Peruana	28/03/36	3	Activo
Total de autores: 6				
Página 1 de 1				

Qué representa: la página única del PDF resultante con los seis autores del JSON. Los valores de la fecha están formateados como dd/MM/yyyy. El estado se muestra como Activo o Inactivo según el valor booleano. Los premios se muestran como enteros.

Cómo verificarlo: abrir el archivo output/informe_autores_json.pdf con un lector de PDF y comprobar que aparecen los seis autores con todos sus datos. Si el estado no se muestra como Activo/Inactivo, revisar la expresión del campo.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```
EditorialReports/
├── ECOSISTEMA.md, ENTORNO.md, BANDAS.md, JRXML.md
├── TEXTO.md, CAMPOS.md, IMAGENES.md, ESTILOS.md, EXPRESIONES.md
├── BASEDATOS.md, CSV.md, XML.md, JSON.md (nuevo)
├── data/
│   ├── catalogo.csv
│   ├── distribucion.xml
│   └── autores.json (nuevo)
├── reports/
│   ├── informe_concepto.jrxml
│   ├── informe_catalogo_csv.jrxml
│   ├── informe_distribucion_xml.jrxml
│   └── informe_autores_json.jrxml (nuevo)
├── resources/
│   └── (logotipo, iconos y portadas)
└── output/
    ├── informe_concepto.pdf
    ├── informe_catalogo_csv.pdf
    ├── informe_distribucion_xml.pdf
    └── informe_autores_json.pdf (nuevo)

EditorialReportsJava/
├── lib/
│   ├── jasperreports-6.20.0.jar
│   ├── commons-digester-2.1.jar
│   ├── commons-collections-3.2.2.jar
│   ├── commons-logging-1.2.jar
│   ├── ecj-3.24.0.jar
│   ├── sqlite-jdbc-3.44.0.0.jar
│   ├── jackson-core-*.jar (nuevo)
│   ├── jackson-databind-*.jar (nuevo)
│   └── jackson-annotations-*.jar (nuevo)
├── data/
│   └── editorial.db
└── src/
    ├── GeneradorInformeConcepto.java
    ├── GeneradorCatalogoCSV.java
    ├── GeneradorDistribucionXML.java
    ├── GeneradorAutoresJSON.java (nueva clase)
    ├── Libro.java
    ├── CatalogoDataSource.java
    └── InicializadorBD.java
```

Qué representa: el estado de los dos proyectos tras completar los quince pasos. La novedad respecto al punto 3.3 es el archivo autores.json, el informe informe_autores_json.jrxml, la clase GeneradorAutoresJSON, los tres JAR de Jackson en la carpeta lib y el archivo JSON.md.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si los JAR de Jackson no aparecen, repetir los pasos 1 y 2.

Errores comunes del ejercicio completo

Error	Causa	Solución
NoClassDefFoundError: com/fasterxml/jackson/databind/ObjectMapper	Los JAR de Jackson no están en el classpath	Copiar los tres JAR a la carpeta lib y añadirlos al Build Path
FileNotFoundException: data/autores.json	El programa se ejecuta desde un directorio distinto a la raíz del proyecto	Configurar el Working Directory en Run Configurations
JRException: No JSON source was detected	La expresión de selección es incorrecta o el archivo no es un JSON válido	Verificar la expresión autores y la estructura del archivo
Field not found: premios	El campo no está declarado o el nombre no coincide con la propiedad del JSON	Añadir <field name="premios" class="java.lang.Integer"/>
ClassCastException al resolver un campo	El tipo declarado no coincide con el tipo del valor del JSON	Verificar que el tipo declarado coincide con el tipo del valor
La fecha no se formatea correctamente	El formato de la fecha en el JSON no es compatible con el que espera el motor	Ajustar el formato del JSON al formato ISO 8601 yyyy-MM-dd
El adaptador JSON no se conecta en Jaspersoft Studio	La expresión de selección es incorrecta	Usar la expresión autores sin barra inicial
El informe se previsualiza vacío	La expresión de selección no devuelve elementos	Verificar la estructura del JSON con un editor y ajustar la expresión
Las vocales acentuadas aparecen corruptas	El archivo JSON no está guardado en UTF-8	Guardar el archivo como UTF-8
JsonParseException al leer el archivo	El archivo JSON tiene un error de sintaxis	Validar el archivo con un editor JSON y corregir el error
El campo vivo no se muestra correctamente	Se usó \${vivo} sin invocar booleanValue() en la expresión	Usar \${vivo}.booleanValue() ? "Activo" : "Inactivo"

Reto resuelto paso a paso

Enunciado: añadir un filtro a la expresión de selección para que el informe muestre solo los autores vivos. Modificar la expresión de selección del JRXML y verificar el resultado en el PDF.

Paso 1. Hacer doble clic sobre el archivo informe_autores_json.jrxml en el panel Project Explorer.

Paso 2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.

Paso 3. Localizar la línea que contiene <![CDATA[autores]]>.

Paso 4. Seleccionar el contenido de la expresión autores y eliminarlo.

Paso 5. Escribir exactamente autoresautores(vivo == true) y pulsar Enter.

Paso 6. Pulsar Ctrl+S para guardar el archivo.

Paso 7. Pulsar Ctrl+Mayús+B para compilar el informe.

Paso 8. Hacer clic sobre el panel Problems y verificar que no hay errores.

Paso 9. Hacer clic con el botón derecho sobre GeneradorAutoresJSON.java y seleccionar Run As > Java Application.

Paso 10. Abrir el archivo output/informe_autores_json.pdf y verificar que solo aparecen los autores vivos.

Simulación ASCII del PDF tras el reto

TEXT

```

|| Nombre | Nacionalidad | Nacim. | Prem. | Estado ||
|| -----|-----|-----|-----|-----||
|| Isabel Allende | Chilena | 02/08/42 | 2 | Activo ||
|| Mario Vargas Llosa | Peruana | 28/03/36 | 3 | Activo ||
|| Total de autores: 2 ||

```

Resultado del reto: la expresión de selección con filtro `autores(autores(vivo == true))` reduce los registros del informe a los dos autores cuyo campo `vivo` es verdadero. El resto de autores queda excluido. La sintaxis del filtro es similar a la de `JSONPath` y permite seleccionar subconjuntos de registros sin procesar el archivo completo en el programa Java.

Analogía final con el contexto de la editorial

El archivo JSON es el listado de autores que la editorial recibe de un servicio web o de una API externa. Cada objeto del arreglo es un autor con sus datos biográficos. La expresión de selección `autores` es la instrucción que indica al motor qué arreglo debe recorrer. Las expresiones de los campos son las instrucciones que indican al motor qué propiedad extraer de cada autor. El adaptador JSON de Jaspersoft Studio es la herramienta que permite consultar el archivo desde la mesa de diseño. La clase `JsonDataSource` es la herramienta equivalente desde la consola de control. El filtro `autores(vivo == true)` es la condición que permite seleccionar solo los autores vivos. La conversión automática de tipos es la ventaja que hace que el JSON sea más cómodo que el CSV y el XML para datos con tipos definidos.

Resultado esperado

Al finalizar este punto, el alumno dispone de:

- Los tres JAR de Jackson en la carpeta `lib` del proyecto Java y referenciados en el Build Path.
- El archivo `data/autores.json` con el arreglo de seis autores.
- El adaptador `AutoresJSON` en el panel Repository Explorer de Jaspersoft Studio.
- El archivo `reports/informe_autores_json.jrxml` con la consulta JSON y los cinco campos con sus tipos.
- El archivo `output/informe_autores_json.pdf` con los seis autores y los valores formateados.
- La clase `GeneradorAutoresJSON.java` que lee el JSON desde código Java.
- El archivo `JSON.md` en la raíz del proyecto con la documentación del uso de JSON.
- Comprensión operativa de la lectura de JSON, de las expresiones de selección y de la conversión automática de tipos.

Conclusión y enlace al siguiente punto

El punto 3.4 ha introducido la lectura de archivos JSON como fuente de datos para el informe. Han quedado configurados el adaptador JSON en Jaspersoft Studio y la clase `JsonDataSource` en el programa Java. El informe `informe_autores_json.jrxml` se alimenta del archivo `autores.json` y muestra los seis autores con el nombre, la nacionalidad, la fecha de nacimiento ISO, el número de premios y el estado. La conversión tipada se demuestra con `premios` y `vivo`; `nacimiento` se mantiene como texto ISO de forma explícita.

Punto 3.5 — Consultas SQL

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial

Punto: 3.5 — Consultas SQL

Objetivos de aprendizaje

- Escribir consultas SQL completas dentro del elemento `queryString` del JRXML.
- Utilizar alias de columna para mapear nombres de la base de datos a nombres de campo.
- Aplicar funciones de agregación (SUM, AVG, COUNT, MAX, MIN) en las consultas.
- Construir consultas con JOIN entre dos o más tablas.
- Utilizar GROUP BY y ORDER BY para agrupar y ordenar resultados.
- Documentar las consultas SQL del proyecto EditorialReports.

Parte práctica

Parte A — Práctica visual

Paso 1: Ampliar la base de datos con la tabla ventas

Acciones:

1. Hacer doble clic sobre el archivo `InicializadorBD.java` en el panel Project Explorer (superior izquierdo).
2. Hacer clic al final de la línea que contiene `sentencia.executeUpdate("DROP TABLE IF EXISTS libros");` y pulsar Enter.
3. Escribir exactamente `sentencia.executeUpdate("DROP TABLE IF EXISTS ventas");` y pulsar Enter.
4. Hacer clic al final de la línea que contiene `"disponible INTEGER NOT NULL");` y pulsar Enter.
5. Escribir exactamente `sentencia.executeUpdate(` y pulsar Enter.
6. Escribir exactamente `"CREATE TABLE ventas ( "` y pulsar Enter.
7. Escribir exactamente `"id INTEGER PRIMARY KEY AUTOINCREMENT, " "` y pulsar Enter.
8. Escribir exactamente `"titulo_libro TEXT NOT NULL, " "` y pulsar Enter.
9. Escribir exactamente `"cantidad INTEGER NOT NULL, " "` y pulsar Enter.
10. Escribir exactamente `"precio_unitario REAL NOT NULL, " "` y pulsar Enter.
11. Escribir exactamente `"fecha_venta TEXT NOT NULL");` y pulsar Enter.
12. Hacer clic al final de la última sentencia INSERT de la tabla libros y pulsar Enter.
13. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (1, 'Cien años de soledad', 3, 19.95, '2026-09-01'))");` y pulsar Enter.
14. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (2, 'Cien años de soledad', 5, 19.95, '2026-09-05'))");` y pulsar Enter.
15. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (3, 'Rayuela', 2, 22.50, '2026-09-03'))");` y pulsar Enter.
16. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (4, 'Rayuela', 4, 22.50, '2026-09-07'))");` y pulsar Enter.
17. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (5, 'Pedro Páramo', 6, 15.90, '2026-09-04'))");` y pulsar Enter.
18. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (6, 'Ficciones', 3, 21.00, '2026-09-06'))");` y pulsar Enter.
19. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (7, 'La casa de los espíritus', 5, 23.40, '2026-09-08'))");` y pulsar Enter.
20. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (8, 'Comala', 2, 19.20, '2026-09-09'))");` y pulsar Enter.
21. Escribir exactamente `sentencia.executeUpdate("INSERT INTO ventas VALUES (9, 'Paradiso', 1, 25.00, '2026-09-10'))");` y pulsar Enter.
22. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el editor central muestra la clase `InicializadorBD` con las sentencias de creación de la tabla `ventas` y las nueve sentencias `INSERT`. El panel `Problems` permanece vacío.

Qué hace: amplía la clase `InicializadorBD` con la tabla `ventas` y los datos de ejemplo.

Por qué: la tabla de `ventas` permite construir consultas con `JOIN` y con funciones de agregación.

Error común: olvidar el `DROP TABLE IF EXISTS ventas` y provocar un error al ejecutar la clase por segunda vez.

Solución: incluir la sentencia `DROP` antes del `CREATE TABLE`.

Analogía: es como añadir al archivador de la editorial un registro de `ventas` además de las fichas de libros.

Paso 2: Ejecutar el inicializador para recrear la base de datos

Acciones:

1. Hacer clic con el botón derecho sobre el archivo `InicializadorBD.java` en el panel `Project Explorer`.
2. Hacer clic sobre la opción `Run As` en el menú contextual.
3. Hacer clic sobre la opción `Java Application` en el submenú.
4. Hacer clic sobre la vista `Console` en el panel inferior y observar el resultado.

Verificación visual: la vista `Console` muestra la línea `Base de datos inicializada correctamente en: jdbc:sqlite:../EditorialReportsJava/data/editorial.db`.

Qué hace: ejecuta el inicializador que recrea la base de datos con las dos tablas.

Por qué: la base de datos debe contener la tabla `ventas` antes de construir la consulta.

Error común: ejecutar la clase sin haber guardado los cambios y obtener un error de sintaxis.

Solución: pulsar `Ctrl+S` antes de ejecutar.

Analogía: es como actualizar el archivador de la editorial con el nuevo registro de `ventas`.

Paso 3: Crear el informe `informe_ventas.jrxml`

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta `reports` en el panel `Project Explorer` (superior izquierdo).
2. Hacer clic sobre la opción `New` en el menú contextual.
3. Hacer clic sobre la opción `Jasper Report` en el submenú.
4. Hacer clic sobre la plantilla `Blank A4` en la lista de plantillas del asistente.
5. Hacer clic sobre el botón `Next`.
6. Escribir exactamente `informe_ventas` en el campo `File name`.
7. Hacer clic sobre el botón `Next`.
8. Hacer clic sobre `SQLiteEditorial` en la lista de adaptadores disponibles.
9. Hacer clic sobre el botón `Finish`.

Verificación visual: el editor central muestra el archivo `informe_ventas.jrxml` con las bandas por defecto.

Qué hace: crea un nuevo informe asociado al adaptador `SQLite`.

Por qué: el informe de `ventas` obtiene los datos de la base de datos.

Error común: seleccionar el adaptador `CatalogoCSV` o `AutoresJSON` por error.

Solución: cerrar el asistente y repetir el paso seleccionando `SQLiteEditorial`.

Analogía: es como abrir un nuevo pliego del catálogo con la estructura de la tabla de ventas.

Paso 4: Declarar la consulta SQL con JOIN y agregación

Acciones:

1. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
2. Localizar la línea que contiene `<queryString language="sql">` y seleccionar el bloque completo.
3. Eliminar el bloque con la tecla Suprimir.
4. Escribir exactamente `<queryString language="sql">` y pulsar Enter.
5. Escribir exactamente `<![CDATA[` y pulsar Enter.
6. Escribir exactamente `SELECT l.titulo,` y pulsar Enter.
7. Escribir exactamente `SUM(v.cantidad) AS unidades_vendidas,` y pulsar Enter.
8. Escribir exactamente `SUM(v.cantidad * v.precio_unitario) AS importe_total,` y pulsar Enter.
9. Escribir exactamente `AVG(v.precio_unitario) AS precio_medio` y pulsar Enter.
10. Escribir exactamente `FROM libros l` y pulsar Enter.
11. Escribir exactamente `INNER JOIN ventas v ON l.titulo = v.titulo_libro` y pulsar Enter.
12. Escribir exactamente `GROUP BY l.titulo` y pulsar Enter.
13. Escribir exactamente `ORDER BY importe_total DESC` y pulsar Enter.
14. Escribir exactamente `]]>` y pulsar Enter.
15. Escribir exactamente `</queryString>` y pulsar Enter.
16. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el editor central muestra la consulta SQL con el JOIN, el GROUP BY y las funciones de agregación. El panel Outline muestra el nodo QueryString con la consulta.

Qué hace: declara la consulta SQL que combina las tablas libros y ventas y calcula las agregaciones por título.

Por qué: la consulta obtiene el total de unidades vendidas y el importe total por libro.

Error común: olvidar la cláusula GROUP BY y provocar que la consulta devuelva una sola fila con los totales globales.

Solución: añadir GROUP BY l.titulo antes de ORDER BY.

Analogía: es como pedir al archivero un resumen de las ventas de cada libro.

Paso 5: Declarar los campos del informe

Acciones:

1. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
2. Localizar la línea que contiene `<field name="disponible" class="java.lang.Boolean"/>` y seleccionar las líneas de campos existentes.
3. Eliminar las líneas con la tecla Suprimir.
4. Escribir exactamente `<field name="titulo" class="java.lang.String"/>` y pulsar Enter.
5. Escribir exactamente `<field name="unidades_vendidas" class="java.lang.Integer"/>` y pulsar Enter.
6. Escribir exactamente `<field name="importe_total" class="java.lang.Double"/>` y pulsar Enter.
7. Escribir exactamente `<field name="precio_medio" class="java.lang.Double"/>` y pulsar Enter.
8. Pulsar Ctrl+S para guardar el archivo.
9. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
10. Expandir el nodo Fields en el panel Outline y verificar que aparecen los cuatro campos.

Verificación visual: el panel Outline muestra el nodo Fields con cuatro entradas: titulo, unidades_vendidas, importe_total y precio_medio.

Qué hace: declara los cuatro campos que corresponden a las cuatro columnas de la consulta.

Por qué: los nombres de los campos deben coincidir con los alias de la consulta.

Error común: declarar el campo `importe_total` como `java.lang.Integer`. La consulta devuelve un valor decimal y el motor lanza una excepción de conversión.

Solución: declarar el campo como `java.lang.Double`.

Analogía: es como definir las columnas del resumen de ventas.

Paso 6: Ajustar las bandas del informe

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `informe_ventas` en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la opción Delete en el menú contextual para eliminar el nodo Page Header.
3. Hacer clic con el botón derecho sobre el nodo `informe_ventas` y eliminar el nodo Column Footer.
4. Hacer clic con el botón derecho sobre el nodo `informe_ventas` y eliminar el nodo Summary.
5. Hacer clic sobre el nodo Title en el panel Outline y ajustar su Band height a 60 píxeles desde el panel Properties.

Verificación visual: el panel Outline muestra solo las bandas Title, Column Header, Detail 1, Page Footer y Background.

Qué hace: simplifica el informe para que contenga solo las bandas necesarias.

Por qué: el informe de ventas necesita un número reducido de bandas.

Error común: eliminar la banda Background pensando que no se usa.

Solución: si se elimina por error, cerrar el archivo sin guardar y volver a abrirlo.

Analogía: es como reducir el pliego del resumen de ventas a las secciones necesarias.

Paso 7: Añadir el título en la banda Title

Acciones:

1. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
2. Hacer clic sobre el icono Static Text (una letra T mayúscula).
3. Arrastrar el icono Static Text y soltarlo dentro de la banda Title, en la coordenada aproximada $x=0, y=15$.
4. Hacer doble clic sobre el Static Text creado en la acción anterior.
5. Escribir exactamente Informe de Ventas - Agregación por Título.
6. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
7. Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 0 y pulsar Enter.
8. Hacer clic sobre el campo Y, escribir 15 y pulsar Enter.
9. Hacer clic sobre el campo Width, escribir 555 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 30 y pulsar Enter.
11. Hacer clic sobre el campo Font size y escribir 18. Pulsar Enter.
12. Marcar la casilla Bold.
13. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.

Verificación visual: la banda Title muestra el texto Informe de Ventas - Agregación por Título centrado y en negrita.

Qué hace: inserta el título del informe de ventas.

Por qué: el título identifica el documento y su propósito.

Error común: olvidar el centrado.

Solución: seleccionar Center en el desplegable Horizontal Text Alignment.

Analogía: es como titular el resumen de ventas con el nombre de la editorial.

Paso 8: Añadir los encabezados de columna

Acciones:

1. Hacer clic sobre el nodo Column Header en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Static Text.
4. Arrastrar el icono Static Text y soltarlo dentro de la banda Column Header, en la coordenada aproximada x=0, y=5.
5. Hacer doble clic sobre el Static Text creado en la acción anterior.
6. Escribir exactamente **Título**.
7. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
8. Hacer clic sobre el campo Width en el panel Properties, escribir 250 y pulsar Enter.
9. Marcar la casilla Bold.
10. Repetir las acciones 3 a 9 para los encabezados Unidades (x=250, ancho 90), Importe total (x=340, ancho 130) y Precio medio (x=470, ancho 85).

Verificación visual: la banda Column Header muestra los cuatro encabezados en negrita.

Qué hace: inserta los encabezados de las columnas del informe de ventas.

Por qué: los encabezados identifican las columnas de la tabla.

Error común: dejar los encabezados sin negrita.

Solución: marcar la casilla Bold.

Analogía: es como añadir los títulos de las columnas al resumen de ventas.

Paso 9: Añadir los campos en la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).
4. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=0, y=0.
5. Hacer clic sobre el campo Width en el panel Properties, pestaña Properties, escribir 250 y pulsar Enter.
6. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.
7. Hacer clic sobre el campo Text Field Expression y escribir exactamente \$F{titulo} y pulsar Enter.
8. Repetir las acciones 3 a 7 para el campo unidades_vendidas (x=250, ancho 90, alineación derecha).
9. Para el campo del importe total, hacer clic sobre el icono Text Field y arrastrarlo dentro de la banda Detail 1, en la coordenada aproximada x=340, y=0.
10. Hacer clic sobre el campo Width y escribir 130. Pulsar Enter.
11. Hacer clic sobre el campo Height y escribir 20. Pulsar Enter.
12. Hacer clic sobre el campo Text Field Expression y escribir exactamente \$F{importe_total} y pulsar Enter.
13. Hacer clic sobre el campo Pattern y escribir exactamente #,##0.00 €. Pulsar Enter.
14. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Right.
15. Para el campo del precio medio, hacer clic sobre el icono Text Field y arrastrarlo dentro de la banda Detail 1, en la coordenada aproximada x=470, y=0.
16. Hacer clic sobre el campo Width y escribir 85. Pulsar Enter.
17. Hacer clic sobre el campo Height y escribir 20. Pulsar Enter.
18. Hacer clic sobre el campo Text Field Expression y escribir exactamente \$F{precio_medio} y pulsar Enter.
19. Hacer clic sobre el campo Pattern y escribir exactamente #,##0.00 €. Pulsar Enter.
20. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Right.

Verificación visual: la banda Detail 1 muestra cuatro campos con las expresiones correspondientes. Los campos numéricos tienen el patrón #,##0.00 €.

Qué hace: inserta los campos que se imprimen para cada libro con sus agregaciones.

Por qué: los campos resuelven las columnas calculadas por la consulta SQL.

Error común: olvidar el patrón en los campos numéricos y provocar que se muestren sin decimales.

Solución: añadir el patrón #,##0.00 €.

Analogía: es como rellenar las celdas del resumen de ventas con los datos agregados.

Paso 10: Ajustar la altura de la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 20 y pulsar Enter.
3. Hacer clic sobre el desplegable Split Type en el panel Properties y verificar que está en Stretch.

Verificación visual: la banda Detail 1 aparece con 20 píxeles de altura.

Qué hace: fija la altura de la banda Detail para que cada libro ocupe una fila.

Por qué: la altura determina el espacio de cada fila.

Error común: dejar la altura por defecto.

Solución: ajustar la altura a 20 píxeles.

Analogía: es como ajustar la altura de cada fila del resumen de ventas.

Paso 11: Añadir el pie de página con totales

Acciones:

1. Hacer clic sobre el nodo Page Footer en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 60 y pulsar Enter.
3. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
4. Hacer clic sobre el icono Static Text.
5. Arrastrar el icono Static Text y soltarlo dentro de la banda Page Footer, en la coordenada aproximada x=0, y=5.
6. Hacer doble clic sobre el Static Text creado en la acción anterior.
7. Escribir exactamente Total de títulos:.
8. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
9. Hacer clic sobre el campo Width en el panel Properties, escribir 150 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
11. Hacer clic sobre la pestaña Elements en el panel Palette.
12. Hacer clic sobre el icono Text Field.
13. Arrastrar el icono Text Field y soltarlo dentro de la banda Page Footer, a la derecha del rótulo, en la coordenada aproximada x=150, y=5.
14. Hacer clic sobre el campo Width y escribir 50. Pulsar Enter.
15. Hacer clic sobre el campo Height y escribir 15. Pulsar Enter.
16. Hacer clic sobre el campo Text Field Expression y escribir exactamente \$V{REPORT_COUNT} y pulsar Enter.
17. Marcar la casilla Bold.
18. Hacer clic sobre la pestaña Elements en el panel Palette y hacer clic sobre el icono Text Field.
19. Arrastrar el icono Text Field y soltarlo dentro de la banda Page Footer, debajo del rótulo anterior, en la coordenada aproximada x=0, y=25.

20. Hacer clic sobre el campo Width y escribir 555. Pulsar Enter.
21. Hacer clic sobre el campo Height y escribir 15. Pulsar Enter.
22. Hacer clic sobre el campo Text Field Expression y escribir exactamente "Página " + `$V{PAGE_NUMBER}` + " de " + [segundo campo `PAGE_NUMBER` con `evaluationTime=Report`] y pulsar Enter.
23. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.
24. Hacer clic sobre el campo Font size y escribir 9. Pulsar Enter.
25. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: la banda Page Footer muestra el rótulo del total de títulos con su valor en negrita y el campo de paginación centrado debajo.

Qué hace: inserta un pie de página con el recuento de títulos y la paginación.

Por qué: el recuento y la paginación informan al lector sobre el volumen y la posición en el documento.

Error común: olvidar el centrado del campo de paginación.

Solución: seleccionar Center.

Analogía: es como anotar al pie del resumen de ventas el total de títulos y la página.

Paso 12: Compilar y previsualizar con el adaptador SQLite

Acciones:

1. Pulsar Ctrl+Mayús+B para compilar el informe.
2. Hacer clic sobre el panel Problems (inferior) y verificar que no hay errores.
3. Pulsar el botón Preview de la barra de herramientas superior.
4. En el diálogo, verificar que el adaptador `SQLiteEditorial` está seleccionado.
5. Hacer clic sobre el botón OK.
6. Esperar a que se abra la pestaña Preview en el editor central.

Verificación visual: la pestaña Preview muestra el informe con los libros que tienen ventas, ordenados por importe total descendente. Cada fila muestra el título, las unidades vendidas, el importe total y el precio medio.

Qué hace: ejecuta el informe con los datos de la base de datos y muestra el resultado.

Por qué: la previsualización confirma que la consulta SQL con JOIN y agregación funciona correctamente.

Error común: obtener `SQLException: no such table: ventas`. Indica que la tabla no existe.

Solución: ejecutar de nuevo el `InicializadorBD`.

Analogía: es como revisar la prueba de color del resumen de ventas.

Paso 13: Crear la clase `GeneradorInformeVentas`

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta `src` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Class en el submenú.
4. Escribir exactamente `GeneradorInformeVentas` en el campo Name del diálogo.
5. Marcar la casilla `public static void main(String[] args)`.
6. Hacer clic sobre el botón Finish.
7. En el editor central, escribir el código completo de la clase `GeneradorInformeVentas` que se muestra en la Parte C de este punto.
8. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `GeneradorInformeVentas.java` dentro de la carpeta `src`.

Qué hace: crea la clase que genera el informe de ventas desde la base de datos.

Por qué: el informe puede ejecutarse desde código Java sin depender de Jaspersoft Studio.

Error común: olvidar importar `java.sql.Connection` y `java.sql.DriverManager`.

Solución: añadir las importaciones correspondientes.

Analogía: es como preparar la consola de control para que el operario genere el resumen de ventas.

Paso 14: Ejecutar el programa y verificar el PDF

Acciones:

1. Hacer clic con el botón derecho sobre el archivo `GeneradorInformeVentas.java` en el panel Project Explorer.
2. Hacer clic sobre la opción `Run As` en el menú contextual.
3. Hacer clic sobre la opción `Java Application` en el submenú.
4. Hacer clic sobre la vista `Console` en el panel inferior y observar el resultado.
5. Abrir el explorador de archivos del sistema operativo.
6. Navegar hasta la carpeta `output` del proyecto `EditorialReports`.
7. Hacer doble clic sobre el archivo `informe_ventas.pdf`.
8. Verificar que el PDF muestra los libros con ventas, ordenados por importe total descendente.

Verificación visual: la vista `Console` muestra la línea `Informe generado en: ...` con la ruta absoluta del PDF. El archivo PDF muestra los libros con ventas.

Qué hace: ejecuta el programa Java que consulta la base de datos y genera el informe.

Por qué: la ejecución confirma que la consulta SQL funciona desde código Java.

Error común: ejecutar el programa desde un directorio distinto a la raíz del proyecto.

Solución: comprobar en `Run Configurations` que el `Working Directory` apunta a la raíz del proyecto.

Analogía: es como imprimir el resumen de ventas desde la base de datos.

Paso 15: Documentar las consultas SQL

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `EditorialReports` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción `New` en el menú contextual.
3. Hacer clic sobre la opción `File` en el submenú.
4. Escribir exactamente `CONSULTAS.md` en el campo `File name` del diálogo.
5. Hacer clic sobre el botón `Finish`.
6. En el editor central, escribir exactamente `# Consultas SQL del proyecto` y pulsar `Enter` dos veces.
7. Escribir exactamente `## Tablas` y pulsar `Enter` dos veces.
8. Escribir exactamente `- libros (titulo, precio, paginas, fecha_publicacion, disponible)` y pulsar `Enter`.
9. Escribir exactamente `- ventas (id, titulo_libro, cantidad, precio_unitario, fecha_venta)` y pulsar `Enter` dos veces.
10. Escribir exactamente `## Consultas` y pulsar `Enter` dos veces.
11. Escribir exactamente `### informe_ventas.jrxml` y pulsar `Enter` dos veces.
12. Escribir exactamente `- Consulta: SELECT l.titulo, SUM(v.cantidad), SUM(v.cantidad * v.precio_unitario), AVG(v.precio_unitario) FROM libros l INNER JOIN ventas v ON l.titulo = v.titulo_libro GROUP BY l.titulo ORDER BY importe_total DESC` y pulsar `Enter`.
13. Pulsar `Ctrl+S` para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo CONSULTAS.md en la raíz del proyecto EditorialReports.

Qué hace: incorpora al proyecto un documento que registra las consultas SQL.

Por qué: la documentación de las consultas facilita el mantenimiento y la incorporación de nuevos desarrolladores.

Error común: olvidar documentar las tablas.

Solución: incluir las secciones especificadas.

Analogía: es como dejar en la editorial una ficha técnica con las consultas utilizadas en el resumen de ventas.

Parte B — JRXML completo explicado línea por línea [VALIDADO]

JRXML canónico del checkpoint. Es el mismo archivo que se compila en GitHub Actions.

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_ventas"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
  leftMargin="20"
  rightMargin="20"
  topMargin="20"
  bottomMargin="20"
  uuid="e6b7c8d9-f0a1-2b3c-4d5e-6f7a8b9c0d1e">
  <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
  <style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
  <queryString language="sql">
    <![CDATA[
      SELECT l.titulo,
             SUM(v.cantidad) AS unidades_vendidas,
             SUM(v.cantidad * v.precio_unitario) AS importe_total,
             AVG(v.precio_unitario) AS precio_medio
      FROM libros l
      INNER JOIN ventas v ON l.titulo = v.titulo_libro
      GROUP BY l.titulo
      ORDER BY importe_total DESC
    ]]>
  </queryString>
  <field name="titulo" class="java.lang.String"/>
  <field name="unidades_vendidas" class="java.lang.Integer"/>
  <field name="importe_total" class="java.lang.Double"/>
  <field name="precio_medio" class="java.lang.Double"/>
  <background>
    <band height="0"/>
  </background>

  <title>
    <band height="60">
      <staticText>
        <reportElement x="0" y="15" width="555" height="30" uuid="f7c8d9e0-a1b2-3c4d-5e6f-7a8b9c0d1e2f"/>
        <textElement textAlignment="Center" verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="18" isBold="true"/>
        </textElement>
        <text><![CDATA[Informe de Ventas - Agregación por Título]]></text>
      </staticText>
    </band>
  </title>
  <columnHeader>
    <band height="25">
      <staticText>
        <reportElement x="0" y="5" width="250" height="15" uuid="a8d9e0f1-b2c3-4d5e-6f7a-8b9c0d1e2f3a"/>
        <textElement verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Título]]></text>
      </staticText>
      <staticText>
        <reportElement x="250" y="5" width="90" height="15" uuid="b9e0f1a2-c3d4-5e6f-7a8b-9c0d1e2f3a4b"/>
        <textElement textAlignment="Right" verticalAlignment="Middle">
          <font fontName="DejaVu Sans" size="10" isBold="true"/>
        </textElement>
        <text><![CDATA[Unidades]]></text>
      </staticText>
    </band>
  </columnHeader>
```

XML

```

<staticText>
  <reportElement x="340" y="5" width="130" height="15" uuid="c0f1a2b3-d4e5-6f7a-8b9c-0d1e2f3a4b5c"/>
  <textElement textAlignment="Right" verticalAlignment="Middle">
    <font fontName="DejaVu Sans" size="10" isBold="true"/>
  </textElement>
  <text><![CDATA[Importe total]]></text>
</staticText>
<staticText>
  <reportElement x="470" y="5" width="85" height="15" uuid="d1a2b3c4-e5f6-7a8b-9c0d-1e2f3a4b5c6d"/>
  <textElement textAlignment="Right" verticalAlignment="Middle">
    <font fontName="DejaVu Sans" size="10" isBold="true"/>
  </textElement>
  <text><![CDATA[Precio medio]]></text>
</staticText>
</band>
</columnHeader>
<detail>
  <band height="20" splitType="Stretch">
    <textField textAdjust="StretchHeight">
      <reportElement x="0" y="0" width="250" height="20" uuid="e2b3c4d5-f6a7-8b9c-0d1e-2f3a4b5c6d7e"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[${titulo}]]></textFieldExpression>
    </textField>
    <textField>
      <reportElement x="250" y="0" width="90" height="20" uuid="f3c4d5e6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[${unidades_vendidas}]]></textFieldExpression>
    </textField>
    <textField pattern="#,##0.00 €">
      <reportElement x="340" y="0" width="130" height="20" uuid="a4d5e6f7-b8c9-0d1e-2f3a-4b5c6d7e8f9a"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[${importe_total}]]></textFieldExpression>
    </textField>
    <textField pattern="#,##0.00 €">
      <reportElement x="470" y="0" width="85" height="20" uuid="b5e6f7a8-c9d0-1e2f-3a4b-5c6d7e8f9a0b"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[${precio_medio}]]></textFieldExpression>
    </textField>
  </band>
</detail>
<pageFooter>
  <band height="45">
    <staticText>
      <reportElement x="0" y="3" width="150" height="15" uuid="44444444-4444-4444-8444-444444444441"/>
      <textElement verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
      <text><![CDATA[Total de títulos:]]></text>
    </staticText>
    <textField>
      <reportElement x="150" y="3" width="70" height="15" uuid="44444444-4444-4444-8444-444444444442"/>
      <textElement verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9" isBold="true"/></textElement>
      <textFieldExpression><![CDATA[${V{REPORT_COUNT}}]]></textFieldExpression>
    </textField>
    <textField>
      <reportElement x="170" y="3" width="190" height="15" uuid="44444444-4444-4444-8444-444444444443"/>
      <textElement textAlignment="Right" verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
      <textFieldExpression><![CDATA["Página " + ${V{PAGE_NUMBER}} + " de"]></textFieldExpression>
    </textField>
    <textField evaluationTime="Report">
      <reportElement x="365" y="3" width="30" height="15" uuid="44444444-4444-4444-8444-444444444444"/>
      <textElement textAlignment="Left" verticalAlignment="Middle"><font fontName="DejaVu Sans" size="9"/></textElement>
      <textFieldExpression><![CDATA[${V{PAGE_NUMBER}}]]></textFieldExpression>
    </textField>
  </band>
</pageFooter>
</jasperReport>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → Declara el documento XML y la codificación UTF-8.
Línea 2	<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" → Abre la plantilla JasperReports y define sus atributos principales.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → Completa la definición declarativa del informe.

Línea 4	<code>xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd"</code> → Declara el espacio de nombres/XSD usado para validar el JRXML.
Línea 5	<code>name="informe_ventas"</code> → Completa la definición declarativa del informe.
Línea 6	<code>language="java"</code> → Completa la definición declarativa del informe.
Línea 7	<code>pageWidth="595"</code> → Completa la definición declarativa del informe.
Línea 8	<code>pageHeight="842"</code> → Completa la definición declarativa del informe.
Línea 9	<code>columnWidth="555"</code> → Completa la definición declarativa del informe.
Línea 10	<code>leftMargin="20"</code> → Completa la definición declarativa del informe.
Línea 11	<code>rightMargin="20"</code> → Completa la definición declarativa del informe.
Línea 12	<code>topMargin="20"</code> → Completa la definición declarativa del informe.
Línea 13	<code>bottomMargin="20"</code> → Completa la definición declarativa del informe.
Línea 14	<code>uuid="e6b7c8d9-f0a1-2b3c-4d5e-6f7a8b9c0d1e"></code> → Completa la definición declarativa del informe.
Línea 15	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/></code> → Asocia el Data Adapter usado por Jaspersoft Studio durante Preview.
Línea 16	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/></code> → Declara un estilo compatible con JasperReports 6.20.0.
Línea 17	<code><queryString language="sql"></code> → Abre la consulta del dataset e indica el lenguaje de consulta.
Línea 18	<code><![CDATA[</code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 19	<code>SELECT l.titulo,</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 20	<code>SUM(v.cantidad) AS unidades_vendidas,</code> → Completa la definición declarativa del informe.
Línea 21	<code>SUM(v.cantidad * v.precio_unitario) AS importe_total,</code> → Completa la definición declarativa del informe.
Línea 22	<code>AVG(v.precio_unitario) AS precio_medio</code> → Completa la definición declarativa del informe.
Línea 23	<code>FROM libros l</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 24	<code>INNER JOIN ventas v ON l.titulo = v.titulo_libro</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 25	<code>GROUP BY l.titulo</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 26	<code>ORDER BY importe_total DESC</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 27	<code>]]></code> → Completa la definición declarativa del informe.
Línea 28	<code></queryString></code> → Cierra el elemento XML abierto anteriormente.
Línea 29	<code><field name="titulo" class="java.lang.String"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 30	<code><field name="unidades_vendidas" class="java.lang.Integer"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 31	<code><field name="importe_total" class="java.lang.Double"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 32	<code><field name="precio_medio" class="java.lang.Double"/></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 33	<code><background></code> → Abre una sección/banda estructural del informe.

Línea 34	<code><band height="0"/></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 35	<code></background></code> → Cierra el elemento XML abierto anteriormente.
Línea 36	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 37	<code><title></code> → Abre una sección/banda estructural del informe.
Línea 38	<code><band height="60"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 39	<code><staticText></code> → Abre un elemento de texto estático.
Línea 40	<code><reportElement x="0" y="15" width="555" height="30" uuid="f7c8d9e0-a1b2-3c4d-5e6f-7a8b9c0d1e2f"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 41	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 42	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 43	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 44	<code><text><![CDATA[Informe de Ventas - Agregación por Título]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 45	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 46	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 47	<code></title></code> → Cierra el elemento XML abierto anteriormente.
Línea 48	<code><columnHeader></code> → Abre una sección/banda estructural del informe.
Línea 49	<code><band height="25"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 50	<code><staticText></code> → Abre un elemento de texto estático.
Línea 51	<code><reportElement x="0" y="5" width="250" height="15" uuid="a8d9e0f1-b2c3-4d5e-6f7a-8b9c0d1e2f3a"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 52	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 53	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 54	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 55	<code><text><![CDATA[Título]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 56	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 57	<code><staticText></code> → Abre un elemento de texto estático.
Línea 58	<code><reportElement x="250" y="5" width="90" height="15" uuid="b9e0f1a2-c3d4-5e6f-7a8b-9c0d1e2f3a4b"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 59	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 60	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 61	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 62	<code><text><![CDATA[Unidades]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 63	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.

Línea 64	<code><staticText></code> → Abre un elemento de texto estático.
Línea 65	<code><reportElement x="340" y="5" width="130" height="15" uuid="c0f1a2b3-d4e5-6f7a-8b9c-0d1e2f3a4b5c"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 66	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 67	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 68	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 69	<code><text><![CDATA[Importe total]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 70	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 71	<code><staticText></code> → Abre un elemento de texto estático.
Línea 72	<code><reportElement x="470" y="5" width="85" height="15" uuid="d1a2b3c4-e5f6-7a8b-9c0d-1e2f3a4b5c6d"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 73	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 74	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 75	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 76	<code><text><![CDATA[Precio medio]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 77	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 78	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 79	<code></columnHeader></code> → Cierra el elemento XML abierto anteriormente.
Línea 80	<code><detail></code> → Abre una sección/banda estructural del informe.
Línea 81	<code><band height="20" splitType="Stretch"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 82	<code><textField textAdjust="StretchHeight"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 83	<code><reportElement x="0" y="0" width="250" height="20" uuid="e2b3c4d5-f6a7-8b9c-0d1e-2f3a4b5c6d7e"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 84	<code><textElement verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 85	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 86	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 87	<code><textFieldExpression><![CDATA[\${titulo}]]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 88	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 89	<code><textField></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 90	<code><reportElement x="250" y="0" width="90" height="20" uuid="f3c4d5e6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.

Línea 91	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 92	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 93	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 94	<code><textFieldExpression><![CDATA[\${F{unidades_vendidas}}]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 95	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 96	<code><textField pattern="# ,##0.00 €"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 97	<code><reportElement x="340" y="0" width="130" height="20" uuid="a4d5e6f7-b8c9-0d1e-2f3a-4b5c6d7e8f9a"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 98	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 99	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 100	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 101	<code><textFieldExpression><![CDATA[\${F{importe_total}}]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 102	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 103	<code><textField pattern="# ,##0.00 €"></code> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 104	<code><reportElement x="470" y="0" width="85" height="20" uuid="b5e6f7a8-c9d0-1e2f-3a4b-5c6d7e8f9a0b"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 105	<code><textElement textAlignment="Right" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 106	<code></code> → Configura tipografía, tamaño y énfasis.
Línea 107	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 108	<code><textFieldExpression><![CDATA[\${F{precio_medio}}]></textFieldExpression></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 109	<code></textField></code> → Cierra el elemento XML abierto anteriormente.
Línea 110	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 111	<code></detail></code> → Cierra el elemento XML abierto anteriormente.
Línea 112	<code><pageFooter></code> → Abre una sección/banda estructural del informe.
Línea 113	<code><band height="45"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 114	<code><staticText></code> → Abre un elemento de texto estático.
Línea 115	<code><reportElement x="0" y="3" width="150" height="15" uuid="44444444-4444-4444-8444-444444444441"/></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 116	<code><textElement verticalAlignment="Middle"></textElement></code> → Configura alineación y propiedades del contenido textual.
Línea 117	<code><text><![CDATA[Total de títulos:]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.

Línea 118	</staticText> → Cierra el elemento XML abierto anteriormente.
Línea 119	<textField> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 120	<reportElement x="150" y="3" width="70" height="15" uuid="44444444-4444-4444-8444-444444444442"/> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 121	<textElement verticalAlignment="Middle"></textElement> → Configura alineación y propiedades del contenido textual.
Línea 122	<textFieldExpression><![CDATA[\${REPORT_COUNT}]]></textFieldExpression> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 123	</textField> → Cierra el elemento XML abierto anteriormente.
Línea 124	<textField> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 125	<reportElement x="170" y="23" width="190" height="15" uuid="44444444-4444-4444-8444-444444444443"/> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 126	<textElement textAlignment="Right" verticalAlignment="Middle"></textElement> → Configura alineación y propiedades del contenido textual.
Línea 127	<textFieldExpression><![CDATA["Página " + \${PAGE_NUMBER} + " de"]]></textFieldExpression> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 128	</textField> → Cierra el elemento XML abierto anteriormente.
Línea 129	<textField evaluationTime="Report"> → Abre un campo de texto dinámico; puede incluir patrón o gestión de nulos.
Línea 130	<reportElement x="365" y="23" width="30" height="15" uuid="44444444-4444-4444-8444-444444444444"/> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 131	<textElement textAlignment="Left" verticalAlignment="Middle"></textElement> → Configura alineación y propiedades del contenido textual.
Línea 132	<textFieldExpression><![CDATA[\${PAGE_NUMBER}]]></textFieldExpression> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 133	</textField> → Cierra el elemento XML abierto anteriormente.
Línea 134	</band> → Cierra el elemento XML abierto anteriormente.
Línea 135	</pageFooter> → Cierra el elemento XML abierto anteriormente.
Línea 136	</jasperReport> → Cierra el elemento XML abierto anteriormente.

Comprobación: las coordenadas se mantienen dentro de `columnWidth="555"`, el orden estructural es compatible con JasperReports 6.20.0 y no se usa sintaxis retirada de la baseline.

Parte C — Código Java explicado línea por línea [VALIDADO]

El código siguiente es el código real incluido en el checkpoint y ejecutado por el workflow E2E. Java no redibuja el informe: **compila -> llena -> exporta**.

Clase `GeneradorInformeVentas.java`

JAVA

```
import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeVentas {
```

```

public static void main(String[] args) {
    try {
        String rutaJrxml = "reports/informe_ventas.jrxml";
        String rutaJasper = "reports/informe_ventas.jasper";
        String rutaPdf = "output/informe_ventas.pdf";
        String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
        new File("output").mkdirs();
        JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
        Map<String, Object> parametros = new HashMap<String, Object>();
        try (Connection conexion = DriverManager.getConnection(urlBD)) {
            JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);
            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);
            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
        }
    } catch (Exception e) {
        e.printStackTrace();
        System.exit(1);
    }
}
}

```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 2	<code>import java.sql.Connection;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 3	<code>import java.sql.DriverManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 4	<code>import java.util.HashMap;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 5	<code>import java.util.Map;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 6	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 7	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 8	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 9	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 10	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 11	<code>public class GeneradorInformeVentas {</code> → Declara la clase Java del checkpoint.
Línea 12	<code>public static void main(String[] args) {</code> → Declara el punto de entrada ejecutable.
Línea 13	<code>try {</code> → Abre un bloque protegido; si contiene recursos, se cerrarán automáticamente.
Línea 14	<code>String rutaJrxml = "reports/informe_ventas.jrxml";</code> → Fija la ruta del JRXML desde el Working Directory EditorialReports.
Línea 15	<code>String rutaJasper = "reports/informe_ventas.jasper";</code> → Fija la ruta del artefacto .jasper compilado.
Línea 16	<code>String rutaPdf = "output/informe_ventas.pdf";</code> → Fija la ruta del PDF que se exportará.
Línea 17	<code>String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Define la URL JDBC hacia EditorialReportsJava/data/editorial.db.
Línea 18	<code>new File("output").mkdirs();</code> → Crea el directorio necesario antes de escribir datos o salidas.
Línea 19	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML a .jasper con JasperReports 6.20.0.
Línea 20	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa de parámetros que se entrega al motor de llenado.
Línea 21	<code>try (Connection conexion = DriverManager.getConnection(urlBD)) {</code> → Define la URL JDBC hacia EditorialReportsJava/data/editorial.db.

Línea 22	JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion); → Llena el informe y obtiene un JasperPrint en memoria.
Línea 23	JasperExportManager.exportReportToPdfFile(documento, rutaPdf); → Exporta el JasperPrint a PDF.
Línea 24	System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath()); → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 25	System.out.println("Paginas del documento: " + documento.getPages().size()); → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 26	} → Cierra el bloque Java actual.
Línea 27	} catch (Exception e) { → Captura cualquier fallo de compilación, datos, llenado o exportación.
Línea 28	e.printStackTrace(); → Imprime la traza completa para facilitar el diagnóstico.
Línea 29	System.exit(1); → Termina con código distinto de cero para que CI detecte el fallo.
Línea 30	} → Cierra el bloque Java actual.
Línea 31	} → Cierra el bloque Java actual.
Línea 32	} → Cierra el bloque Java actual.

Criterio de fallo: todo catch termina con `System.exit(1)` para que una excepción no pueda aparecer como ejecución verde en CI.

Parte D — Simulación del PDF esperado y de la estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT

informe_ventas.jrxml
[Design] [Source]

Ruler: 0 100 200 300 400 500 555

Title
h = 60

Informe de Ventas - Agregación por Título

Column Header
h = 25

Título | Unidades | Importe total | Precio med.

Detail 1
h = 20

[\${F{titulo}}] [\${F{unidades}}] [\${F{importe}}] [\${F{medio}}]

Page Footer
h = 60

Total de títulos: [\${V{REPORT_COUNT}}]
"Página " + \${V{PAGE_NUMBER}} + " de " + [segundo campo PAGE_NUMBER con evaluationTime=Report]

Panel Outline muestra:
Properties
└ com.jaspersoft.studio.data.defaultdataadapter = SQLiteEditorial
QueryString
└ SELECT l.titulo, SUM(v.cantidad) AS unidades_vendidas, ...
Fields
└ titulo [java.lang.String]
└ unidades_vendidas [java.lang.Integer]
└ importe_total [java.lang.Double]
└ precio_medio [java.lang.Double]

Qué representa: la disposición del informe en el editor tras completar los quince pasos. El panel Outline muestra la consulta SQL con JOIN y agregación, y los cuatro campos declarados.

Cómo verificarlo: comparar la vista del editor con este esquema. El panel Outline debe mostrar el nodo QueryString con la consulta SQL.

D.2 — Jerarquía del Outline

TEXT

```

informe_ventas
├── Properties
│   └── com.jaspersoft.studio.data.defaultdataadapter = SQLiteEditorial
├── Styles
│   └── Sans_Normal [isDefault=true]
├── QueryString
│   └── SELECT l.titulo, SUM(v.cantidad) AS unidades_vendidas,
│       SUM(v.cantidad * v.precio_unitario) AS importe_total,
│       AVG(v.precio_unitario) AS precio_medio
│       FROM libros l
│       INNER JOIN ventas v ON l.titulo = v.titulo_libro
│       GROUP BY l.titulo
│       ORDER BY importe_total DESC
├── Fields
│   ├── titulo [java.lang.String]
│   ├── unidades_vendidas [java.lang.Integer]
│   ├── importe_total [java.lang.Double]
│   └── precio_medio [java.lang.Double]
├── Title [band, height=60]
│   └── staticText "Informe de Ventas - Agregación por Título"
├── Column Header [band, height=25]
│   ├── staticText "Título" (bold)
│   ├── staticText "Unidades" (bold, right)
│   ├── staticText "Importe total" (bold, right)
│   └── staticText "Precio medio" (bold, right)
├── Detail 1 [band, height=20, splitType=Stretch]
│   ├── textField [textAdjust=StretchHeight] ${titulo}
│   ├── textField [right] ${unidades_vendidas}
│   ├── textField [pattern=#,##0.00 €, right] ${importe_total}
│   └── textField [pattern=#,##0.00 €, right] ${precio_medio}
├── Page Footer [band, height=60]
│   ├── staticText "Total de títulos: "
│   ├── textField [bold] ${REPORT_COUNT}
│   └── textField [center] "Página " + ${PAGE_NUMBER} + " de " + [segundo campo PAGE_NUMBER con evaluationTime=Report]
└── Background [band, height=0]

```

Qué representa: el árbol de nodos del informe tal como aparece en el panel Outline tras completar los quince pasos. La novedad respecto al punto 3.4 es la consulta SQL con JOIN y agregación.

Cómo verificarlo: expandir el nodo informe_ventas en el panel Outline y comparar la estructura.

D.3 — Documento PDF resultante, página por página

TEXT

```

INFORME: informe_ventas.pdf
PÁGINAS TOTALES: 1
TAMAÑO DE PÁGINA: 595 × 842 píxeles (A4 vertical)
ORIGEN DE DATOS: jdbc:sqlite:../EditorialReportsJava/data/editorial.db
CONSULTA: SELECT ... INNER JOIN ventas ... GROUP BY l.titulo
REGISTROS OBTENIDOS: 7
BANDAS EMITIDAS: Title, Column Header, Detail (7 veces),
                 Page Footer, Background

```

Página 1 de 1

Informe de Ventas - Agregación por Título			
Título	Unid.	Importe total	Precio
Cien años de soledad	8	159,60 €	19,95 €
Rayuela	6	135,00 €	22,50 €
La casa de los espíritus	5	117,00 €	23,40 €
Pedro Páramo	6	95,40 €	15,90 €
Ficciones	3	63,00 €	21,00 €
Comala	2	38,40 €	19,20 €
Paradiso	1	25,00 €	25,00 €
Total de títulos: 7			
Página 1 de 1			

Qué representa: la página única del PDF resultante con los siete libros que tienen ventas registradas. Los libros aparecen ordenados por importe total descendente. Cada fila muestra el título, las unidades vendidas, el importe total y el precio medio.

Cómo verificarlo: abrir el archivo `output/informe_ventas.pdf` con un lector de PDF y comprobar que aparecen los siete libros ordenados por importe total. Si el orden es distinto, revisar la cláusula `ORDER BY`.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├── ECOSISTEMA.md, ENTORNO.md, BANDAS.md, JRXML.md
├── TEXTO.md, CAMPOS.md, IMAGENES.md, ESTILOS.md, EXPRESIONES.md
├── BASEDATOS.md, CSV.md, XML.md, JSON.md, CONSULTAS.md      (nuevo)
├── data/
│   ├── catalogo.csv
│   ├── distribucion.xml
│   └── autores.json
├── reports/
│   ├── informe_concepto.jrxml
│   ├── informe_catalogo_csv.jrxml
│   ├── informe_distribucion_xml.jrxml
│   ├── informe_autores_json.jrxml
│   └── informe_ventas.jrxml                                (nuevo)
├── resources/
│   └── (logotipo, iconos y portadas)
└── output/
    ├── informe_concepto.pdf
    ├── informe_catalogo_csv.pdf
    ├── informe_distribucion_xml.pdf
    ├── informe_autores_json.pdf
    └── informe_ventas.pdf                                  (nuevo)

EditorialReportsJava/
├── lib/
│   └── (6 JAR de JasperReports y dependencias)
├── data/
│   └── editorial.db                                        (con tabla ventas)
└── src/
    ├── GeneradorInformeConcepto.java
    ├── GeneradorCatalogoCSV.java
    ├── GeneradorDistribucionXML.java
    ├── GeneradorAutoresJSON.java
    ├── GeneradorInformeVentas.java                       (nueva clase)
    ├── Libro.java
    ├── CatalogoDataSource.java
    └── InicializadorBD.java                               (ampliada con tabla ventas)

```

Qué representa: el estado de los dos proyectos tras completar los quince pasos. La novedad respecto al punto 3.4 es la ampliación del `InicializadorBD` con la tabla ventas, el informe `informe_ventas.jrxml`, la clase `GeneradorInformeVentas` y el archivo `CONSULTAS.md`.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si el archivo `informe_ventas.jrxml` no aparece, repetir el paso 3.

Errores comunes del ejercicio completo

Error	Causa	Solución
SQLException: no such table: ventas	La tabla ventas no existe en la base de datos	Ejecutar de nuevo el <code>InicializadorBD</code>
SQLException: no such column: v.cantidad	El alias de tabla es incorrecto	Verificar que el alias <code>v</code> esté declarado en <code>FROM ventas v</code>
La consulta devuelve una sola fila	Falta la cláusula <code>GROUP BY</code>	Añadir <code>GROUP BY l.titulo</code> antes de <code>ORDER BY</code>
Field not found: unidades_vendidas	El alias de la consulta no coincide con el nombre del campo	Verificar que el alias <code>AS unidades_vendidas</code> coincide con el campo
ClassCastException al resolver importe_total	El campo está declarado como <code>Integer</code> pero la consulta devuelve un <code>Double</code>	Declarar el campo como <code>java.lang.Double</code>

Error	Causa	Solución
SQLException: misuse of aggregate function SUM()	La función SUM se utiliza sin GROUP BY y con columnas no agregadas	Añadir GROUP BY o eliminar las columnas no agregadas
La conexión queda abierta y bloquea la base de datos	No se cerró la conexión después del llenado	Usar un bloque try-with-resources
El informe se previsualiza con cero registros	La tabla ventas está vacía o el JOIN no encuentra coincidencias	Verificar los datos de la tabla ventas
SQLException: no such column: l.titulo_libro	El nombre de la columna de JOIN es incorrecto	Verificar el nombre de la columna en la tabla libros
Los importes se muestran sin decimales	El campo no tiene el patrón #,##0.00 €	Añadir el patrón en el panel Properties del campo
ClassCastException al resolver precio_medio	El campo está declarado como Integer	Declarar el campo como java.lang.Double

Reto resuelto paso a paso

Enunciado: añadir una segunda consulta que devuelva las ventas agrupadas por mes. Crear un segundo informe `informe_ventas_mes.jrxml` con la consulta y ejecutarlo desde Java.

Paso 1. Hacer clic con el botón derecho sobre la carpeta `reports` en el panel Project Explorer.

Paso 2. Hacer clic sobre la opción New en el menú contextual.

Paso 3. Hacer clic sobre la opción Jasper Report en el submenú.

Paso 4. Hacer clic sobre la plantilla Blank A4 y pulsar el botón Next.

Paso 5. Escribir exactamente `informe_ventas_mes` en el campo File name.

Paso 6. Hacer clic sobre el botón Next y seleccionar `SQLiteEditorial` en la lista de adaptadores.

Paso 7. Hacer clic sobre el botón Finish.

Paso 8. Hacer clic sobre la pestaña Source y eliminar el bloque de la consulta SQL existente.

Paso 9. Escribir exactamente `<queryString language="sql">` y pulsar Enter.

Paso 10. Escribir exactamente `<![CDATA[` y pulsar Enter.

Paso 11. Escribir exactamente `SELECT SUBSTR(v.fecha_venta, 1, 7) AS mes,` y pulsar Enter.

Paso 12. Escribir exactamente `COUNT(*) AS num_ventas,` y pulsar Enter.

Paso 13. Escribir exactamente `SUM(v.cantidad) AS unidades,` y pulsar Enter.

Paso 14. Escribir exactamente `SUM(v.cantidad * v.precio_unitario) AS importe` y pulsar Enter.

Paso 15. Escribir exactamente `FROM ventas v` y pulsar Enter.

Paso 16. Escribir exactamente `GROUP BY mes` y pulsar Enter.

Paso 17. Escribir exactamente `ORDER BY mes` y pulsar Enter.

Paso 18. Escribir exactamente `]]>` y pulsar Enter.

Paso 19. Escribir exactamente `</queryString>` y pulsar Enter.

Paso 20. Localizar las declaraciones de campos y sustituirlas por las cuatro nuevas.

Paso 21. Pulsar Ctrl+S para guardar el archivo.

Paso 22. Pulsar Ctrl+Mayús+B para compilar.

Paso 23. Crear la clase `GeneradorVentasMes` siguiendo el mismo patrón que `GeneradorInformeVentas` pero con las rutas del nuevo informe.

Paso 24. Ejecutar la clase con Run As > Java Application.

Paso 25. Abrir el archivo `output/informe_ventas_mes.pdf` y verificar las ventas agrupadas por mes.

Simulación ASCII del PDF tras el reto

TEXT

```

|| Mes      | N° ventas | Unidades | Importe total
||-----|-----|-----|-----
|| 2026-09  | 9         | 31        | 648,40 €

```

Resultado del reto: la consulta `SELECT SUBSTR(v.fecha_venta, 1, 7) AS mes, COUNT(*), SUM(v.cantidad), SUM(v.cantidad * v.precio_unitario) FROM ventas v GROUP BY mes` agrupa las ventas por mes (utilizando los primeros siete caracteres de la fecha como clave de agrupación). El informe muestra el número de ventas, las unidades totales y el importe total del mes. La función `SUBSTR` es específica de SQLite y extrae una subcadena de la fecha.

Analogía final con el contexto de la editorial

Las consultas SQL son las preguntas que el editor hace al archivador de la editorial. La consulta básica recupera las fichas de los libros. La consulta con `JOIN` cruza las fichas de los libros con el registro de ventas. Las funciones de agregación resumen los datos de las ventas por título o por mes. La cláusula `GROUP BY` agrupa las ventas por criterio. La cláusula `ORDER BY` ordena los resultados. Los alias de columna permiten dar nombres descriptivos a las columnas calculadas. El resultado es un informe que no solo presenta los datos en bruto, sino que los resume y los ordena según el criterio más útil para el lector. Las consultas SQL son la herramienta más potente para obtener información agregada de una base de datos.

Resultado esperado

Al finalizar este punto, el alumno dispone de:

- La tabla `ventas` en la base de datos `editorial.db` con nueve registros de ejemplo.
- El archivo `reports/informe_ventas.jrxml` con la consulta SQL con `JOIN`, `GROUP BY` y funciones de agregación.
- El archivo `output/informe_ventas.pdf` con los siete libros que tienen ventas, ordenados por importe total descendente.
- La clase `GeneradorInformeVentas.java` que consulta la base de datos y genera el informe.
- El archivo `CONSULTAS.md` en la raíz del proyecto con la documentación de las consultas.
- Comprensión operativa de los alias de columna, de las funciones de agregación, de las cláusulas `GROUP BY` y `ORDER BY`, y de los `JOIN`.

Conclusión y enlace al siguiente punto

El punto 3.5 ha introducido las consultas SQL complejas y ha demostrado su uso con una consulta que combina la tabla `libros` con la tabla `ventas` mediante un `INNER JOIN` y calcula agregaciones por título con `SUM` y `AVG`. El informe `informe_ventas.jrxml` se alimenta de la base de datos y muestra los siete libros que tienen ventas registradas, ordenados por importe total descendente.

Punto 3.6 — Fields

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.6 — Fields

Objetivos de aprendizaje

- Comprender el campo como contrato entre la consulta SQL y la plantilla JRXML.
- Establecer la correspondencia entre los tipos SQL y los tipos Java de los campos.
- Gestionar valores nulos en los campos mediante `isBlankWhenNull` y expresiones condicionales.
- Depurar errores de resolución de campos con las herramientas del entorno.
- Aplicar buenas prácticas en la declaración de campos de un informe profesional.
- Documentar los campos del informe de ventas en el proyecto EditorialReports.

Parte práctica

Parte A — Práctica visual

Paso 1: Abrir el informe de ventas

Acciones:

1. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción Refresh en el menú contextual.
3. Hacer doble clic sobre el archivo `informe_ventas.jrxml` en el panel Project Explorer.
4. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
5. Expandir el nodo `informe_ventas` en el panel Outline (inferior izquierdo).
6. Hacer clic sobre el nodo Fields en el panel Outline y verificar que aparecen los cuatro campos.

Verificación visual: el editor central muestra el informe de ventas con las cuatro bandas. El panel Outline muestra el nodo Fields con los campos `titulo`, `unidades_vendidas`, `importe_total` y `precio_medio`.

Qué hace: abre el informe de ventas y localiza la sección de campos.

Por qué: los campos del informe de ventas son el punto de partida para las modificaciones de este punto.

Error común: abrir el archivo en la vista Source en lugar de Design.

Solución: hacer clic sobre la pestaña Design.

Analogía: es como abrir el resumen de ventas del catálogo para revisar los campos que contiene.

Paso 2: Ampliar la consulta con nuevas columnas

Acciones:

1. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
2. Localizar la línea que contiene `SUM(v.cantidad * v.precio_unitario) AS importe_total`, y pulsar Enter al final.
3. Escribir exactamente `MAX(v.fecha_venta) AS ultima_venta`, y pulsar Enter.
4. Escribir exactamente `MIN(v.fecha_venta) AS primera_venta`, y pulsar Enter.
5. Pulsar Ctrl+S para guardar el archivo.
6. Hacer clic sobre la pestaña Design en la parte inferior del editor central.

Verificación visual: la vista Source muestra las dos nuevas columnas calculadas en la consulta SQL. El panel Outline no muestra todavía los nuevos campos porque no se han declarado.

Qué hace: amplía la consulta SQL con dos columnas calculadas que devuelven la fecha de la última y de la primera venta de cada libro.

Por qué: las nuevas columnas aportan información sobre el periodo de ventas de cada libro.

Error común: olvidar la coma al final de la línea anterior. El compilador SQL lanza un error de sintaxis.

Solución: revisar que cada columna excepto la última termina con coma.

Analogía: es como añadir al resumen de ventas las fechas de la primera y la última venta.

Paso 3: Declarar los nuevos campos

Acciones:

1. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
2. Localizar la línea que contiene `<field name="precio_medio" class="java.lang.Double"/>` y pulsar Enter al final.
3. Escribir exactamente `<field name="ultima_venta" class="java.lang.String"/>` y pulsar Enter.
4. Escribir exactamente `<field name="primera_venta" class="java.lang.String"/>` y pulsar Enter.
5. Pulsar Ctrl+S para guardar el archivo.
6. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
7. Expandir el nodo Fields en el panel Outline y verificar que aparecen los cinco campos.

Verificación visual: el panel Outline muestra el nodo Fields con seis entradas: titulo, unidades_vendidas, importe_total, precio_medio, ultima_venta y primera_venta.

Qué hace: declara los dos nuevos campos con tipo `java.lang.String` porque la fecha se almacena como texto en la base de datos SQLite.

Por qué: las funciones MAX y MIN sobre una columna TEXT devuelven una cadena.

Error común: declarar los campos como `java.util.Date`. El motor lanza `ClassCastException` porque el `ResultSet` devuelve una cadena.

Solución: declarar los campos como `java.lang.String` y convertir en la expresión si es necesario.

Analogía: es como añadir los dos campos nuevos al listado de datos del resumen de ventas.

Paso 4: Añadir los encabezados de las nuevas columnas

Acciones:

1. Hacer clic sobre el nodo Column Header en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Static Text (una letra T mayúscula).
4. Arrastrar el icono Static Text y soltarlo dentro de la banda Column Header, en la coordenada aproximada x=0, y=25.
5. Hacer clic sobre el campo X en el panel Properties (inferior derecho), pestaña Properties, escribir 0 y pulsar Enter.
6. Hacer clic sobre el campo Y, escribir 25 y pulsar Enter.
7. Hacer clic sobre el campo Width, escribir 150 y pulsar Enter.
8. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
9. Hacer doble clic sobre el Static Text creado en la acción anterior.
10. Escribir exactamente `Primera venta`.
11. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
12. Marcar la casilla Bold.
13. Repetir las acciones 3 a 12 para el encabezado `Última venta` en la coordenada x=150, y=25, ancho 150.

Verificación visual: la banda Column Header muestra los dos nuevos encabezados *Primera venta* y *Última venta* en negrita, debajo de los encabezados existentes.

Qué hace: inserta los encabezados de las dos nuevas columnas.

Por qué: los encabezados identifican las nuevas columnas en la tabla del informe.

Error común: soltar el elemento fuera de los límites de la banda y provocar que se coloque en otra banda.

Solución: comprobar en el panel Outline que el nodo cuelga de Column Header.

Analogía: es como añadir los títulos de las nuevas columnas al resumen de ventas.

Paso 5: Ampliar la altura de la banda Column Header

Acciones:

1. Hacer clic sobre el nodo Column Header en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties (inferior derecho), pestaña Properties, escribir 45 y pulsar Enter.
3. Verificar que los encabezados existentes y los nuevos siguen visibles sin solaparse.

Verificación visual: la banda Column Header aparece con 45 píxeles de altura. Los cuatro encabezados originales están en la parte superior y los dos nuevos en la parte inferior.

Qué hace: amplía la altura de la banda de cabecera para alojar los dos nuevos encabezados.

Por qué: los encabezados están situados en la coordenada Y=25 y necesitan espacio adicional.

Error común: olvidar ampliar la altura y provocar que los nuevos encabezados se solapen con la banda Detail.

Solución: ajustar la altura a 45 píxeles.

Analogía: es como ampliar la cabecera de la tabla del resumen de ventas para acomodar las nuevas columnas.

Paso 6: Añadir los campos de las nuevas columnas en la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 40 y pulsar Enter.
3. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
4. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).
5. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=0, y=20.
6. Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 0 y pulsar Enter.
7. Hacer clic sobre el campo Y, escribir 20 y pulsar Enter.
8. Hacer clic sobre el campo Width, escribir 150 y pulsar Enter.
9. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
10. Hacer clic sobre el campo Text Field Expression y escribir exactamente `$F{primera_venta}` y pulsar Enter.
11. Hacer clic sobre el campo Font size y escribir 9. Pulsar Enter.
12. Dejar el campo Pattern vacío: `primera_venta` y `ultima_venta` son `String` ISO y no deben recibir un patrón de fecha de Date.
13. Repetir las acciones 4 a 12 para el campo `ultima_venta` en la coordenada x=150, y=20, ancho 150.

Verificación visual: la banda Detail 1 muestra los dos nuevos campos con las expresiones correspondientes debajo de los campos existentes.

Qué hace: inserta los dos campos que muestran la primera y la última fecha de venta de cada libro.

Por qué: los campos resuelven las columnas calculadas `MAX(v.fecha_venta)` y `MIN(v.fecha_venta)`.

Error común: aplicar un patrón de fecha a un campo `String`.

Solución: mostrar el texto ISO directamente o convertir explícitamente a `Date` antes de aplicar un patrón.

Analogía: es como rellenar las celdas de las nuevas columnas con las fechas de la primera y la última venta.

Paso 7: Añadir un campo con expresión condicional para el precio medio

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el Text Field que contiene la expresión `$F{precio_medio}` en el editor central.
3. Hacer clic sobre el campo Text Field Expression en el panel Properties, pestaña Properties.
4. Seleccionar el contenido actual del campo y eliminarlo con la tecla Suprimir.
5. Escribir exactamente `$F{precio_medio} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€' ").format($F{precio_medio})` y pulsar Enter.
6. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
7. Localizar el `<textField>` del precio medio y verificar que la expresión contiene el operador ternario.
8. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: la vista Source muestra la expresión condicional en el campo del precio medio.

Qué hace: modifica la expresión del precio medio para mostrar Sin datos cuando el valor es nulo.

Por qué: la expresión evita que el campo quede vacío cuando la media no se puede calcular.

Error común: olvidar el paréntesis en la comprobación de nulo.

Solución: escribir `$F{precio_medio} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€' ").format($F{precio_medio})` con el operador ternario completo.

Analogía: es como indicar Sin datos en las celdas del resumen de ventas donde no hay información.

Paso 8: Añadir la propiedad `isBlankWhenNull` al campo de unidades

Acciones:

1. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
2. Hacer clic sobre el Text Field que contiene la expresión `$F{unidades_vendidas}` en el editor central.
3. Expandir la sección Text Field en el panel Properties (inferior derecho), pestaña Properties.
4. Marcar la casilla Blank When Null.
5. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
6. Localizar el `<textField>` del campo de unidades y verificar que contiene el atributo `isBlankWhenNull="true"`.
7. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: la vista Source muestra el atributo `isBlankWhenNull="true"` en el campo de unidades.

Qué hace: activa la propiedad que hace que el campo se muestre vacío cuando el valor es nulo.

Por qué: las unidades vendidas pueden ser nulas si un libro no tiene ventas registradas.

Error común: dejar la propiedad desactivada y provocar que el campo muestre el texto `null`.

Solución: marcar la casilla Blank When Null.

Analogía: es como dejar en blanco las celdas del resumen de ventas donde no hay datos en lugar de imprimir un error.

Paso 9: Añadir la propiedad isBlankWhenNull al campo de importe total**Acciones:**

1. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
2. Hacer clic sobre el Text Field que contiene la expresión `$F{importe_total}` en el editor central.
3. Expandir la sección Text Field en el panel Properties (inferior derecho), pestaña Properties.
4. Marcar la casilla Blank When Null.
5. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el campo del importe total tiene la casilla Blank When Null marcada en el panel Properties.

Qué hace: activa la propiedad que hace que el campo se muestre vacío cuando el valor es nulo.

Por qué: el importe total puede ser nulo si un libro no tiene ventas registradas.

Error común: olvidar la propiedad y provocar que el motor intente aplicar el patrón a un valor nulo.

Solución: marcar la casilla Blank When Null.

Analogía: es como dejar en blanco las celdas del importe total donde no hay datos.

Paso 10: Añadir el campo de la primera venta al Column Header**Acciones:**

1. Hacer clic sobre el nodo Column Header en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Static Text (una letra T mayúscula).
4. Arrastrar el icono Static Text y soltarlo dentro de la banda Column Header, en la coordenada aproximada x=300, y=25.
5. Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 300 y pulsar Enter.
6. Hacer clic sobre el campo Y, escribir 25 y pulsar Enter.
7. Hacer clic sobre el campo Width, escribir 150 y pulsar Enter.
8. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
9. Hacer doble clic sobre el Static Text creado en la acción anterior.
10. Escribir exactamente `Periodo de ventas`.
11. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
12. Marcar la casilla Bold.

Verificación visual: la banda Column Header muestra el nuevo encabezado `Periodo de ventas` en la coordenada 300.

Qué hace: inserta un encabezado para el campo que combina la primera y la última fecha.

Por qué: el encabezado identifica la columna que muestra el periodo de ventas.

Error común: olvidar marcar la casilla Bold.

Solución: marcar la casilla Bold en el panel Properties.

Analogía: es como añadir el título de la columna del periodo de ventas al resumen.

Paso 11: Añadir el campo del periodo de ventas en la banda Detail**Acciones:**

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
3. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).
4. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=300, y=20.

5. Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 300 y pulsar Enter.
6. Hacer clic sobre el campo Y, escribir 20 y pulsar Enter.
7. Hacer clic sobre el campo Width, escribir 150 y pulsar Enter.
8. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
9. Hacer clic sobre el campo Text Field Expression y escribir exactamente `$F{primera_venta} + " → " + $F{ultima_venta}` y pulsar Enter.
10. Hacer clic sobre el campo Font size y escribir 9. Pulsar Enter.
11. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.

Verificación visual: la banda Detail 1 muestra el nuevo campo con la expresión que concatena la primera y la última fecha.

Qué hace: inserta un campo que muestra el periodo de ventas como un rango de fechas.

Por qué: el rango de fechas resume el periodo durante el cual el libro se ha vendido.

Error común: usar un guion simple en lugar de la flecha.

Solución: usar el carácter → o la secuencia " a " para separar las fechas.

Analogía: es como mostrar el periodo de ventas de cada libro en el resumen.

Paso 12: Ampliar la altura de la banda Detail

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 40 y pulsar Enter.
3. Verificar que los campos existentes y los nuevos siguen visibles sin solaparse.

Verificación visual: la banda Detail 1 aparece con 40 píxeles de altura. Los campos de la primera fila están en la parte superior y los de la segunda fila en la parte inferior.

Qué hace: amplía la altura de la banda de detalle para alojar los nuevos campos.

Por qué: los nuevos campos están situados en la coordenada Y=20 y necesitan espacio adicional.

Error común: olvidar ampliar la altura y provocar que los nuevos campos se solapen con la banda siguiente.

Solución: ajustar la altura a 40 píxeles.

Analogía: es como ampliar las filas del resumen de ventas para acomodar las nuevas columnas.

Paso 13: Compilar y previsualizar el informe

Acciones:

1. Pulsar Ctrl+S para guardar el archivo.
2. Pulsar Ctrl+Mayús+B para compilar el informe.
3. Hacer clic sobre el panel Problems (inferior) y verificar que no hay errores.
4. Pulsar el botón Preview de la barra de herramientas superior.
5. En el diálogo, verificar que el adaptador SQLiteEditorial está seleccionado.
6. Hacer clic sobre el botón OK.
7. Esperar a que se abra la pestaña Preview en el editor central.

Verificación visual: la pestaña Preview muestra el informe con las nuevas columnas: primera venta, última venta y periodo de ventas. El campo del precio medio muestra el valor o Sin datos según el caso.

Qué hace: compila y previsualiza el informe con los nuevos campos.

Por qué: la previsualización confirma que los nuevos campos se resuelven correctamente y que la propiedad `isBlankWhenNull` funciona.

Error común: obtener `Field not found: primera_venta`. Indica que el campo no está declarado o el alias de la consulta no coincide.

Solución: revisar la declaración del campo y el alias de la consulta.

Analogía: es como revisar la prueba de color del resumen de ventas con las nuevas columnas.

Paso 14: Crear el archivo CAMPOS.md con la documentación de los campos

Acciones:

1. Hacer clic con el botón derecho sobre el nodo `EditorialReports` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción `New` en el menú contextual.
3. Hacer clic sobre la opción `File` en el submenú.
4. Escribir exactamente `CAMPOS_VENTAS.md` en el campo `File name` del diálogo.
5. Hacer clic sobre el botón `Finish`.
6. En el editor central, escribir exactamente `# Campos del informe de ventas` y pulsar `Enter` dos veces.
7. Escribir exactamente `| Campo | Tipo Java | Tipo SQL | Origen |` y pulsar `Enter`.
8. Escribir exactamente `---|---|---|---` y pulsar `Enter`.
9. Escribir exactamente `| titulo | java.lang.String | TEXT | l.titulo |` y pulsar `Enter`.
10. Escribir exactamente `| unidades_vendidas | java.lang.Integer | INTEGER | SUM(v.cantidad) |` y pulsar `Enter`.
11. Escribir exactamente `| importe_total | java.lang.Double | REAL | SUM(v.cantidad * v.precio_unitario) |` y pulsar `Enter`.
12. Escribir exactamente `| precio_medio | java.lang.Double | REAL | AVG(v.precio_unitario) |` y pulsar `Enter`.
13. Escribir exactamente `| primera_venta | java.lang.String | TEXT | MIN(v.fecha_venta) |` y pulsar `Enter`.
14. Escribir exactamente `| ultima_venta | java.lang.String | TEXT | MAX(v.fecha_venta) |` y pulsar `Enter`.
15. Pulsar `Ctrl+S` para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo `CAMPOS_VENTAS.md` en la raíz del proyecto `EditorialReports` con la tabla de campos documentada.

Qué hace: incorpora al proyecto un documento que registra los campos del informe de ventas con sus tipos y su origen.

Por qué: la documentación de los campos facilita el mantenimiento y la incorporación de nuevos desarrolladores.

Error común: olvidar la barra vertical al final de cada línea de la tabla Markdown.

Solución: revisar cada línea.

Analogía: es como dejar en la editorial una ficha técnica con los campos utilizados en el resumen de ventas.

Paso 15: Ejecutar el programa Java y verificar el PDF

Acciones:

1. Hacer clic con el botón derecho sobre el archivo `GeneradorInformeVentas.java` en el panel Project Explorer.
2. Hacer clic sobre la opción `Run As` en el menú contextual.
3. Hacer clic sobre la opción `Java Application` en el submenú.
4. Hacer clic sobre la vista `Console` en el panel inferior y observar el resultado.
5. Abrir el explorador de archivos del sistema operativo.
6. Navegar hasta la carpeta `output` del proyecto `EditorialReports`.
7. Hacer doble clic sobre el archivo `informe_ventas.pdf`.
8. Verificar que el PDF muestra las nuevas columnas: primera venta, última venta y periodo de ventas.

Verificación visual: la vista Console muestra la línea Informe generado en: ... con la ruta absoluta del PDF. El archivo PDF muestra las nuevas columnas en la banda Detail.

Qué hace: ejecuta el programa Java que consulta la base de datos y genera el informe con los nuevos campos.

Por qué: la ejecución confirma que los nuevos campos se resuelven correctamente desde el programa Java.

Error común: ejecutar el programa sin haber compilado el informe previamente y obtener un PDF con la versión anterior.

Solución: pulsar Ctrl+Mayús+B antes de ejecutar.

Analogía: es como imprimir el resumen de ventas con las nuevas columnas.

Parte B — JRXML completo explicado línea por línea [VALIDADO]

Se reproducen las secciones modificadas: consulta SQL, campos, Column Header y Detail.

```
<queryString language="sql"><![CDATA[
    SELECT l.titulo,
           SUM(v.cantidad) AS unidades_vendidas,
           SUM(v.cantidad * v.precio_unitario) AS importe_total,
           AVG(v.precio_unitario) AS precio_medio,
           MIN(v.fecha_venta) AS primera_venta,
           MAX(v.fecha_venta) AS ultima_venta
    FROM libros l
    LEFT JOIN ventas v ON l.titulo = v.titulo_libro
    GROUP BY l.titulo
    ORDER BY COALESCE(importe_total, 0) DESC, l.titulo
]]></queryString>
<field name="titulo" class="java.lang.String"/>
<field name="unidades_vendidas" class="java.lang.Integer"/>
<field name="importe_total" class="java.lang.Double"/>
<field name="precio_medio" class="java.lang.Double"/>
<field name="primera_venta" class="java.lang.String"/>
<field name="ultima_venta" class="java.lang.String"/>
<background>
  <band height="0"/>
</background>

<title>
  <band height="60">
    <staticText>
      <reportElement x="0" y="15" width="555" height="30" uuid="f7c8d9e0-a1b2-3c4d-5e6f-7a8b9c0d1e2f"/>
      <textElement textAlignment="Center" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="18" isBold="true"/>
      </textElement>
      <text><![CDATA[Informe de Ventas - Agregación por Título]]></text>
    </staticText>
  </band>
</title>
<columnHeader><band height="45"><staticText><reportElement x="0" y="5" width="250" height="15"/><textElement><font isBold="true"/></textElement><text><![CDATA[Título]]></text></staticText><staticText><reportElement x="250" y="5" width="90" height="15"/><textElement textAlignment="Right"><font isBold="true"/></textElement><text><![CDATA[Unidades]]></text></staticText><staticText><reportElement x="340" y="5" width="130" height="15"/><textElement textAlignment="Right"><font isBold="true"/></textElement><text><![CDATA[Importe total]]></text></staticText><staticText><reportElement x="470" y="5" width="85" height="15"/><textElement textAlignment="Right"><font isBold="true"/></textElement><text><![CDATA[Precio medio]]></text></staticText><staticText><reportElement x="0" y="25" width="150" height="15"/><textElement><font isBold="true"/></textElement><text><![CDATA[Primera venta]]></text></staticText><staticText><reportElement x="150" y="25" width="150" height="15"/><textElement><font isBold="true"/></textElement><text><![CDATA[Última venta]]></text></staticText><staticText><reportElement x="300" y="25" width="255" height="15"/><textElement textAlignment="Center"><font isBold="true"/></textElement><text><![CDATA[Periodo de ventas]]></text></staticText></band></columnHeader>
<detail><band height="42" splitType="Stretch"><textField textAdjust="StretchHeight"><reportElement x="0" y="1" width="250" height="20"/><textFieldExpression><![CDATA[{$f{titulo}]]></textFieldExpression></textField><textField pattern="#0.00 €" isBlankWhenNull="true"><reportElement x="340" y="1" width="130" height="20"/><textElement textAlignment="Right"><font isBold="true"/></textElement><text><![CDATA[{$f{importe_total}]]></textFieldExpression></textField><textField><reportElement x="470" y="1" width="85" height="20"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{$f{precio_medio} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€").format({$f{precio_medio})}]]></textFieldExpression></textField><textField isBlankWhenNull="true"><reportElement x="0" y="22" width="150" height="18"/><textFieldExpression><![CDATA[{$f{primera_venta}]]></textFieldExpression></textField><textField isBlankWhenNull="true"><reportElement x="150" y="22" width="150" height="18"/><textFieldExpression><![CDATA[{$f{ultima_venta}]]></textFieldExpression></textField><textField><reportElement x="300" y="22" width="255" height="18"/><textElement textAlignment="Center"/><textFieldExpression><![CDATA[{$f{primera_venta} == null ? "Sin ventas" : $f{primera_venta} + " -> " + $f{ultima_venta}]]></textFieldExpression></textField></band></detail>
```

XML

Explicación línea por línea

Línea 1	<code><queryString language="sql"><![CDATA[</code> → Abre la consulta del dataset e indica el lenguaje de consulta.
Línea 2	<code>SELECT l.titulo,</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 3	<code>SUM(v.cantidad) AS unidades_vendidas,</code> → Completa la definición declarativa del informe.
Línea 4	<code>SUM(v.cantidad * v.precio_unitario) AS importe_total,</code> → Completa la definición declarativa del informe.
Línea 5	<code>AVG(v.precio_unitario) AS precio_medio,</code> → Completa la definición declarativa del informe.
Línea 6	<code>MIN(v.fecha_venta) AS primera_venta,</code> → Completa la definición declarativa del informe.
Línea 7	<code>MAX(v.fecha_venta) AS ultima_venta</code> → Completa la definición declarativa del informe.
Línea 8	<code>FROM libros l</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 9	<code>LEFT JOIN ventas v ON l.titulo = v.titulo_libro</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 10	<code>GROUP BY l.titulo</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 11	<code>ORDER BY COALESCE(importe_total, 0) DESC, l.titulo</code> → Forma parte de la consulta SQL que selecciona, combina, agrupa u ordena los datos.
Línea 12	<code>]]></queryString></code> → Completa la definición declarativa del informe.
Línea 13	<code><field name="titulo" class="java.lang.String" /></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 14	<code><field name="unidades_vendidas" class="java.lang.Integer" /></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 15	<code><field name="importe_total" class="java.lang.Double" /></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 16	<code><field name="precio_medio" class="java.lang.Double" /></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 17	<code><field name="primera_venta" class="java.lang.String" /></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 18	<code><field name="ultima_venta" class="java.lang.String" /></code> → Declara un campo y su tipo Java; su nombre debe coincidir con el origen o el alias.
Línea 19	<code><background></code> → Abre una sección/banda estructural del informe.
Línea 20	<code><band height="0" /></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 21	<code></background></code> → Cierra el elemento XML abierto anteriormente.
Línea 22	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 23	<code><title></code> → Abre una sección/banda estructural del informe.
Línea 24	<code><band height="60"></code> → Define la altura y, cuando procede, la política de división de la banda.
Línea 25	<code><staticText></code> → Abre un elemento de texto estático.
Línea 26	<code><reportElement x="0" y="15" width="555" height="30" uuid="f7c8d9e0-a1b2-3c4d-5e6f-7a8b9c0d1e2f" /></code> → Define geometría y posición del elemento dentro del ancho útil del informe.
Línea 27	<code><textElement textAlignment="Center" verticalAlignment="Middle"></code> → Configura alineación y propiedades del contenido textual.
Línea 28	<code></code> → Configura tipografía, tamaño y énfasis.

Línea 29	<code></textElement></code> → Cierra el elemento XML abierto anteriormente.
Línea 30	<code><text><![CDATA[Informe de Ventas - Agregación por Título]]></text></code> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.
Línea 31	<code></staticText></code> → Cierra el elemento XML abierto anteriormente.
Línea 32	<code></band></code> → Cierra el elemento XML abierto anteriormente.
Línea 33	<code></title></code> → Cierra el elemento XML abierto anteriormente.
Línea 34	<pre> <columnHeader><band height="45"><staticText><reportElement x="0" y="5" width="250" height="15"/><textElement></textElement><text><![CDATA[Título]]></ text></staticText><staticText><reportElement x="250" y="5" width="90" height="15"/ ><textElement textAlignment="Right"></textElement><text><![CDATA[Unidades]]></text></staticText><staticText><reportElement x="340" y="5" width="130" height="15"/><textElement textAlignment="Right"></textElement><text><![CDATA[Importe total]]></text></staticText><staticText><reportElement x="470" y="5" width="85" height="15"/><textElement textAlignment="Right"></ textElement><text><![CDATA[Precio medio]]></text></staticText><staticText><reportElement x="0" y="25" width="150" height="15"/><textElement></ textElement><text><![CDATA[Primera venta]]></text></staticText><staticText><reportElement x="150" y="25" width="150" height="15"/><textElement></ textElement><text><![CDATA[Última venta]]></text></staticText><staticText><reportElement x="300" y="25" width="255" height="15"/><textElement textAlignment="Center"></textElement><text><![CDATA[Periodo de ventas]]></text></staticText></band></ columnHeader><detail><band height="42" splitType="Stretch"><textField textAdjust="StretchHeight"><reportElement x="0" y="1" width="250" height="20"/ ><textFieldExpression><![CDATA[\${titulo}]]></textFieldExpression></textField><textField isBlankWhenNull="true"><reportElement x="250" y="1" width="90" height="20"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${unidades_vendidas}]]></ textFieldExpression></textField><textField pattern="#0.00 €" isBlankWhenNull="true"><reportElement x="340" y="1" width="130" height="20"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${importe_total}]]></ textFieldExpression></textField><textField><reportElement x="470" y="1" width="85" height="20"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[\${precio_medio} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00 '€').format(\${precio_medio})]]></textFieldExpression></textField><textField isBlankWhenNull="true"><reportElement x="0" y="22" width="150" height="18"/ ><textFieldExpression><![CDATA[\${primera_venta}]]></textFieldExpression></ textField><textField isBlankWhenNull="true"><reportElement x="150" y="22" width="150" height="18"/><textFieldExpression><![CDATA[\${ultima_venta}]]></textFieldExpression></ textField><textField><reportElement x="300" y="22" width="255" height="18"/><textElement textAlignment="Center"><textFieldExpression><![CDATA[\${primera_venta} == null ? "Sin ventas" : \${primera_venta} + " -> " + \${ultima_venta}]]></textFieldExpression></ textField></band></detail></pre> → Protege una consulta o expresión para que XML no interprete sus caracteres especiales.

Comprobación: las coordenadas se mantienen dentro de `columnWidth="555"`, el orden estructural es compatible con JasperReports 6.20.0 y no se usa sintaxis retirada de la baseline.

Parte C — Código Java explicado línea por línea [VALIDADO]

El código siguiente es el código real incluido en el checkpoint y ejecutado por el workflow E2E. Java no redibuja el informe: **compila -> llena -> exporta**.

Clase `GeneradorInformeVentas.java`

JAVA

```

import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.util.HashMap;
import java.util.Map;
import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeVentas {
    public static void main(String[] args) {
        try {

```

```

String rutaJrxml = "reports/informe_ventas.jrxml";
String rutaJasper = "reports/informe_ventas.jasper";
String rutaPdf = "output/informe_ventas.pdf";
String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";
new File("output").mkdirs();
JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);
Map<String, Object> parametros = new HashMap<String, Object>();
try (Connection conexion = DriverManager.getConnection(urlBD)) {
    JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion);
    JasperExportManager.exportReportToPdfFile(documento, rutaPdf);
    System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
    System.out.println("Paginas del documento: " + documento.getPages().size());
}
} catch (Exception e) {
    e.printStackTrace();
    System.exit(1);
}
}
}

```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 2	<code>import java.sql.Connection;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 3	<code>import java.sql.DriverManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 4	<code>import java.util.HashMap;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 5	<code>import java.util.Map;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 6	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 7	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 8	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 9	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → Importa una clase necesaria para compilar o ejecutar el generador.
Línea 10	<code>``</code> → Línea en blanco usada para separar bloques lógicos y mejorar la legibilidad.
Línea 11	<code>public class GeneradorInformeVentas {</code> → Declara la clase Java del checkpoint.
Línea 12	<code>public static void main(String[] args) {</code> → Declara el punto de entrada ejecutable.
Línea 13	<code>try {</code> → Abre un bloque protegido; si contiene recursos, se cerrarán automáticamente.
Línea 14	<code>String rutaJrxml = "reports/informe_ventas.jrxml";</code> → Fija la ruta del JRXML desde el Working Directory EditorialReports.
Línea 15	<code>String rutaJasper = "reports/informe_ventas.jasper";</code> → Fija la ruta del artefacto <code>.jasper</code> compilado.
Línea 16	<code>String rutaPdf = "output/informe_ventas.pdf";</code> → Fija la ruta del PDF que se exportará.
Línea 17	<code>String urlBD = "jdbc:sqlite:../EditorialReportsJava/data/editorial.db";</code> → Define la URL JDBC hacia EditorialReportsJava/data/editorial.db.
Línea 18	<code>new File("output").mkdirs();</code> → Crea el directorio necesario antes de escribir datos o salidas.
Línea 19	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → Compila el JRXML a <code>.jasper</code> con JasperReports 6.20.0.
Línea 20	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → Crea el mapa de parámetros que se entrega al motor de llenado.
Línea 21	<code>try (Connection conexion = DriverManager.getConnection(urlBD)) {</code> → Define la URL JDBC hacia EditorialReportsJava/data/editorial.db.

Línea 22	JasperPrint documento = JasperFillManager.fillReport(rutaJasper, parametros, conexion); → Llena el informe y obtiene un JasperPrint en memoria.
Línea 23	JasperExportManager.exportReportToPdfFile(documento, rutaPdf); → Exporta el JasperPrint a PDF.
Línea 24	System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath()); → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 25	System.out.println("Paginas del documento: " + documento.getPages().size()); → Emite evidencia de ejecución para el usuario y GitHub Actions.
Línea 26	} → Cierra el bloque Java actual.
Línea 27	} catch (Exception e) { → Captura cualquier fallo de compilación, datos, llenado o exportación.
Línea 28	e.printStackTrace(); → Imprime la traza completa para facilitar el diagnóstico.
Línea 29	System.exit(1); → Termina con código distinto de cero para que CI detecte el fallo.
Línea 30	} → Cierra el bloque Java actual.
Línea 31	} → Cierra el bloque Java actual.
Línea 32	} → Cierra el bloque Java actual.

Criterio de fallo: todo catch termina con `System.exit(1)` para que una excepción no pueda aparecer como ejecución verde en CI.

Parte D — Simulación del PDF esperado y de la estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT

informe_ventas.jrxml
[Design] [Source]

Ruler: 0 100 200 300 400 500 555

Title
h = 60

Informe de Ventas - Agregación por Título

Column Header
h = 45

Título	Unid.	Importe total	Precio med.
Primera venta	Última venta	Periodo de ventas	

Detail 1
h = 40

[\${F{titulo}}] [\${F{unid.}}] [\${F{importe}}] [\${F{medio}}]

[\${F{primera}}] [\${F{ultima}}] [\${F{primera}} - \${F{ultima}}]

Page Footer
h = 60

Total de títulos: [\${V{REPORT_COUNT}}]

"Página " + \${V{PAGE_NUMBER}} + " de " + [segundo campo PAGE_NUMBER con evaluationTime=Report]

Panel Outline muestra:

Fields

titulo	[java.lang.String]
unidades_vendidas	[java.lang.Integer]
importe_total	[java.lang.Double]
precio_medio	[java.lang.Double]
ultima_venta	[java.lang.String]
primera_venta	[java.lang.String]

Qué representa: la disposición del informe en el editor tras completar los quince pasos. La banda Column Header tiene dos filas de encabezados y la banda Detail tiene dos filas de campos.

Cómo verificarlo: comparar la vista del editor con este esquema. La banda Column Header debe tener 45 píxeles de altura y la banda Detail 40 píxeles.

D.2 — Jerarquía del Outline

TEXT

```

informe_ventas
├── Properties
│   └── com.jaspersoft.studio.data.defaultdataadapter = SQLiteEditorial
├── Styles
│   └── Sans_Normal [isDefault=true]
├── QueryString
│   └── SELECT l.titulo, SUM(v.cantidad) AS unidades_vendidas,
│       SUM(v.cantidad * v.precio_unitario) AS importe_total,
│       AVG(v.precio_unitario) AS precio_medio,
│       MAX(v.fecha_venta) AS ultima_venta,
│       MIN(v.fecha_venta) AS primera_venta
│       FROM libros l
│       LEFT JOIN ventas v ON l.titulo = v.titulo_libro
│       GROUP BY l.titulo
│       ORDER BY COALESCE(importe_total, 0) DESC, l.titulo
├── Fields
│   ├── titulo [java.lang.String]
│   ├── unidades_vendidas [java.lang.Integer]
│   ├── importe_total [java.lang.Double]
│   ├── precio_medio [java.lang.Double]
│   ├── ultima_venta [java.lang.String]
│   └── primera_venta [java.lang.String]
├── Title [band, height=60]
│   └── staticText "Informe de Ventas - Agregación por Título"
├── Column Header [band, height=45]
│   ├── staticText "Título" (bold)
│   ├── staticText "Unidades" (bold, right)
│   ├── staticText "Importe total" (bold, right)
│   ├── staticText "Precio medio" (bold, right)
│   ├── staticText "Primera venta" (bold)
│   ├── staticText "Última venta" (bold)
│   └── staticText "Periodo de ventas" (bold, center)
├── Detail 1 [band, height=40, splitType=Stretch]
│   ├── textField ${titulo}
│   ├── textField [isBlankWhenNull=true, right] ${unidades_vendidas}
│   ├── textField [pattern=#,##0.00 €, isBlankWhenNull=true, right] ${importe_total}
│   ├── textField [texto formateado, right] ${precio_medio} == null ? "Sin datos" : new java.text.DecimalFormat("#0.00
│   €").format(${precio_medio})
│   ├── textField [size=9] ${primera_venta}
│   ├── textField [size=9] ${ultima_venta}
│   └── textField [size=9, center] ${primera_venta} + " → " + ${ultima_venta}
├── Page Footer [band, height=60]
│   ├── staticText "Total de títulos: "
│   ├── textField [bold] ${REPORT_COUNT}
│   └── textField [center] "Página " + ${PAGE_NUMBER} + " de " + [segundo campo PAGE_NUMBER con evaluationTime=Report]
└── Background [band, height=0]

```

Qué representa: el árbol de nodos del informe tal como aparece en el panel Outline. La novedad respecto al punto 3.5 es la ampliación de los campos, de la banda Column Header y de la banda Detail.

Cómo verificarlo: expandir el nodo `informe_ventas` en el panel Outline y comparar la estructura.

D.3 — Documento PDF resultante, página por página

TEXT

```

INFORME: informe_ventas.pdf
PÁGINAS TOTALES: 1
TAMAÑO DE PÁGINA: 595 × 842 píxeles (A4 vertical)
ORIGEN DE DATOS: jdbc:sqlite:../EditorialReportsJava/data/editorial.db
CAMPOS DECLARADOS: 6
REGISTROS OBTENIDOS: 7
BANDAS EMITIDAS: Title, Column Header, Detail (14 veces),
                  Page Footer, Background

```

— Página 1 de 1 —

Informe de Ventas - Agregación por Título				
Título	Unid.	Importe total	Precio	
Primera venta	Última venta	Periodo de ventas		
Cien años de soledad	8	159,60 €	19,95 €	
2026-09-01	2026-09-05	2026-09-01 → ...		
Rayuela	6	135,00 €	22,50 €	
2026-09-03	2026-09-07	2026-09-03 → ...		

La casa de los espíritus	5	117,00 €	23,40 €
2026-09-08	2026-09-08	2026-09-08 → ...	
...			
Total de títulos: 7			
Página 1 de 1			

Qué representa: la página única del PDF resultante con las nuevas columnas. Cada libro muestra el título, las unidades vendidas, el importe total, el precio medio, la primera fecha de venta, la última fecha de venta y el periodo de ventas.

Cómo verificarlo: abrir el archivo `output/informe_ventas.pdf` con un lector de PDF y comprobar que aparecen las nuevas columnas en la segunda fila de la banda Detail. Si el periodo de ventas no se muestra correctamente, revisar la expresión del campo.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├── ECOSISTEMA.md, ENTORNO.md, BANDAS.md, JRXML.md
├── TEXTO.md, CAMPOS.md, IMAGENES.md, ESTILOS.md, EXPRESIONES.md
├── BASEDATOS.md, CSV.md, XML.md, JSON.md, CONSULTAS.md
├── CAMPOS_VENTAS.md (nuevo)
├── data/
│   ├── catalogo.csv
│   ├── distribucion.xml
│   └── autores.json
├── reports/
│   ├── informe_concepto.jrxml
│   ├── informe_catalogo_csv.jrxml
│   ├── informe_distribucion_xml.jrxml
│   ├── informe_autores_json.jrxml
│   └── informe_ventas.jrxml (ampliado)
├── resources/
│   └── (logotipo, iconos y portadas)
├── output/
│   ├── informe_concepto.pdf
│   ├── informe_catalogo_csv.pdf
│   ├── informe_distribucion_xml.pdf
│   ├── informe_autores_json.pdf
│   └── informe_ventas.pdf (actualizado)
└── EditorialReportsJava/
    ├── lib/
    │   └── (6 JAR de JasperReports y dependencias)
    ├── data/
    │   └── editorial.db
    └── src/
        ├── GeneradorInformeConcepto.java
        ├── GeneradorCatalogoCSV.java
        ├── GeneradorDistribucionXML.java
        ├── GeneradorAutoresJSON.java
        ├── GeneradorInformeVentas.java
        ├── Libro.java
        ├── CatalogoDataSource.java
        └── InicializadorBD.java

```

Qué representa: el estado de los dos proyectos tras completar los quince pasos. La novedad respecto al punto 3.5 es el archivo `CAMPOS_VENTAS.md` y la ampliación del informe `informe_ventas.jrxml` con dos nuevos campos.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si el archivo `CAMPOS_VENTAS.md` no aparece, repetir el paso 14.

Errores comunes del ejercicio completo

Error	Causa	Solución
Field not found: primera_venta	El campo no está declarado o el alias de la consulta no coincide	

Error	Causa	Solución
		Añadir <code><field name="primera_venta" class="java.lang.String"/></code> y verificar el alias <code>MIN(v.fecha_venta) AS primera_venta</code>
ClassCastException al resolver primera_venta	El campo está declarado como <code>java.util.Date</code> pero la consulta devuelve una cadena	Declarar el campo como <code>java.lang.String</code>
La fecha se muestra en formato ISO sin transformación	El campo no tiene patrón o el patrón no coincide con el formato	No aplicar un patrón de Date a un String; mostrar el ISO o convertir explícitamente
El campo de unidades muestra null	La propiedad <code>isBlankWhenNull</code> está desactivada	Marcar la casilla Blank When Null
El campo del precio medio produce un error de formato	El patrón se aplica a un valor nulo	Usar una expresión condicional o activar <code>isBlankWhenNull</code>
Los nuevos encabezados se solapan con la banda Detail	La altura de la banda Column Header es insuficiente	Ampliar la altura a 45 píxeles
Los nuevos campos se solapan con la banda Column Footer	La altura de la banda Detail es insuficiente	Ampliar la altura a 40 píxeles
SQLException: no such column: v.fecha_venta	El nombre de la columna es incorrecto	Verificar el nombre de la columna en la tabla ventas
El periodo de ventas muestra el mismo valor en las dos fechas	La tabla ventas tiene una sola venta por libro	Añadir más ventas a la tabla para que las fechas sean distintas
El panel Outline no muestra los nuevos campos	El archivo no se ha guardado o no se ha refrescado	Pulsar Ctrl+S y refrescar el panel Project Explorer

Reto resuelto paso a paso

Enunciado: añadir un campo calculado que muestre el número de días entre la primera y la última venta de cada libro. El campo debe llamarse `dias_venta` y mostrar el valor calculado.

Paso 1. Hacer doble clic sobre el archivo `informe_ventas.jrxml` en el panel Project Explorer.

Paso 2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.

Paso 3. Localizar la línea que contiene `<field name="primera_venta" class="java.lang.String"/>` y pulsar Enter al final.

Paso 4. Escribir exactamente `<variable name="DiasVenta" class="java.lang.Integer">` y pulsar Enter.

Paso 5. Escribir exactamente `<variableExpression><![CDATA[${F{primera_venta} == null ? null : Integer.valueOf((int) java.time.temporal.ChronoUnit.DAYS.between(java.time.LocalDate.parse(${F{primera_venta}},{F{ultima_venta}}))] ]></variableExpression>` y pulsar Enter.

Paso 6. Escribir exactamente `</variable>` y pulsar Enter.

Paso 7. Pulsar Ctrl+S para guardar el archivo.

Paso 8. Pulsar Ctrl+Mayús+B para compilar el informe.

Paso 9. Hacer clic sobre la pestaña Design en la parte inferior del editor central.

Paso 10. Hacer clic sobre el nodo Detail 1 en el panel Outline.

Paso 11. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 55 y pulsar Enter.

Paso 12. Hacer clic sobre la pestaña Elements en el panel Palette.

Paso 13. Hacer clic sobre el icono Text Field.

Paso 14. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=450, y=35.

Paso 15. Hacer clic sobre el campo X en el panel Properties, escribir 450 y pulsar Enter.

Paso 16. Hacer clic sobre el campo Y, escribir 35 y pulsar Enter.

Paso 17. Hacer clic sobre el campo Width, escribir 105 y pulsar Enter.

Paso 18. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.

Paso 19. Hacer clic sobre el campo Text Field Expression y escribir exactamente `${DiasVenta}` y pulsar Enter.

Paso 20. Hacer clic sobre el campo Font size y escribir 9. Pulsar Enter.

Paso 21. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.

Paso 22. Pulsar Ctrl+S para guardar el archivo.

Paso 23. Pulsar Ctrl+Mayús+B para compilar el informe.

Paso 24. Hacer clic con el botón derecho sobre `GeneradorInformeVentas.java` y seleccionar Run As > Java Application.

Paso 25. Abrir el archivo `output/informe_ventas.pdf` y verificar que cada libro muestra el número de días entre la primera y la última venta.

Simulación ASCII del PDF tras el reto

TEXT

```
|| Título | Unid. | Importe total | Precio | Días ||
|| Cien años de soledad | 8 | 159,60 € | 19,95 € | 4 ||
|| Rayuela | 6 | 135,00 € | 22,50 € | 4 ||
|| La casa de los espíritus | 5 | 117,00 € | 23,40 € | 0 ||
|| ... ||
```

Resultado del reto: la variable `DiasVenta` usa `java.time.LocalDate.parse` y `ChronoUnit.DAYS.between`, disponibles en Java 8. Devuelve `null` cuando no existe una primera venta y, en los libros con ventas, muestra el número de días entre la primera y la última fecha sin introducir una excepción comprobada en la expresión JRXML.

Analogía final con el contexto de la editorial

Los campos son los datos que el editor extrae de la base de datos para el resumen de ventas. Cada campo tiene un nombre y un tipo que determinan cómo se comporta en el informe. El nombre del campo es la clave que el editor utiliza para localizar el dato en el `ResultSet`. El tipo del campo es la etiqueta que indica cómo interpretar el valor. La propiedad `isBlankWhenNull` es la decisión del editor de dejar en blanco las celdas sin datos. La expresión condicional es la decisión de mostrar un texto alternativo. La documentación de los campos es la ficha técnica que el editor guarda para el mantenimiento. La correcta declaración de campos es la base sobre la que se construye un informe que funciona y que se puede mantener a lo largo del tiempo.

Resultado esperado

Al finalizar este punto, el alumno dispone de:

- El archivo `reports/informe_ventas.jrxml` con seis campos declarados y la consulta SQL ampliada con las funciones MAX y MIN.
- El archivo `output/informe_ventas.pdf` con las nuevas columnas: primera venta, última venta y periodo de ventas.
- La propiedad `isBlankWhenNull` aplicada a los campos de unidades e importe total.
- Una expresión condicional en el campo del precio medio para mostrar Sin datos cuando el valor es nulo.
- El archivo `CAMPOS_VENTAS.md` en la raíz del proyecto con la documentación de los campos.
- Comprensión operativa de la correspondencia entre tipos SQL y tipos Java, de la gestión de nulos y de la depuración de errores de resolución de campos.

Conclusión del Módulo 3 y enlace al Módulo 4

El punto 3.6 cierra el Módulo 3 con la profundización en la declaración de campos a partir de consultas SQL. A lo largo de los seis puntos del módulo, el alumno ha conectado el informe a cuatro fuentes de datos distintas: base de datos SQLite mediante JDBC, archivos CSV, archivos XML y archivos JSON. Ha aprendido a configurar los adaptadores en Jaspersoft Studio, a leer las fuentes desde código Java y a combinar datos de varias fuentes. El proyecto `EditorialReports` contiene ahora cinco informes que cubren las fuentes de datos más habituales en el ámbito empresarial.